

Internet Security

Loris Di Dio

May 2026

Indice

0	Introduzione	7
0.1	Proprietà di sicurezza fondamentali	7
0.2	P.I.I. (Personal Identifiable Information)	7
0.3	Specimen (campione)	7
0.4	Protocollo di Trust	7
0.5	BIOS ed UEFI	7
0.5.1	UEFI	7
0.5.2	BIOS	7
0.6	Standard di sicurezza ISO-IEC 27001	8
0.7	Fattore socio-tecnologico	8
0.8	Firewall	8
0.9	Ransomware e Phishing	8
0.9.1	Ransomware	8
0.9.2	Phishing	8
0.10	Dati at rest, in transit ed in RAM	8
0.11	Ransomware: soluzioni	9
0.12	Soluzioni preventive e mitigazione del problema	9
0.13	Keylogger	9
0.14	Pagamenti online	9
0.15	Security VS Usability	9
0.16	Trust Anchor	10
0.17	Aggiornamenti Software	10
0.18	Moneta elettronica	10
0.19	Ambiti di sicurezza informatica	10
0.19.1	Programmazione	10
0.19.2	Sistemi operativi	10
0.19.3	Servizi innovativi:	11
0.20	Database	11
0.21	Regole (Policy)	11
0.22	Buffer Overflow	11
0.23	SQL Injection e SqlMap	11
0.23.1	SQL Injection	11

1	Esempi reali e falsi miti	12
1.1	Trojan Horse	12
1.1.1	Esempio di Trojan Horse	12
1.2	La cartella Download	13
1.3	Variabile PATH	13
1.4	Strumenti Generali di Sicurezza	14
1.5	Cifratura	14
1.6	Limiti dell'Uso di Password	15
1.7	Limiti di un Firewall	15
1.8	Definizione di Sicurezza	15
1.8.1	Rischi base per la sicurezza	16
1.8.2	Rischi digitali per la sicurezza	16
1.9	Il gioco della sicurezza (Attaccare e difendere)	17
1.9.1	Metodologia d'attacco	17
1.9.2	Metodologia di difesa	17
1.10	Port Scanning	18
2	Proprietà, Attacchi e Attaccanti	18
2.1	Pirateria digitale	19
2.2	Proprietà di sicurezza	19
2.3	Definizione di Segretezza (Confidenzialità)	20
2.3.1	Misure di sicurezza	20
2.3.2	Differenza tra codifica e cifratura	20
2.4	Definizione di Autenticazione	21
2.5	Definizione di Integrità	21
2.6	Definizione di Privatezza (Privacy)	22
2.7	Definizione di Anonimato	22
2.8	Definizione di Non-Ripudio	23
2.8.1	Autenticazione, anonimato e non-ripudio	23
2.9	WATA	24
2.10	WATA2	25
2.10.1	Fase 1: Preparazione dell'esame	26
2.10.2	Fase 2: Effettuazione dell'esame	26
2.10.3	Fase 3: Valutazione dell'esame	27
2.10.4	Fase 4: Notifica della valutazione	27
2.11	Disponibilità / Affidabilità (non Denial of Service)	27
2.12	Controllo d'accesso	28
2.13	Esempio di policy di sicurezza	28
2.14	Elementi di una policy	29
2.15	Mandatory Access Control (MAC)	30
2.16	Role-Based Access Control (RBAC)	30
2.17	Access Control Matrix (ACM)	30
2.18	Tassonomia e Modelli di attaccante	31
2.19	Modello di attaccante Dolev-Yao (DY)	31
2.20	Modello di attaccante General Attacker (GA)	32
2.21	Problemi sulle inconsistenze	32

3	Autenticazione	32
3.1	Autenticazione utente-computer	33
3.2	Autenticazione basata su conoscenza	33
3.2.1	Guessing: Attacchi e contromisure	33
3.3	Autenticazione basata sul possesso	34
3.3.1	Smart Token	35
3.4	Autenticazione basata su biometria	35
4	Cenni di Crittografia	37
4.1	Crittosistema	37
4.2	Crittografia in rete	38
4.3	Segretezza di una chiave	38
4.4	Crittografia simmetrica	38
4.4.1	Crittografia simmetrica in rete	39
4.4.2	Limiti della crittografia simmetrica	39
4.5	Crittografia asimmetrica	40
4.5.1	Crittografia asimmetrica in rete	40
4.5.2	Limite della crittografia asimmetrica	41
4.6	Riepilogo crittografia simmetrica ed asimmetrica	41
4.7	Crittosistema sicuro	41
4.8	Hash crittografico	41
4.9	CTSS + Hashing	42
4.9.1	Salting	42
4.10	Unix: aggiunta di una nuova password	43
4.10.1	Tabella Rainbow	43
4.11	Unix: verifica di una password	43
4.12	Freshness (Essere recente)	44
5	Protocolli di sicurezza basilari	45
5.1	Protocolli basilari per la freshness	45
5.2	Dalla crittografia alla sicurezza	45
5.3	Sintassi dei messaggi crittografici	46
5.4	Sintassi di un protocollo di sicurezza	46
5.5	Protocolli basilari per la segretezza	46
5.5.1	Crittografia simmetrica	47
5.5.2	Chiave asimmetrica	47
5.6	Protocolli basilari per l'autenticazione	47
5.6.1	Crittografia simmetrica	47
5.6.2	Crittografia asimmetrica	47
5.7	Combinare segretezza ed autenticazione	48
5.7.1	Crittografia simmetrica	48
5.7.2	Crittografia asimmetrica	48
5.8	Come ottenere integrità	48
5.9	La firma digitale	49
5.9.1	Creare una firma	49
5.10	Verificare una firma	49

5.11	Protocolli basilari per l'integrità	49
5.12	Crittografia asimmetrica	49
5.13	Combinare tutte le proprietà di livello 1	50
5.14	Protocollo Diffie-Hellman	50
5.15	Attacco MiTM su Diffie-Hellman	51
5.16	Irrobustire Diffie-Hellman	51
5.17	RSA Key Exchange	51
5.18	Personal Identifiable Information (P.I.I.)	52
5.19	Certificazione (crittografia asimmetrica)	52
5.20	Certificato X.509	53
5.21	Public-Key Infrastructure (PKI)	54
5.22	Chain of Trust (Catena di fiducia)	54
5.23	Segretezza vs Fiducia	54
5.24	Certificati auto-emessi (self-issued)	54
5.25	Certificate Revocation List (CRL)	56
6	Protocolli di Sicurezza storici	56
6.1	Meccanismo di challenge-response	56
6.2	Ottimizzazione protocollo	57
6.2.1	Analisi del protocollo di sicurezza	58
6.3	Scenario d'attacco	58
6.3.1	Potenziati soluzioni	58
6.3.2	Conseguenze dell'attacco	59
6.3.3	Vendetta nel modello general attacker	60
6.4	Protocollo di Woo-Lam	60
6.4.1	Attacco su Woo-Lam	61
6.4.2	Soluzione all'attacco su Woo-Lam	62
6.5	Principio di disegno: Explicitness	63
6.6	Symmetric Needham-Schroder	63
6.6.1	Replay Attack	63
7	IPSec	64
7.1	Alternative alla crittografia: Chaffing & Winnowing	64
7.2	Sicurezza a Livello di Rete	65
7.3	Sicurezza a quale livello	65
7.3.1	Sicurezza a livello rete	65
7.3.2	Sicurezza a livello trasporto	65
7.3.3	Sicurezza a livello applicazione	66
7.4	IPSec	66
7.5	Security Association (SA)	66
7.6	Security Association Database (SAD)	67
7.7	Authentication Header (AH)	67
7.7.1	Header AH: Formato	67
7.8	Funzionamento di base IPSEC	68
7.9	Prevenzione dei replay attack	68
7.10	Protezione mediante AH: trasporto vs tunnel	69

7.10.1	AH in IPv4 e in IPv6	70
7.10.2	Usi di AH: trasporto vs tunnel	71
7.10.3	AH in tunnel	71
7.11	Encapsulating Security Protocol (ESP)	71
7.11.1	ESP- formato	72
7.11.2	ESP in IPv4 e IPv6	73
7.11.3	ESP in modalità trasporto	74
7.12	Come rendere la rete domestica più sicura?	74
7.13	Combinazioni di SA	75
7.13.1	Tipo 1: sicurezza punto-punto	75
7.13.2	Tipo 2: sicurezza fra intermediari	76
7.13.3	Tipo 3: tipo 1 & tipo 2	76
7.13.4	Tipo 4: tipo 1 & sicurezza punto intermedio	77
7.14	ESP con/senza autentica	77
7.15	Internet Key Exchange (IKE)	78
8	Intrusion Detection	78
8.1	Intusion Detection Working Group	78
8.2	Intrusione versus Malware	78
8.3	Intrusione versus Dos	78
8.4	Negazione del servizio DoS	79
8.4.1	DOS vs DDOS	79
8.4.2	Come reagire ad un DoS	79
8.4.3	Anti-DoS: Cookie Transformation	80
8.5	Intrusione	80
8.5.1	Tecniche di intrusione	80
8.6	Intrusion Detection vs Intrusion Management	80
8.7	Intrusion Detection System (IDS)	81
8.7.1	Definire i comportamenti	81
8.7.2	Record di auditing	82
8.7.3	Record di Denning	82
8.8	Tecniche di rilevamento	83
8.8.1	Rilevamento Statistico - Modelli	83
8.8.2	Rilevamento a Regole	84
8.9	Tecniche di Protezione	84
8.10	Intrusion Management System (IMS)	85
9	Software nocivo (Malware)	86
9.1	Approfondimento sulle Vulnerabilità	86
9.2	Caratteristiche di un software nocivo	87
9.3	Tassonomia di un software nocivo	87
9.3.1	Trapdoor (Backdoor)	87
9.3.2	Bomba logica	88
9.3.3	Trojan	88
9.3.4	Zombie	88
9.3.5	Worm	89

9.3.6	Worm Morris	89
9.3.7	Virus	91
9.4	Distinzione delicata del SW nocivo	92
9.5	Struttura di un Semplice Virus	92
9.6	Struttura ideale di un Antivirus	93
9.7	Virus con Funzionalità di Compressione	93
9.7.1	Esempio di Virus: Brain	94
9.8	Rimozione di Software Nocivo	94
9.8.1	Ruolo delle Difese	95
10	WWW Security Protocols	95
10.1	HTTPS	96
10.2	Attacco di stripping su SSL	96
10.3	SSL vs TLS	96
10.4	SSL Protocol Suite	96
10.4.1	Record Protocol	97
10.4.2	MAC di SSL	97
10.5	SSL CCSP	98
10.6	SSL AP	98
10.7	SSL Handshake Protocol	98
10.8	Messaggi SSL	98
10.8.1	Fase 1: Stabilimento della connessione	99
10.8.2	Fase 2: Autenticazione del server e scambio delle chiavi	101
10.8.3	Fase 3: Autenticazione del client e scambio delle chiavi	101
10.8.4	Fase 4: Finish	101
10.9	Master Secret	101
10.10	Key Blocks	102
10.11	Ridurre la latenza di rete	102
10.12	Transport Layer Security (TLS)	102
10.13	MAC	103
10.14	P_hash	103
10.15	Pseudo Random Function (PRF)	104
10.16	Evoluzione di TLS	105
11	Domande d'esame	105

0 Introduzione

DISCLAIMER: manca la parte del prof. Esposito, riguardante Firewall, Iptables e NFTables, fattibilissima dalle sue stesse slide caricate su Teams.

0.1 Proprietà di sicurezza fondamentali

- Autenticazione: Verifica dell'identità di un utente o di un sistema per garantire che chi richiede l'accesso sia chi dice di essere;
- Integrità: Assicurazione che i dati non siano stati alterati o modificati in modo non autorizzato durante il loro ciclo di vita;
- Confidenzialità: Protezione dei dati sensibili da accessi non autorizzati, garantendo che solo le persone autorizzate possano accedervi e che non vengano divulgati a individui non autorizzati.

0.2 P.I.I. (Personal Identifiable Information)

Le P.I.I. (Personal Identifiable Information) sono un insieme stabile di dati che permettono di identificare un individuo univocamente. È fondamentale proteggere le P.I.I. per preservare la privacy e la sicurezza delle persone coinvolte.

0.3 Specimen (campione)

A.B.A. (Attribute-Based Authentication) L'autenticazione basata su attributi è un metodo di verifica dell'identità che si concentra su caratteristiche specifiche dell'utente anziché solo sulle credenziali come la password.

0.4 Protocollo di Trust

Il Protocollo di Trust è un meccanismo fondamentale nell'ambito della sicurezza informatica che si basa sul concetto di fiducia reciproca tra le parti coinvolte in una comunicazione. Le misure di sicurezza scardinano la fiducia per mezzo di strumenti verificati.

0.5 BIOS ed UEFI

0.5.1 UEFI

Versione moderna e più sicura del BIOS, identifica univocamente la scheda madre (TPM, legato crittograficamente alla motherboard).

0.5.2 BIOS

Concetto storico e non sicuro, software che fa partire la scheda madre.

0.6 Standard di sicurezza ISO-IEC 27001

Si tratta di uno standard internazionale che definisce le best practice per i Sistemi di Gestione della Sicurezza delle Informazioni (SGSI).

0.7 Fattore socio-tecnologico

Il fattore socio-tecnologico nella sicurezza informatica riguarda l'interazione tra comportamenti umani e tecnologie per proteggere dati e sistemi. Non va sottovalutato perché gli errori umani, la mancanza di consapevolezza e la scarsa formazione possono costituire vulnerabilità significative, compromettendo la sicurezza complessiva e aprendo la porta a attacchi informatici.

0.8 Firewall

Un firewall è un componente software o hardware che protegge una rete informatica controllando il traffico in entrata e in uscita. Il suo compito principale è decidere quali pacchetti di dati possono attraversare la rete, basandosi su regole predefinite.

In sistemi operativi Linux, un firewall comune è iptables. Nel 2016, di default, iptables su molte distribuzioni Linux non aveva regole configurate, permettendo al sistema di accettare o rifiutare pacchetti secondo la politica predefinita, solitamente permissiva.

0.9 Ransomware e Phishing

0.9.1 Ransomware

Tipo di attacco potenzialmente senza soluzione, consiste nello scaricare qualcosa che crittografa la memoria di massa. Se conosco la chiave per fare l'operazione inversa possiamo utilizzare questo strumento a nostro favore. Chi fa questo tipo di attacchi conosce la chiave per poterlo effettuare. È un tipo di malware che blocca l'accesso ai dati o al sistema dell'utente e richiede un pagamento (ransom) per ripristinarli.

0.9.2 Phishing

È un attacco informatico che mira a ottenere informazioni sensibili o personali, come password o dati finanziari, facendosi passare per una fonte affidabile tramite email, messaggi o siti web contraffatti.

0.10 Dati at rest, in transit ed in RAM

- Dati in riposo: Dati salvati quando non vengono utilizzati, come su un disco rigido.
- Dati in transito: Dati che si muovono attraverso una rete, come quando si invia un'email.

- Dati in RAM: Dati temporanei memorizzati mentre si utilizza un dispositivo, come nella RAM del computer.

0.11 Ransomware: soluzioni

BitLocker

BitLocker è un software di crittografia integrato in alcuni sistemi Windows che può essere utilizzato per crittografare i dati memorizzati su un'unità, proteggendo così i dati in riposo. Tuttavia, BitLocker protegge principalmente l'accesso fisico all'unità crittografata e non necessariamente l'accesso logico, il che significa che se qualcuno ottiene accesso al sistema con le credenziali di accesso, i dati potrebbero essere comunque vulnerabili.

0.12 Soluzioni preventive e mitigazione del problema

- Partizionare le unità e disconnetterle: Separare fisicamente le unità di archiviazione, ad esempio tramite partizioni separate o dispositivi esterni, e disconnetterle dalla rete quando non sono in uso, può aiutare a limitare la diffusione di eventuali attacchi ransomware.
- Backup del sistema: Mantenere regolari backup dei dati è fondamentale per proteggersi da attacchi ransomware. Se i dati vengono compromessi, è possibile ripristinarli dai backup senza dover pagare il riscatto.

0.13 Keylogger

Strumento software o hardware che registra segretamente le attività della tastiera di un computer o di un dispositivo mobile. Può essere utilizzato per raccogliere informazioni sensibili come password, dati finanziari o altri dati personali digitati dagli utenti. Tastiere di terze parti installabili dagli app store sono potenziali keylogger.

0.14 Pagamenti online

Effettuare pagamenti online inserendo il numero della propria carta di credito online è un enorme atto di fede. Questo comporta il rischio di esporre le informazioni finanziarie personali a potenziali minacce.

0.15 Security VS Usability

La facilità d'uso ci spinge ad effettuare atti di fede. Questo perché, se da un lato la sicurezza richiede procedure rigorose per proteggere i dati sensibili, dall'altro eccessiva complessità potrebbe scoraggiare gli utenti. Trovare un equilibrio tra sicurezza e usabilità è essenziale per garantire pagamenti sicuri e senza intoppi online.

0.16 Trust Anchor

Il lucchetto verde visualizzato su un sito web o un'applicazione durante il processo di pagamento è un esempio di trust anchor. La presenza di un trust anchor aiuta gli utenti a sentirsi più sicuri nel fornire le proprie informazioni finanziarie, in quanto indica che il sito è autenticato e che le comunicazioni sono crittografate.

0.17 Aggiornamenti Software

Gli aggiornamenti software sono fondamentali misure di sicurezza perché correggono le vulnerabilità di sicurezza esistenti nel software, fornendo patch e miglioramenti che mantengono i sistemi al passo con le minacce emergenti. Senza di essi, i dispositivi rimarrebbero esposti a rischi significativi di violazioni della sicurezza e di compromissione dei dati.

0.18 Moneta elettronica

Distributed Public Ledger (Bollettino Pubblico Distribuito). Annota tutte le transazioni di compravendita, o di qualunque tipo in altri casi. Innumerevoli problemi di sicurezza risolvibili tramite crittografia.

0.19 Ambiti di sicurezza informatica

0.19.1 Programmazione

- Null Pointer Dereference: Si verifica quando un programma tenta di accedere o dereferenziare un puntatore che punta a NULL, causando spesso un crash del programma o comportamenti imprevisti.
- Buffer Overflow: Accade quando un programma tenta di scrivere più dati in un buffer di quelli che può contenere, potenzialmente sovrascrivendo altre parti della memoria e portando a vulnerabilità di sicurezza.

0.19.2 Sistemi operativi

- Protezione memoria: Meccanismi implementati dai sistemi operativi per proteggere le regioni di memoria da accessi non autorizzati, impedendo a un processo di accedere direttamente alla memoria di un altro processo.
- Controllo d'accesso: Il controllo d'accesso regola l'accesso alle risorse del sistema operativo da parte degli utenti e dei processi, garantendo che solo gli utenti autorizzati possano eseguire determinate azioni o accedere a determinati file.

0.19.3 Servizi innovativi:

- Moneta elettronica: Sistemi di pagamento elettronici che consentono a individui e aziende di effettuare transazioni finanziarie online in modo sicuro e conveniente.
- Autenticazione audio: Tecniche di autenticazione basate sull'analisi delle caratteristiche vocali per verificare l'identità degli utenti.

0.20 Database

- Linking Attack: Un attacco che sfrutta le relazioni tra diverse entità in un database per ottenere informazioni sensibili o compromettere la sicurezza del sistema.
- K-Anonymity: Una tecnica di anonimizzazione dei dati che garantisce che ogni individuo in un dataset sia anonimo rispetto ad almeno K altri individui nel dataset.

0.21 Regole (Policy)

Le policy rappresentano le scelte progettuali per regolare il comportamento degli utenti e dei sistemi stessi. Le policy possono riguardare diversi aspetti, come l'accesso ai dati, l'utilizzo delle risorse informatiche, la sicurezza delle informazioni e così via. Un evento non viene considerato un attacco informatico se non viola le policy stabilite.

0.22 Buffer Overflow

Un buffer overflow è una vulnerabilità di sicurezza informatica che si verifica quando l'attaccante tenta di scrivere dati oltre i limiti di un buffer di memoria, causando sovrascritture di memoria in aree critiche del sistema o di altri processi. Questo può portare a comportamenti imprevisti, crash del sistema o, cosa più preoccupante, può essere sfruttato da un attaccante per eseguire codice dannoso o ottenere privilegi non autorizzati sul sistema. Altera il flusso di esecuzione del programma.

0.23 SQL Injection e SqlMap

0.23.1 SQL Injection

Consiste nell'iniettare una query all'interno di un DB (esempio bypassare password amministratore del DB) Permette ad un attaccante di inserire del codice SQL malevolo all'interno di campi di input dell'applicazione web, sfruttando le falle nel codice sottostante per manipolare le query eseguite sul database.

sqlmap

Per individuare e sfruttare tali vulnerabilità, gli esperti di sicurezza informatica

utilizzano strumenti come sqlmap, un software open source progettato per automatizzare il rilevamento e lo sfruttamento delle vulnerabilità di SQL Injection. Sqlmap opera principalmente a linea di comando e utilizza tecniche avanzate per individuare e sfruttare le falle nel codice SQL sottostante, consentendo agli esperti di valutare il rischio e implementare le contromisure necessarie per proteggere i sistemi dall'accesso non autorizzato e dalle violazioni della sicurezza dei dati.

1 Esempi reali e falsi miti

1.1 Trojan Horse

Il Trojan Horse, o Cavallo di Troia, è un tipo di software che, ingannevolmente, si presenta come innocuo o addirittura benefico per l'utente, ma in realtà nasconde intenzioni malevole. Questo tipo di malware può essere progettato per svolgere una vasta gamma di azioni dannose, come il furto di dati sensibili, il monitoraggio delle attività dell'utente, il danneggiamento del sistema o la creazione di backdoor per futuri attacchi.

Nei sistemi Unix ogni file ha un proprietario e un gruppo. Normalmente, un programma viene eseguito con i permessi dell'utente che lo lancia. Tuttavia esiste il bit **SUID (Set Owner User ID upon execution)**: Il bit SUID (Set User ID) è un attributo speciale nei permessi di un file eseguibile in sistemi UNIX e UNIX-like. Quando il bit SUID è impostato su un file eseguibile, il programma viene eseguito con i privilegi dell'utente che è proprietario del file anziché con i privilegi dell'utente che lo ha avviato. Questo significa che il programma ha temporaneamente accesso a risorse di sistema che normalmente sarebbero inaccessibili all'utente che lo sta eseguendo.

Un esempio è il comando **passwd**, che serve per cambiare la password di un utente. Questo programma appartiene all'utente root e utilizza il bit SUID, così da poter modificare il file delle password anche quando viene eseguito da un utente normale. In questo modo l'utente può cambiare la propria password inserendo quella precedente. In pratica, durante l'esecuzione il programma eredita i privilegi del proprietario (root), risultando quindi molto potente.

1.1.1 Esempio di Trojan Horse

Supponiamo che una vittima riceva un trojan sotto forma di file eseguibile chiamato ls.

Il programma malevolo potrebbe eseguire i seguenti comandi:

```
cp /bin/sh /tmp/.xxsh
```

Questo comando copia la shell in /tmp. La copia mantiene i permessi del file originale, ma non diventa automaticamente di proprietà di root.

```
chmod u+s,o+x /tmp/.xxsh
```

Questo comando imposta il bit SUID e rende la shell eseguibile, permettendo a un utente di eseguirla con i permessi del proprietario.

Successivamente il trojan esegue:

```
rm ./ls
ls $*
```

In questo modo il comportamento del comando ls appare normale, così la vittima non sospetta nulla. Tuttavia il programma ha già creato una shell nascosta.

Se l'utente esegue questo file, l'attaccante potrebbe sfruttarlo per ottenere accesso al sistema. Ad esempio, potrebbe creare o utilizzare un utente come **victim**. Un altro utente, come **giamp**, potrebbe riuscire a creare file nella home di victim grazie alla shell creata in precedenza. A questo punto l'attaccante può eseguire /tmp/.xxsh con i permessi della vittima, ottenendo una shell con privilegi elevati. Oggi questo specifico tipo di attacco non funziona più perché le vulnerabilità che lo rendevano possibile sono state corrette. Senza l'uso del bit SUID, una possibile soluzione per consentire agli utenti di cambiare le proprie password sarebbe la duplicazione del programma passwd per ogni utente, o la delega di privilegi speciali a singoli utenti. Tuttavia, queste soluzioni sarebbero meno efficienti in termini di sicurezza e manutenzione del sistema rispetto all'utilizzo del bit SUID su un singolo eseguibile come passwd. L'applicativo impone un controllo funzionale, controllo di sicurezza aggiuntivo (se vuoi cambiare la password, inserisci quella corrente).

1.2 La cartella Download

La cartella "Download" nei sistemi operativi ha uno scopo pratico di fornire agli utenti un luogo centralizzato per trovare i file scaricati dalla rete. Tuttavia nasce per motivi di sicurezza, la sicurezza dei file scaricati dipende dai permessi impostati sulla cartella stessa e dai permessi dei file specifici. Se il trojan sta nella cartella Download come fa la vittima ad eseguire questo trojan? Perché in realtà fino a qualche anno fa la cartella download non esisteva e tutto veniva salvato nella home. Gli applicativi erano file standalone, l'applicativo non aveva un installer e si lavorava esclusivamente sulla home directory. Si presuppone quindi che il file trojan si trovi quindi nella cartella home (predefinita) dell'utente. Quindi quando si eseguiva ls andava in esecuzione il trojan e non l'ls di sistema.

1.3 Variabile PATH

La variabile PATH è una componente delle variabili di ambiente del sistema che il SO utilizza per determinare i percorsi in cui cercare i programmi eseguibili. Questa informazione è essenziale per localizzare i comandi che gli utenti digitano nella shell.

```
$PATH
usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

Serie di percorsi dove il sistema ricerca i comandi da eseguire ognuno separata da :

Quando si esegue un comando, il sistema consulta una serie di percorsi specificati nella variabile PATH fino a trovare il comando desiderato. In passato, nella variabile PATH di Linux, era presente il punto (.), che indicava la possibilità di eseguire comandi direttamente dalla directory home dell'utente, ed era solitamente posizionato all'inizio della lista di percorsi da seguire. Tuttavia, questa configurazione era vulnerabile ad attacchi, quindi una mitigazione è stata rimuovere il punto (.) da PATH.

Nota bene: Questa soluzione non risolveva completamente il problema poiché impediva agli utenti tradizionali di eseguire i comandi dalla propria home. Una soluzione più efficace è stata rimuovere la priorità dalla cartella in cui vengono scaricati i file. Questo aiuta a proteggere il sistema da possibili attacchi, in quanto i file scaricati non hanno più la priorità di esecuzione rispetto ai comandi di sistema. Attualmente, per motivi di sicurezza, nella variabile PATH non troviamo neanche la cartella ./downloads. Questa è una precauzione aggiuntiva adottata per proteggere il sistema da potenziali minacce.

1.4 Strumenti Generali di Sicurezza

- Cifratura: Utilizzata per proteggere i dati attraverso la cifratura simmetrica (stessa chiave per cifrare e decifrare) e asimmetrica (due chiavi, pubblica e privata).
- Policy di Sicurezza: Definiscono regole per distinguere comportamenti leciti da tentativi di attacco, stabilendo il comportamento accettabile del sistema.
- Autenticazione: Processo di verifica dell'identità tramite password, PIN, smart card, biometria, etc., per garantire l'accesso autorizzato alle risorse.
- Programmi di Protezione: Software come antivirus, IDS (Intrusion Detection Systems), e firewall, che proteggono i sistemi da malware, monitorano il traffico di rete e filtrano accessi non autorizzati.
- Protocolli di Sicurezza: SSH e SSL assicurano la protezione dei dati durante il transito su reti non sicure, utilizzando crittografia e autenticazione.
- Fattore Umano: Sensibilizzazione degli utenti attraverso informazione e formazione riguardo alle pratiche di sicurezza, riducendo il rischio di attacchi basati sull'ingegneria sociale.

1.5 Cifratura

La crittografia è una disciplina precisa, fondata su presupposti matematici chiari e incontestabili. Tuttavia, la sicurezza associata alla crittografia è tutto tranne che esatta. Un esempio tangibile è rappresentato dalla *"sindrome del post-it"*,

in cui gli utenti attaccano pezzetti di carta con le password scritte sopra ai loro monitor. L'autenticazione a due fattori è resa obbligatoria solo per i sistemi bancari in base alla legislazione europea. Nel contesto della posta elettronica, l'autenticazione a due fattori non è di default, nonostante l'utilizzo di un sistema di single sign-on.

1.6 Limiti dell'Uso di Password

L'uso di un generatore di password solleva una questione fondamentale: non è sensibile alla sicurezza perché chi lo genera conosce la password. Questo implica un grande atto di fiducia. Le linee guida del National Institute of Standards and Technology (NIST) hanno definito nel 2004 i criteri per la creazione di password robuste, stabilendo delle politiche di robustezza universalmente accettate. In un momento successivo, nel 2016-2017, il NIST ha introdotto un nuovo set di regole rivoluzionario che non è ancora stato completamente recepito. Queste nuove linee guida cercano di risolvere lo stato di tensione tra requisiti contrastanti per le password. L'idea è di fornire agli utenti password semplici e basate su numeri, facili da ricordare, ma al contempo limitare al minimo il numero di tentativi di accesso. Un metodo comune per mitigare l'attacco di enumerazione, noto anche come attacco di brute force, è l'implementazione di un controllo a soglia, una misura universale che limita il numero di tentativi di inserimento della password consentiti prima che venga intrapresa un'azione di sicurezza.

1.7 Limiti di un Firewall

Un firewall, sia hardware che software, controlla e gestisce il traffico in ingresso verso una rete o un sistema. Tuttavia, presenta alcuni limiti che richiedono una riflessione sulla politica da adottare:

- Politiche di Filtraggio: Le politiche di filtraggio dovrebbero essere attentamente progettate per consentire il traffico solo dalle sorgenti attendibili e verso le destinazioni autorizzate.
- Filtraggio del Traffico in Uscita: È cruciale considerare anche il filtraggio del traffico in uscita. Ad esempio, bisogna essere consapevoli del rischio legato all'invio di dati sensibili da parte di malware tramite una connessione di tipo reverse shell. Filtrare il traffico in uscita può aiutare a mitigare questo rischio.
- Bind Shell vs Reverse Shell: Una bind shell ascolta su una porta specifica e attende che un utente si connetta ad essa, mentre una reverse shell si connette a una porta specifica sul sistema remoto.

1.8 Definizione di Sicurezza

La sicurezza informatica può essere definita in modo generale come il processo volto a prevenire gli attacchi e garantire il corretto funzionamento del sistema. Definiamo la Sicurezza per punti:

- È importante comprendere che la sicurezza informatica non è un prodotto fisico come una rete di computer o un sistema operativo, ma piuttosto un processo continuo che coinvolge la formazione delle persone, l'implementazione di aggiornamenti e la conduzione di sessioni di valutazione della vulnerabilità e dei test di penetrazione (VAPT).
- È essenziale riconoscere che la sicurezza informatica rappresenta il punto più vulnerabile di un sistema. La sicurezza di un sistema è tanto solida quanto il suo anello più debole.
- La sicurezza informatica si basa su ciò che si vuole proteggere e su quali minacce sono più rilevanti. È fondamentale contestualizzare il livello di sicurezza in base al tipo di attaccante o al modello di rischio.
- La valutazione della sicurezza informatica è sempre soggetta a un'analisi costi/benefici. È importante bilanciare il costo degli investimenti in sicurezza con i potenziali danni derivanti da una violazione della sicurezza.
- È cruciale perseguire sempre un livello elevato di sicurezza, poiché un sistema potrebbe essere protetto da attacchi condotti da hacker meno esperti ma risultare vulnerabile a quelli orchestrati da individui più competenti e sofisticati.

1.8.1 Rischi base per la sicurezza

1. Complessità del Sistema da mettere in Sicurezza: Un problema fondamentale riguarda la necessità dei browser di comunicare in modo chiaro e accurato al loro utente il livello effettivo di sicurezza dei siti web visitati.
2. Combinazione di Punti: È essenziale combinare i sistemi con attenzione per garantire una protezione efficace.
3. Predisposizione ai Bug: È cruciale comprendere le proprietà inattese di un software, definizione fondamentale per gestire i rischi.
4. Proprietà Emergenti: Le nuove e moderne caratteristiche introdotte nei software possono portare a nuovi problemi di sicurezza. L'evoluzione del software genera nuove funzionalità e, di conseguenza, nuove esigenze di sicurezza connesse a tali funzionalità (conforme a ISO 27005: Sorgenti di Rischio).
5. Fattore Umano: Il fattore umano rimane sempre cruciale in termini di sicurezza. L'uomo rappresenta l'anello più debole della catena.

1.8.2 Rischi digitali per la sicurezza

1. Automatizzazione dell'Offensiva: La possibilità di automatizzare gli attacchi rappresenta una minaccia significativa. Gli attaccanti possono sfruttare strumenti e script per eseguire attacchi in modo efficiente e senza intervento umano.

2. Assenza di Distanza e Attacchi da Remoto: La mancanza di distanza geografica non costituisce più una barriera per gli attacchi. È possibile eseguire attacchi da remoto, aumentando il punteggio di vulnerabilità secondo il CVSS (Common Vulnerability Scoring System), che valuta la gravità di una vulnerabilità su una scala da 1 a 10.
3. Mercato delle Vulnerabilità: Esiste un mercato dinamico per le vulnerabilità informatiche. Gli individui etici che scoprono vulnerabilità spesso le segnalano a database internazionali dedicati, ma vi è anche un mercato in cui le vulnerabilità possono essere acquistate e vendute illegalmente.
4. Difficoltà nella Reazione: La risposta agli attacchi informatici può essere estremamente complessa e difficile. Gli attaccanti spesso operano con grande velocità e astuzia, mentre le organizzazioni devono affrontare sfide tecniche, legali e di comunicazione nel tentativo di mitigare gli effetti degli attacchi e ripristinare la sicurezza del sistema.

1.9 Il gioco della sicurezza (Attaccare e difendere)

Nel processo di sicurezza informatica, si segue un ciclo/gioco continuo di attacco e difesa.

1.9.1 Metodologia d'attacco

1. Studiare il sistema target (port scanning);
2. Cercarne i potenziali punti deboli (daemon flaw);
3. Disegnare (o scaricare) eseguibili per verificare i punti deboli (exploits);
4. Torna al numero 1.

1.9.2 Metodologia di difesa

1. Installare strumenti di difesa (firewall, IDS);
2. Aggiornare il sistema (updates, security patch);
3. Monitorare il sistema;
4. Torna al numero 1.

Questo processo richiede una continua vigilanza e adattamento per contrastare le minacce informatiche in evoluzione.

1.10 Port Scanning

Il port scanning è un processo mediante il quale vengono esaminate sequenzialmente le porte di un sistema al fine di rilevare eventuali aperture e ottenere informazioni sui servizi o sui sistemi operativi in esecuzione dietro di esse.

Scansione Stealth: Si tratta di una tipologia di scansione che mira a non essere rilevata dal sistema bersaglio, con l'obiettivo di sfuggire alla rilevazione da parte dei sistemi di Intrusion Detection.

Originariamente concepito come strumento diagnostico, il port scanning è diventato uno strumento principale utilizzato dai pen-tester, anche se può essere sfruttato per scopi malevoli, simile all'uso iniziale della polvere da sparo per i fuochi d'artificio che oggi è associata a usi più pericolosi.

Esempio di porte aperte (Output di nmap): Nmap non mostra tutte le 65535 porte disponibili su un sistema, ma soltanto quelle effettivamente esposte e accessibili dall'esterno.

```
ftp 21/tcp File Transfer
telnet 23/tcp Telnet
smtp 25/tcp Simple Mail Transfer
finger 79/tcp Finger
login 513/tcp remote login(rlogind)
shell 514/tcp rlogin style (rshd)
printer 515/tcp spooler (lpd)
```

2 Proprietà, Attacchi e Attaccanti

Gli attacchi informatici possono essere classificati in base agli obiettivi che perseguono. Alcuni esempi includono:

- Accesso non autorizzato al sistema;
- Accesso al sistema a nome di un'altra persona;
- Ottenimento di privilegi elevati all'interno del sistema;
- Impersonificazione di un utente autorizzato;
- Furto di dati sensibili;
- Attacco di negazione del servizio (DoS);
- Mancato recapito dei dati o dei servizi.

Se un attaccante ha accesso fisico al dispositivo, può seguire i seguenti passaggi:

1. Prova ad accedere al sistema;
 - Contromisura: Autenticazione tramite PIN, Password o Biometria

2. Cerca di utilizzare le funzionalità del sistema (ad esempio, l'accesso alla casella di posta elettronica);
 - Contromisura: Autenticazione per accedere alle funzionalità
3. Cerca di acquisire dati sensibili;
 - Contromisura: Crittografia dei dati sensibili.

Nella pratica, molte di queste contromisure possono fallire. Ad esempio, i browser memorizzano le password, esiste una forte resistenza generale all'adozione della crittografia (anche se esistono strumenti come BitLocker di Windows per crittografare i dischi).

2.1 Pirateria digitale

La pirateria digitale è un problema serio che riguarda vari aspetti:

- Furto di proprietà intellettuale: La natura digitale dei contenuti permette una copiabilità infinita, rendendo difficile proteggere la proprietà intellettuale. Le contromisure possono includere il watermarking, o l'uso di digital stores che offrono un'esperienza utente superiore rispetto al download illegale.
- Furto di identità: Il furto di identità può avvenire quando qualcuno riesce a fare login con le credenziali di un altro utente. Le contromisure possono includere l'autenticazione a più fattori, che richiede all'utente di verificare la propria identità in più modi prima di ottenere l'accesso.
- Furto di marchio: Il furto di marchio può avvenire quando un logo o un marchio viene utilizzato senza permesso. Le contromisure possono includere l'applicazione delle leggi sul diritto d'autore e dei marchi.

2.2 Proprietà di sicurezza

- Impossibilità di Conoscere Tutte le Proprietà di Sicurezza: Poiché la sicurezza informatica è un campo in continua evoluzione e complesso, è impossibile conoscere tutte le sue proprietà in modo esaustivo.
- Pilastri Fondamentali (ISO 27001): Segretezza, Autenticazione e Integrità sono considerati pilastri fondamentali della sicurezza dell'informazione secondo lo standard ISO 27001.
- CIA (Confidentiality Integrity Availability): Questi tre principi rappresentano la triade classica della sicurezza informatica, enfatizzando la riservatezza, l'integrità e la disponibilità dei dati.
- Ulteriori Proprietà a Livello Più Alto:
 - Livello 2: Non Ripudio, Disponibilità

- Livello 3: Valuta Elettronica, Equilibrio tra Privacy e Legge, Controllo di Accesso e Livelli di Segretezza.

Iniziamo con le proprietà di sicurezza di **Livello 1**.

2.3 Definizione di Segretezza (Confidenzialità)

L'informazione non sia rilasciata ad entità non autorizzate a conoscerla.

La segretezza si riferisce all'informazione che deve rimanere privata e non deve essere divulgata a entità non autorizzate. Una password rappresenta un segreto condiviso tra due parti, come un utente e un server di autenticazione. È fondamentale che entrambe le parti conoscano il segreto per poter autenticare correttamente l'utente.

2.3.1 Misure di sicurezza

- Crittografia: Consiste nel convertire i dati in un formato illeggibile a meno che non si disponga di una chiave di decrittazione. Questo garantisce che anche se i dati sono intercettati, non possono essere compresi senza la chiave corretta.
- Steganografia: È un'alternativa alla crittografia che non modifica l'aspetto esterno dei dati, ma li nasconde all'interno di altri dati. Ad esempio, si possono modificare i bit meno significativi di un'immagine per memorizzare informazioni senza che queste siano visibili all'occhio umano. Esempio: Modifica dei bit meno significativi di un'immagine per nascondere informazioni al suo interno, che rimangono impercettibili all'occhio umano.

Misure di Sicurezza Più Robuste: Alcuni esempi includono l'utilizzo di algoritmi crittografici robusti come AES256 e altri protocolli di sicurezza avanzati.

2.3.2 Differenza tra codifica e cifratura

Codifica:

- Trasforma i dati da un formato all'altro senza l'uso di una chiave segreta.
- Scopo principale è la conversione dei dati in un formato specifico.
- Non è una misura di sicurezza.

Cifratura:

- Utilizza una chiave segreta per trasformare i dati in modo che siano comprensibili solo a chi possiede la chiave.
- Scopo principale è garantire la confidenzialità dei dati.
- È una misura di sicurezza che protegge i dati da accessi non autorizzati.

2.4 Definizione di Autenticazione

Le entità siano esattamente chi dichiarano di essere.

L'autenticazione è il processo di verifica dell'identità delle entità, confermando che esse siano effettivamente chi dichiarano di essere. Coinvolge la validazione dell'identità di individui, amici o siti web. Ad esempio, i siti web devono autenticare gli utenti per consentire l'accesso ai servizi offerti. Questo concetto è ampiamente applicato nei browser web e nelle applicazioni mobili basate su browser. **Esempi:**

- Autenticazione di un interlocutore, di valuta, etc.;
- Autenticazione di un utente, di un URL con un browser, di uno smart-phone, etc.

Alcune misure sono la Conoscenza (password, PIN), Possesso (smartcard, smart token) e la Biometria (impronte, iride, etc..).

2.5 Definizione di Integrità

L'informazione non sia modificata da entità non autorizzate.

L'integrità, come proprietà fondamentale di sicurezza, implica che le informazioni non possono essere modificate da entità non autorizzate. Esempi:

- Integrità di tutti gli elementi di una transazione;
- Integrità dei dati registrati in un database;
- Integrità di un video;
- Integrità di una cache.

Alcune misure sono:

- **Checksum:** Si tratta di un controllo di integrità che verifica la presenza di errori all'interno di un pacchetto dati attraverso una semplice operazione di somma. Tuttavia, non costituisce una misura di sicurezza in quanto non impedisce a entità non autorizzate di modificare i dati.
- **Firma elettronica:** Mentre qualsiasi messaggio che viaggia attraverso la rete può essere alterato da nodi intermediari e anche un messaggio cifrato può essere soggetto a manipolazioni, la firma digitale o elettronica non può essere alterata. Questa firma fornisce un'ulteriore garanzia di integrità, rendendo i messaggi immutabili e autenticati.

2.6 Definizione di Privatezza (Privacy)

Diritto di un'entità di rilasciare o meno i propri dati personali ad altre entità.

La privacy è il diritto alla segretezza e alla riservatezza delle proprie informazioni personali. Una misura di sicurezza fondamentale per proteggere la privacy è quella di **non fornire il consenso** alle politiche che richiedono la condivisione o la divulgazione dei dati personali. Secondo il Regolamento Generale sulla Protezione dei Dati (GDPR), in caso di cambio di fornitore di servizi, è possibile richiedere all'ente di cancellare tutti i propri dati personali entro 72 ore dalla richiesta. La privatezza viene spesso intesa come il diritto alla segretezza delle proprie informazioni. Ad esempio, una password è considerata segreta, **ma non necessariamente privata**, poiché può essere condivisa con altre persone, mentre la privacy implica che queste informazioni siano riservate e non accessibili a terzi senza autorizzazione.

2.7 Definizione di Anonimato

Diritto dell'iniziatore di una transazione di rilasciare o meno la propria identità ad altre entità.

L'anonimato è il diritto di non rivelare la propria identità ad altre entità, consentendo così di nascondere il proprio identificativo. Alcune misure sono:

- TOR (The Onion Router): TOR è un sistema di routing che offre servizi di anonimato su Internet. Utilizza un sistema a cipolla, rendendo difficile il tracciamento dell'indirizzo IP attraverso la rete tramite IP Traceback, grazie a difficoltà strutturali e topologiche che complicano l'autenticazione.
- Server Anonimizzatore: Questi server permettono la comunicazione tra utente e servizio, mantenendo anonima la comunicazione tra il server anonimizzatore e il destinatario. Tuttavia, la comunicazione tra l'utente e il server anonimizzatore non è protetta.
- VPN (Virtual Private Network): A differenza di TLS che protegge i dati in transito, una VPN fornisce all'utente un indirizzo IP locale al chiamato, ma non garantisce l'anonimato.
- Navigazione in Incognito: Questa modalità non elimina i cookie preesistenti, le credenziali o i cookie registrati temporaneamente. Al di fuori del proprio dispositivo, la navigazione in incognito non offre anonimato, in quanto le attività possono essere tracciate come in una sessione di navigazione normale.

Ora passiamo alle proprietà di sicurezza di **Livello 2**

2.8 Definizione di Non-Ripudio

L'entità non possa negare la propria partecipazione ad una transazione con uno specifico ruolo La non-ripudio è una caratteristica di sicurezza che implica la mancanza di fiducia reciproca tra le entità coinvolte, impedendo loro di negare la propria partecipazione o azione nei confronti dell'altra parte. Uno **scenario tipico** è la **Compravendita**: nel contesto di una compravendita, la non-ripudio garantisce che entrambe le parti non possano negare di aver compiuto o accettato una transazione.

La Posta Elettronica Certificata (PEC) è una tecnologia utilizzata per garantire la non-ripudio nella comunicazione via email. Viene impiegata in contesti legali, come i concorsi pubblici, in quanto equivale a una raccomandata con ricevuta di ritorno. Tuttavia, entrambe le parti coinvolte devono essere dotate di un'indirizzo PEC affinché questa garanzia sia valida.

2.8.1 Autenticazione, anonimato e non-ripudio

- L'autenticazione esclude l'anonimato.
- L'anonimato esclude l'autenticazione.
- L'autenticazione garantisce il non ripudio?
 - Quando un utente si autentica su un servizio e le azioni vengono registrate in un log, questo non necessariamente garantisce il non ripudio. I log, simili a checksum, possono essere manipolati durante un attacco se non garantita l'integrità. Pertanto, l'autenticazione da sola non assicura la non ripudiabilità.
- Il non ripudio richiede l'autenticazione.
- L'anonimato implica mancanza di autenticazione, la mancanza di autenticazione implica mancanza di non ripudiabilità. Queste relazioni sono transitive.
- Il non ripudio richiede autenticazione; l'autenticazione, a sua volta, richiede anonimato per transitività.

Autenticazione, anonimato e non-ripudio

Relazioni logiche fra loro:

- 1 autenticazione $\xrightarrow{(ovvio)} \neg$ anonimato
- 2 anonimato $\xrightarrow{(ovvio)} \neg$ autenticazione
 - autenticazione $\xrightarrow{(1,2)} \neg$ anonimato
- 3 autenticazione $\xrightarrow{??}$ non-ripudio
- 4 non-ripudio $\xrightarrow{(ovvio)}$ autenticazione
- 5 anonimato $\xrightarrow{(2, contra(4))} \neg$ non-ripudio
- 6 non-ripudio $\xrightarrow{(4,1)} \neg$ anonimato

In pratica ovviabili tramite enforcing temporaneo

Figure 1: Autenticazione, anonimato e non-ripudio

2.9 WATA

Protocollo di sicurezza che coinvolge attori umani (quando si effettua un esame universitario siamo parte di una cerimonia). Anonimato è il miglior prerequisito per una valutazione equa ad un esame universitario. Come facciamo ad autenticare e rendere anonimo allo stesso tempo uno studente all'esame universitario? Proprietà che richiede la suddivisione dei compiti o delle responsabilità tra più soggetti. Ad esempio, durante gli esami universitari, una persona potrebbe essere responsabile dell'autenticazione dei candidati, mentre un'altra persona si occupa della valutazione degli esami scritti. L'auditabilità pubblica si riferisce all'autenticazione che avviene sotto gli occhi dell'individuo o di un pubblico. Ad esempio, durante un esame universitario, potrebbe essere utilizzato un talloncino contenente il nome dello studente, sigillato all'interno di una busta e poi chiuso con una firma a cavallo della busta per garantire l'integrità fisica e fornire evidenza di eventuali manomissioni (tamper evidence). Successivamente, l'esame scritto e la busta contenente il talloncino vengono inseriti in un'altra busta che viene aperta solo durante la fase di correzione dell'esame. Tuttavia, non c'è garanzia che il nome contenuto nel talloncino non venga visualizzato prima della correzione del compito, sollevando dubbi sulla sicurezza del processo di auditabilità pubblica. Anche se si implementa una efficace separazione dei compiti, non risolviamo completamente il dilemma della conciliazione tra autenticazione ed anonimato. Anche un piccolo segno distintivo nell'esame scritto potrebbe compromettere il sistema, permettendo all'esaminatore di identificare il candidato. L'autenticazione del candidato è un processo separato dall'anonimato richiesto durante l'esame. Tuttavia, l'anonimato può essere infranto quando viene associato il voto all'identità del candidato. Pertanto, cercare di combinare

autenticazione ed anonimato in un'unica soluzione risulta impossibile. Il **processo di WATA**, acronimo di "Written Authenticated Though Anonymous", prevede due principali aspetti:

1. L'autenticazione dello studente, per la protezione dell'integrità dell'esame e per la validità dei risultati da parte del docente;
2. L'anonimato del compito, per garantire la privacy dello studente e ridurre il rischio di influenze esterne sulla valutazione.

Durante l'esame, uno studente riceve un talloncino che lo identifica (token), il quale viene successivamente portato a casa dall'utente al termine del compito. Tuttavia, c'è il rischio che lo studente possa manipolare il talloncino al fine di disassociare il codice a barre dalla propria identità. Per mitigare questo rischio, il token necessita di una misura di integrità simile a quella presente sulle banconote. Nel caso di WATA, tale integrità è garantita da una firma a cavallo apposta dal docente, la quale non deve fuoriuscire dal token, che non può essere compromessa o rimossa dal talloncino. Questa firma non è immune agli attacchi, ma rende più complesso per lo studente alterare l'associazione tra il talloncino e la propria identità. Per garantire l'anonimato durante gli esami, è fondamentale adottare una combinazione di misure di sicurezza:

- Randomizzazione dei compiti: Assegnare i compiti in modo casuale ai candidati per evitare qualsiasi schema o prevedibilità.
- Copertura del barcode e altre informazioni scritte: Durante il processo di autenticazione, è importante coprire il barcode e qualsiasi altra informazione che possa rivelare l'identità dello studente.

2.10 WATA2

Protocollo cyber-fisico garantisce una sicurezza integrata sia materiale (cartacea) che digitale, consentendo all'esaminato di rimanere anonimo durante la fase di valutazione da parte del docente. Il protocollo prevede una suddivisione dell'esame in quattro fasi:

1. Preparazione
2. Test (effettuazione dell'esame)
3. Valutazione
4. Notifica del voto allo studente

La presenza dell'invigilator non risolve i problemi di sicurezza quando esaminatore e invigilator sono la stessa persona. Il protocollo è caratterizzato da numerosi aspetti funzionali e si basa su un database contenente 1000 domande, le quali vengono randomizzate e assegnate casualmente a ciascun foglio di esame, con la possibilità di scegliere il numero di domande per compito.

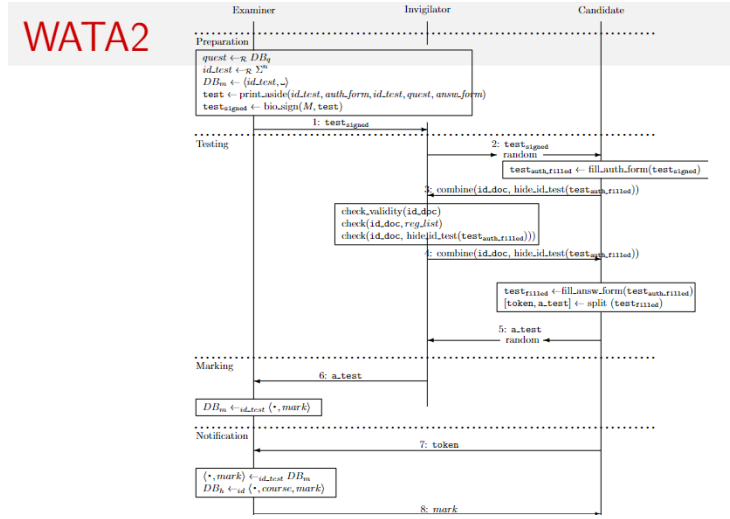


Figure 2: WATA2

2.10.1 Fase 1: Preparazione dell'esame

1. Estrazione casuale delle domande dal database.
2. Generazione casuale di un ID alfanumerico di lunghezza n.
3. Creazione di una voce nel database contenente l'ID del test e inizializzazione della cella per il voto.
4. Emissione di:
 - Primo ID del test (token);
 - Form di autorizzazione (token);
 - ID del test (sul foglio d'esame);
 - Domande d'esame;
 - Spazi vuoti per le risposte dello studente.
5. Apposizione della firma di integrità da parte del docente sopra il codice a barre e il nome e cognome (bio_sign).

2.10.2 Fase 2: Effettuazione dell'esame

1. Consegna formale dei fogli da parte dell'esaminatore all'invigilator;
2. Assegnazione casuale dei compiti da parte dell'invigilator;
3. Compilazione da parte dello studente del modulo di autenticazione (token), firmando nella sezione apposita;

4. Fornisce le informazioni appena scritte all'invigilator per la fase di autenticazione un documento di riconoscimento, nascondendo gli elementi peculiari del compito come ad esempio la prima domanda;
5. Verifica da parte dell'invigilator della validità del documento, dell'iscrizione dello studente nella lista dei registrati all'esame e della coerenza tra i dati del documento di identità e quelli nel token di autenticazione;
6. Risposta dello studente alle domande del compito, completando il test;
7. Rimozione del token dal compito per mantenere l'anonimato, consegna dell'esame all'invigilator per la fase di valutazione.

2.10.3 Fase 3: Valutazione dell'esame

1. Consegna formale dei test all'esaminatore da parte dell'invigilator;
2. Correzione da parte del docente degli esami scritti;
3. Registrazione del voto nel database.

2.10.4 Fase 4: Notifica della valutazione

1. Lo studente o un suo delegato si presenta al dipartimento (irrilevante perchè il voto va comunque all'autore del compito) con il token tagliato durante la fase 2 per la notifica del voto;
2. Registrazione formale, in un ulteriore database (storico - h), delle informazioni del corso, dell'identità dello studente e del voto assegnato. Attualmente questo database è Smartedu.

Nella fase finale, c'è il rischio che solo il docente, attraverso il database, possa vedere il voto dell'esame e manipolarlo, ad esempio dicendo allo studente di essere stato bocciato o di aver ottenuto un voto basso, anche se il punteggio effettivo del compito de-anonimizzato è più alto. Inoltre, potrebbe omettere elementi dall'esame durante la valutazione e tenerli separatamente. Per prevenire questo, la comunicazione del voto dovrebbe essere pubblica, ad esempio tramite screen-sharing, per evitare discrepanze tra il voto comunicato e quello effettivamente assegnato. Inoltre, c'è un problema di integrazione tra il protocollo WATA e il database finale di valutazione (GOMP) per renderlo pubblicamente verificabile.

2.11 Disponibilità / Affidabilità (non Denial of Service)

Il sistema sia operante e funzionante in ogni momento.

L'affidabilità del servizio è una caratteristica di sicurezza fondamentale, ed è essenziale garantire che il sistema sia disponibile e funzionante in ogni momento. Il rifiuto del servizio è considerato un attacco DoS (Denial of Service), come

ad esempio una fork bomb che genera ripetutamente processi figlio, sovraccaricando il sistema e impedendone il corretto funzionamento. Alcune misure di mitigazione sono:

- Autenticazione e restrizione dell'accesso agli utenti (quindi limitare l'accesso ad utenti autenticati, non una buona soluzione in alcuni casi);
- Aumento della complessità dell'accesso al sistema, impegnando il chiamante computazionalmente e trasferendo il carico al chiamante per metterlo in attesa;

Sostanzialmente garantire la disponibilità rimane un problema aperto.

Ora passiamo alle proprietà di sicurezza di **Livello 3**

2.12 Controllo d'accesso

Ogni utente ha accesso solamente alle risorse e ai servizi per i quali è autorizzato. Esempi:

- Autenticazione dell'utente.
- Definizione delle **policy** di sicurezza.
- Implementazione delle politiche (liste di controllo degli accessi (ACL), utenti limitati).

2.13 Esempio di policy di sicurezza

1. Un utente ha il permesso di leggere un qualunque file pubblico;
2. Un utente ha il permesso di scrivere solo su file pubblici di sua proprietà;
3. Un utente ha il divieto di fare il downgrade di un file, ovvero prendere una versione più vecchia di un file;
4. L'utente ha l'obbligo di cambiare la propria password quando scade;
5. Un utente segreto ha il permesso di leggere su un qualunque file non pubblico, come se fosse un utente root;
6. Un utente segreto ha il permesso di scrivere su qualunque file non pubblico;
7. Un amministratore ha il permesso di sostituire un qualunque file con una versione più obsoleta, il nome utente segreto è solo evocativo;
8. Un utente che non cambia la sua password scaduta ha il divieto di compiere qualunque operazione;
9. Un utente che non cambia la password non ha discrezione di cambiarla.

2.14 Elementi di una policy

- Ruoli: Utente, utente segreto, sistemista, utente negligente;
- Utente: Qualunque entità che ricopra un certo ruolo;
- Operazioni: Leggere, scrivere, downgrade, cambio password;
- Modalità (Verbi servili(volare, dovere, potere), nessuna volontà ma obblighi);
- Obbligo, permesso, divieto, discrezionalità.

L'utenza prende il requisito autorizzativo di un ruolo.

Parte 2

Proprietà, Attacchi e Attaccanti

Modalità e relazioni fra loro

- Modalità base
 - 1 $Obbligatorio(x)$
- Modalità derivate
 - 1 $Vietato(x)$
 - 2 $Permesso(x)$
 - 3 $Discrezionale(x)$
- Loro definizioni
 - 1 $Vietato(x) = Obbligatorio(\neg x)$
 - 2 $Permesso(x) = \neg Obbligatorio(\neg x)$
 - 3 $Discrezionale(x) = \neg Obbligatorio(x)$

Giampaolo Bella (giamp@dm.unict.it)

Internet Security, A.A. 2019/20

9 c.f.u. 60 / 1

Figure 3: Modalità e relazioni tra di loro

Inconsistenze di una politica

- **Contraddizione** $\equiv$
 - $Obbligatorio(x) \wedge \neg Obbligatorio(x)$
- **Dilemma** $\equiv$
 - $Obbligatorio(x) \wedge Obbligatorio(\neg x)$

Figure 4: Inconsistenza di una politica

2.15 Mandatory Access Control (MAC)

Il Controllo d'Accesso Obbligatorio (MAC) stabilisce una politica riguardante la scrittura delle policy ed è fondato sull'obbligo, con radici nell'ambito militare. **Esempio:** Il sistema richiede all'utente di cambiare la password del sistema ogni sei mesi. È importante notare che i sistemi Windows e Linux che utilizziamo attualmente non implementano il Controllo d'Accesso Obbligatorio (MAC).

2.16 Role-Based Access Control (RBAC)

Il Controllo d'Accesso Basato sui Ruoli (RBAC) si fonda su politiche non obbligatorie e si basa sulla definizione di ruoli utente, come amministratori e utenti base. In questo contesto, il controllo degli accessi è determinato dal ruolo dell'utente.

Il concetto chiave è rappresentato dalle definizioni di Permesso e Divieto:

- Permesso $\leftrightarrow$ Non è vietato (x);
- Vietato $\leftrightarrow$ Non è permesso (x)

Questa politica è ampiamente utilizzata nei sistemi operativi attuali.

2.17 Access Control Matrix (ACM)

Questa matrice rappresenta lo stato dei permessi all'interno di un sistema. Le righe corrispondono ai soggetti (ad esempio, gli utenti) e le colonne corrispon-

dono agli oggetti (ad esempio, i file). Una cella specifica nella matrice, indicata come $ACM[s, o]$, mostra i permessi che il soggetto 's' ha sull'oggetto 'o'. La dimensione della matrice è proporzionale alla dimensione del sistema. In Linux, un soggetto 's' può essere un gruppo di utenti.

	File1	File2	File3	Program1
Alice	<i>read, write</i>	<i>read</i>		<i>execute</i>
Bob	<i>read</i>		<i>read, write</i>	
Charlie		<i>read</i>		<i>read, execute</i>

Figure 5: ACM

- Lista di Controllo degli Accessi (ACL): Questa è essenzialmente ogni colonna dell'ACM, registrata con l'oggetto specifico. Mostra tutti i permessi che vari soggetti hanno su un particolare oggetto.
- Lista delle Capacità (Capability List): Questa è ogni riga dell'ACM, registrata con il soggetto specifico. Mostra tutti i permessi che un particolare soggetto ha su vari oggetti.

2.18 Tassonomia e Modelli di attaccante

Si possono elencare per vari moventi (Ricchezza, Informazioni sensibili, Potere, Gloria, Divertimento) e per varie classificazioni (Diritti, Risorse, Esperienza, Rischio accettato).

Definizione: Un modello di attaccante specifica (le capacità offensive di) un preciso attaccante.

Un modello di attaccante specifica le capacità offensive di un preciso attaccante. Il modello di attaccante sta alla cyber security come la complessità asintotica sta ad algoritmi e programmazione. *N.B.:* ogni affermazione di sicurezza ha senso limitatamente a un modello, **cambiare il modello** potrebbe cambiare il valore di verità di un'affermazione di sicurezza, fare un'analisi caso peggiore riciede un modello massimamente offensivo.

2.19 Modello di attaccante Dolev-Yao (DY)

L'attaccante è unico e superpotente, ovvero controlla l'intera rete, ma non può violare la crittografia.

Il Modello Dolev-Yao, denominato dai suoi creatori, è un modello altamente aggressivo progettato per valutare la sicurezza dei nostri prodotti. È utilizzato per analizzare la sicurezza delle reti e rappresenta un tipo di attacco. Nella sua implementazione, viene considerato un scenario in cui due servizi segreti, identificati come 007, operano rispettivamente a Londra e a New York, comunicando tra loro in modo sicuro (è importante spiegare in che modo viene definita

la sicurezza durante l'esame). L'obiettivo è creare un modello estremamente robusto che possa rappresentare le situazioni più critiche, ipotizzando che tutti gli attori nel mondo siano coinvolti nel tentativo di compromettere questa comunicazione tra gli agenti segreti 007. *N.B.*: in questo contesto, l'intera rete è considerata ostile e aggressiva al massimo livello. L'aggressività massima indica che l'attaccante, che è rappresentato dalla rete stessa, ha il controllo completo su tutte le operazioni. La configurazione tipica è la seguente: Alice <- >Dolev-Yao <- >Bob.

2.20 Modello di attaccante General Attacker (GA)

Chiunque può essere attaccante senza interesse a colludere con gli altri, al peggio con capacità di totale controllo della rete, ma senza violare la crittografia.

Nel contesto dei modem 56k, la linea telefonica e quella di rete erano integrate, il che significava che una volta connessi a Internet, la linea telefonica diventava inutilizzabile. Con l'ISDN, invece, la linea telefonica e quella di rete erano separate. Il modello proposto da Dolev-Yao non è più considerato realistico poiché ogni attaccante ha i propri interessi e obiettivi. Nello specifico scenario di trasferimento di un messaggio tra Alice e Bob nel contesto del modello Dolev-Yao, si presume che l'attaccante intermedio, ovvero la rete stessa, possa intercettare il messaggio. Tuttavia, nel modello di Attaccante Generico, un attaccante potrebbe sfruttare un attacco senza informare nessuno.

2.21 Problemi sulle inconsistenze

Se un giorno la policy dovesse rispettare nuovi requisiti, un'idea semplice sarebbe aggiungere una nuova regola per soddisfarli. Tuttavia, questo potrebbe causare la comparsa di nuove inconsistenze. Per risolvere queste inconsistenze, si può stabilire un ordine di priorità sulle regole. Ad esempio, la regola più recente ha priorità maggiore rispetto a quelle meno recenti. Ad esempio:

1. Nessuno studente può accedere in aula.
2. Gli studenti dottorandi possono accedere in aula.

Anche se questo crea un'inconsistenza, la seconda regola, inserita più di recente, ha priorità maggiore rispetto alla prima. Nel contesto dei firewall, la regola predefinita (default rule) ha sempre la priorità più bassa rispetto alle altre regole.

3 Autenticazione

Iniziamo il capitolo dell'autenticazione elencando le varie ambientazioni di autenticazione, spiegando dopo cosa sono e cosa rischiano.

- Utente-computer: Accedere ad un sistema tramite password o biometria

- Computer-computer: un computer vuole accedere ad un altro indipendentemente dal suo utente (IP, MAC)
- Computer-utente: L'utente è convinto ed ha evidenza dell'identità del sito
- Utente-utente: meno nota (Kerberos)

3.1 Autenticazione utente-computer

- Basata su conoscenza di segreti prestabiliti e precondivisi: password, passphrase, PIN
- Basata su possesso di dispositivi magnetici o elettronici: smart card, smart token
- Basata su biometria di caratteristiche fisiche dell'utente: impronta, iride, tono vocale

3.2 Autenticazione basata su conoscenza

Conoscere la giusta password comprova identità. Semplice ed economica da implementare, ma si corrono i rischi di:

- Guessing: riuscire ad indovinare la password (brute force);
- Snooping: Sbirciare la password quando viene inserita;
- Spoofing: Scoprire la password tramite fake login (trojan);
- Sniffing: Intercettare la password durante la fase di trasmissione (Wire-shark).

3.2.1 Guessing: Attacchi e contromisure

1. Provo inizialmente con password brevi, tipiche e relative all'utente;
2. Attacco dizionario: Vengono provate tutte le parole di un dizionario, arricchite con doppie parole, (0 al posto di o o 1 al posto di i);
3. Attacco di forza bruta: Vengono provate tutte le parole costruibili in un dato vocabolario (alfanumerico, simbooli e caratteri speciali) di lunghezza via via crescente, tutte le combinazioni di lunghezza 1, poi lunghezza 2... complessità esponenziale

Chiunque conosca la password di un utente per un sistema può impersonare in toto quell'utente col sistema. Esiste un controllo a soglia coerente con le linee guida NIST del 2016, tecnologia già implementata nei bancomat. Le contromisure sono:

1. Controllo sulla password: Lunghezza della password e presenza di lettere, simboli e caratteri speciali;

2. Controllo sul numero di inserimenti: Controllo a soglia sovracitate;
3. Uso di CAPTCHA: Completely Automated Public Turing Test To Tell Computers and Human Apart (Test di Turing Completamente Automatico e Pubblico per Distinguere Computer e Umani.)

Alan Turing fu il primo a teorizzare l'intelligenza artificiale e il test di Turing è stato superato da una macchina solo recentemente. Un esempio pratico è la visualizzazione di parole distorte che devono essere riconosciute. Le CAPTCHA cercano di portare il test di Turing a un livello grafico. Un algoritmo recente di Google, che doveva riconoscere i numeri civici per oscurarli, è stato in grado di superare il 99% delle CAPTCHA alfanumeriche.

Google ReCaptcha è una tecnologia completamente rivoluzionaria che richiede all'utente di dimostrare di essere umano. L'algoritmo dietro a questo sistema non è noto pubblicamente, ma si suppone che tenga traccia dei movimenti del mouse.

Per una buona password, si potrebbero seguire delle norme. La norma fondamentale è bilanciare al meglio **mnemonicità** (sia facile da ricordare cosicché non serva scriverla) e **complessità** (sia robusta verso guessing). Questo implica:

- Non usare una parola del dizionario;
- Usare almeno 8 caratteri;
- Non usare la stessa password per autenticazioni diverse, altrimenti basterebbe una sola compromissione.

La password va mantenuta in qualche modo sul sistema al quale garantisce l'autenticazione utente: la soluzione sarebbe **memorizzare** in un database ciascuna password in qualche forma, del tipo **Compatible Time Sharing System (CTSS, 1960, MIT)**. Le password memorizzate in chiaro su un file di sistema protetto da politica di sicurezza. Si associano in una tabella username e password.

3.3 Autenticazione basata sul possesso

Nell'autenticazione basata sul possesso, l'identità dell'utente è verificata tramite il possesso di un oggetto magnetico o elettronico, come una smart card o un token. Questo oggetto può contenere informazioni sensibili memorizzate in modo completamente leggibile o tramite un'interfaccia funzionale coerente. Tuttavia, ci sono dei limiti:

- Il processo di autenticazione riconosce l'oggetto, non direttamente l'utente;
- Il rischio di smarrimento dell'oggetto è più elevato rispetto alla dimenticanza di una password.

Per affrontare questi limiti, una soluzione comune è l'autenticazione a due fattori, come nel caso dei bancomat. È necessario possedere la carta (oggetto) e conoscere il PIN per utilizzarla, aumentando così il livello di sicurezza.

3.3.1 Smart Token

Lo Smart Token è un dispositivo utilizzato per l'autenticazione che si basa su un PIN e genera un One-Time Password (OTP) accettato una sola volta dal server. È comunemente impiegato per l'autenticazione su siti web, richiedendo la conoscenza di una password oltre all'OTP generato dal dispositivo. Esistono due tipologie principali di token:

- **A pulsante:** Genera un OTP premendo un pulsante. Tuttavia, se il token viene smarrito, chiunque lo trovi potrebbe leggere l'OTP;
- **A PIN:** Richiede l'inserimento di un PIN prima di generare l'OTP, rendendo il passaggio dell'autenticazione attraverso lo smart token più robusto.

Nel primo caso, un sito potrebbe richiedere una password di lunghezza 12 e fornire uno smart token a pulsante. Nel secondo caso, potrebbe richiedere una password di soli 8 caratteri e richiedere un PIN per l'accesso al token. Le due tecnologie sono praticamente equivalenti in termini di sicurezza. Un'alternativa moderna agli smart token è rappresentata dalle app mobili che offrono autenticazione biometrica. Tuttavia, c'è una differenza significativa tra gli smart token e le app: nel primo caso, il software è eseguito su un sistema isolato nel dispositivo, mentre nel secondo caso lo smartphone è connesso al mondo esterno, aumentando potenzialmente le vulnerabilità.

Per quanto riguarda il funzionamento, lo smart token funziona utilizzando una chiave segreta (seme) memorizzata dalla fabbrica. Prende informazioni esterne come il PIN e l'ora per generare tramite un algoritmo di funzione pseudo-casuale (PRF) una one-time password (OTP) tramite un algoritmo pseudo-casuale. La OTP viene visualizzata sul display e rinnovata ogni 30-90 secondi. Lo smart token e il server condividono un algoritmo comune e hanno orologi sincronizzati per garantire che la password generata sia accettata.

In realtà l'autenticazione basata su possesso sarebbe stata aggiunta anche senza OTP, ovvero con un token che generi una password (lunga?) fissa, esattamente come fa una carta magnetica, o come una smartcard, alla quale non viene richiesta computazione (dato che alcuni circuiti sfruttano ancora le smartcard solo in lettura).

Non confondere conoscenza con possesso: possesso di un oggetto tipicamente ovviabile con informazione che rappresenti l'oggetto (stampanti 3D). Il fatto che un token generi una OTP è pertanto un ulteriore miglioramento rispetto al bancomat.

3.4 Autenticazione basata su biometria

L'autenticazione basata su biometria si basa sul possesso di caratteristiche univoche del corpo per confermare l'identità dell'utente. Queste caratteristiche possono essere:

- Fisiche: Come impronte digitali, forma della mano, impronta della retina o del viso;

- Comportamentali: Come la firma, il timbro di voce, la grafia o la dinamica della digitazione.

Anche se tecnicamente meno precisa dei primi due metodi, l'autenticazione basata su biometria è comunque affidabile poiché non esistono due campioni biometrici uguali. Questo metodo si basa sulla misurazione della distanza tra la firma biometrica dell'utente e quella registrata nel sistema, come illustrato nella slide 89. Le impronte digitali sono un metodo di autenticazione storico,

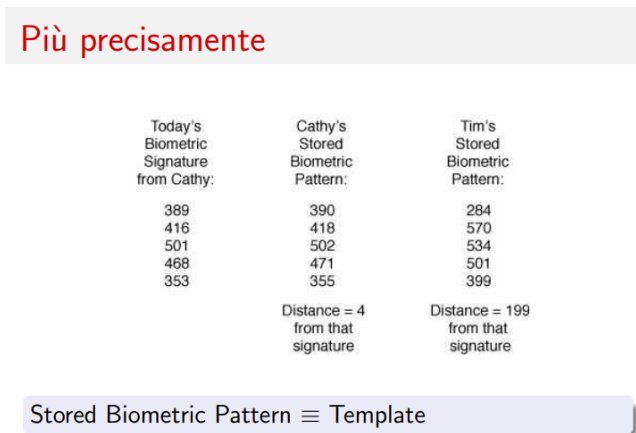


Figure 6: Misurazione

ottenute tradizionalmente con inchiostro, e si suddividono in tre categorie principali: Loop, Arch e Whorl, come mostrato nella slide 92.



Figure 7: Classificazione

4 Cenni di Crittografia

La crittografia è l'arte di trasformare le informazioni da una rappresentazione a un'altra, mantenendo la stessa semantica ma con una sintassi differente. Questa pratica ha radici antiche, risalenti all'antica Grecia, ma ha subito sviluppi cruciali negli anni '70. Prima di questo periodo, la cifratura era principalmente simmetrica, ovvero si utilizzava una sola chiave. Tuttavia, negli anni '70 ha preso forma la cifratura asimmetrica. Ad esempio, l'ENIGMA utilizzava un metodo di cifratura simmetrica con una chiave variabile. Questa tecnologia è ampiamente impiegata su Internet, come nell'utilizzo di HTTPS al posto di HTTP o di SSH invece di Telnet. Cifrare è simile a tradurre in un'altra lingua: si modifica la sintassi (rappresentazione) ma non la semantica:

- Encrypt: Ciao ->Hello
- Decrypt: Hello ->Ciao

Le funzioni di crittografia sono pubbliche e accessibili a tutti. La differenza tra cifratura e traduzione sta nella presenza di un parametro chiamato chiave crittografica. Normalmente, dalla stringa cifrata non è possibile estrarre le informazioni contenute.

Un esempio di cifrario è il Cifrario di Cesare (spostamento alfabetico):

- Encrypt(m, k): CIAO ->DLBP (dove m = messaggio, k = chiave = 1);
- Decrypt(m, k): DLBP ->CIAO.

Il compito della cifratura è nascondere la semantica di un messaggio, ma non la trasmissione del messaggio stesso, il che è significativo in termini di privacy. Romperne la crittografia significa acquisire la semantica senza conoscere la chiave di cifratura (senza dover eseguire il processo di decifratura). Con un po' di fortuna, è anche possibile scoprire la chiave di cifratura, ma è un evento molto raro.

4.1 Crittosistema

Un crittosistema è costituito da una coppia di algoritmi: uno per crittografare e produrre un crittotesto (E), uno per decriptare e produrre un testo in chiaro (D). Ciascun algoritmo richiede due input: un testo e una chiave.

- Dato un testo m e una chiave k , il testo viene codificato come $E(m, k)$, indicato come m_k .
- Dato un crittotesto m_k e una chiave k' , il crittotesto viene decodificato come $D(m_k, k')$, che produce m se specifiche condizioni legano k con k' .
- Non è necessario che k sia uguale a k' .

Quindi l'associazione fra un testo in chiaro e un crittotesto dipende dallo specifico crittosistema e dalla chiave scelta. Quando si ha un crittotesto m_k , non è essenziale che si possiedano le caratteristiche per decifrare il messaggio, ovvero applicare l'algoritmo di decrittazione. Decifrare con successo significa scoprire il testo in chiaro di un messaggio cifrato. La chiave può essere la stessa usata per crittografare (in algoritmi simmetrici) oppure la sua inversa (in crittoalgoritmi moderni, ovvero asimmetrici).

4.2 Crittografia in rete

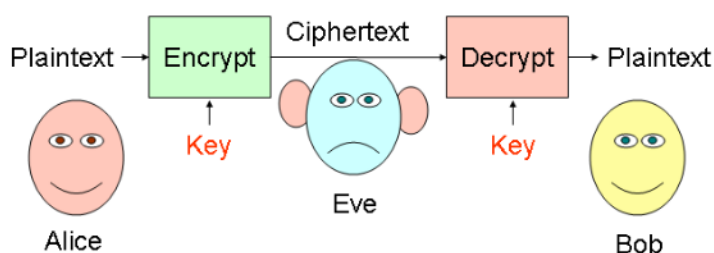


Figure 8: In rete

Assunzione fondamentale: il crittotesto va considerato come pubblico. Le chiavi crittografiche tipicamente vanno mantenute segrete. Esempio: se Alice vuole inviare un messaggio cifrato a Bob e solo Alice e Bob conoscono la chiave o le chiavi crittografiche solo loro potranno comunicare, Eve non conoscendo la chiave crittografica può solo provare a decifrarlo, anche tramite brute force, ma non c'è la certezza di riuscire ad effettuare ciò con successo.

4.3 Segretezza di una chiave

Se la lunghezza del segreto è k , ci sono 2^k possibili combinazioni per generare la chiave di cifratura, e la probabilità di indovinarla è $\frac{1}{2^k}$. È quindi fondamentale avere un valore k elevato, spesso di 256 bit (come nell'AES 256). La probabilità di rivelazione della chiave di un algoritmo crittografico non supera mai $\frac{1}{2^k}$.

4.4 Crittografia simmetrica

La crittografia simmetrica comporta la decifrazione del testo utilizzando la stessa chiave impiegata per cifrare il messaggio. È possibile decifrare correttamente il messaggio solo quando $k' = k$.

Equazione di correttezza

- $D(E(m, k), k) = m$: La decrittazione utilizzando k restituisce m ;
- La condizione $\forall k'. k' \neq k \rightarrow D(E(m, k), k') \neq m$ indica che se $k' \neq k$, non otteniamo il messaggio originale m .

Se un aggressore tenta di decifrare utilizzando chiavi casuali, non otterrà il messaggio corretto.

4.4.1 Crittografia simmetrica in rete

Una chiave a lungo termine, dal punto di vista della sua validità, non è un segreto che cambia frequentemente, come il numero della nostra carta di credito o la nostra password che viene cambiata ogni circa 6 mesi. Un segreto che rimane valido per molto tempo è di grande importanza; più è lungo il periodo di validità, maggiore è l'importanza del segreto. Immagina che Alice abbia una

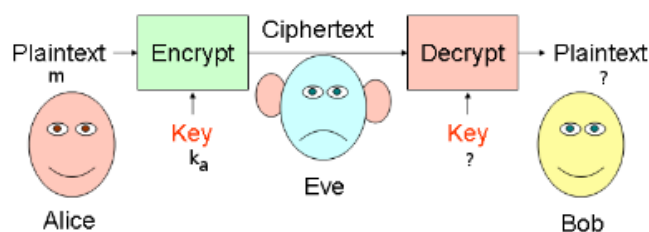


Figure 9: Segretezza

chiave a lungo termine e voglia inviare un messaggio in modo confidenziale, quindi lo cifra con questa chiave. Anche se Eve intercettasse il criptotesto, non riuscirebbe a decifrarlo con successo, garantendo così la confidenzialità. Quando Bob riceverà il criptotesto, proverà a decifrarlo ma dovrà conoscere la chiave. Supponiamo che esista un modo sicuro per trasmettere la chiave in modo confidenziale. Tuttavia, non è necessario che Alice venga a conoscenza della chiave di Bob. Non c'è una connessione diretta tra la chiave cifrata e la password, e non è una buona pratica trasferire la chiave a lungo termine. La differenza principale sta nella lunghezza: le password sono generalmente corte (circa 8 caratteri, 64 bit), mentre le chiavi di cifratura sono molto più lunghe (256 bit). Una chiave a breve termine è condivisa tra Alice e Bob solo per una sessione specifica. Il suo valore è contingente e contestualizzato, quindi è necessario un modo sicuro per trasmetterla. Non possiamo inviarla in chiaro, altrimenti sarebbe facilmente intercettabile, compromettendo così la sicurezza della chiave di cifratura.

4.4.2 Limiti della crittografia simmetrica

Se Alice non vuole divulgare la sua chiave di cifratura a Bob, si può inviare una chiave a breve termine (k_a a B) che può essere condivisa tra i due utenti.

Scalabilità della soluzione: Se Alice desidera comunicare con Bob, necessita di una chiave di sessione specifica; se invece vuole comunicare con Charlie, avrà bisogno di un'altra chiave di sessione, diversa da quella precedente. Ogni coppia di utenti richiede una chiave di sessione unica.

Ma come si può condividere la chiave di sessione in modo sicuro e confidenziale?

4.5 Crittografia asimmetrica

Nella crittografia asimmetrica, a differenza di quella simmetrica:

- Ogni chiave k ha la sua chiave inversa k^{-1} ;
- Non è possibile ricavare k da k^{-1} e viceversa.

Per decifrare con successo un testo cifrato con la chiave k , è necessario utilizzare la sua chiave inversa k^{-1} .

Equazione di correttezza:

- $D(E(m, k), k^{-1}) = m$;
- $\forall k'$, dove $k' \neq k^{-1}$, si ha $D(E(m, k), k') \neq m$.

Ma come si generano le chiavi di cifratura? Il generatore di chiavi produce simultaneamente la coppia di chiavi. Una metà è tenuta privatamente (chiave privata), mentre l'altra metà è resa pubblica (chiave pubblica).

Esempi di crittosistemi asimmetrici: DSA, RSA con chiave di 1024 bit. La chiave è molto più lunga rispetto a quella simmetrica, rendendola più robusta ma anche richiedendo più risorse computazionali.

Le operazioni di encryption asimmetriche sono computazionalmente più costose rispetto a quelle simmetriche. Di conseguenza, è utile utilizzare sistemi ibridi che sfruttino entrambe le tecnologie, simmetrica ed asimmetrica. Nella crittografia asimmetrica, si presuppone che ogni dispositivo disponga di una coppia di chiavi, una pubblica e una privata.

4.5.1 Crittografia asimmetrica in rete

Se si utilizza la chiave privata per cifrare, è necessario utilizzare quella pubblica per decifrare. Quando Alice decide di cifrare un messaggio, ha a disposizione tre chiavi:

- La sua chiave pubblica;
- La chiave pubblica di Bob;
- La sua chiave privata.

Alice rende nota a tutti la sua chiave pubblica e mantiene segreta la sua chiave privata per cifrare il messaggio. Bob riesce a decifrare il messaggio utilizzando la chiave pubblica di Alice. Tuttavia, poiché la chiave è pubblica, può essere utilizzata anche da un attaccante senza problemi. Se l'obiettivo della sicurezza è l'autenticazione e non la segretezza, Alice cifra con la sua chiave privata e tutti possono decifrare utilizzando la chiave pubblica di Alice. Poiché la confidenzialità del messaggio non è importante e si può dedurre che la chiave privata appartiene ad Alice, si riesce ad autenticarla con successo. Se l'obiettivo della sicurezza è la segretezza, allora Alice sceglierà la chiave pubblica di Bob per cifrare. È necessario conoscere l'inversa della chiave pubblica per poterla decifrare correttamente.

4.5.2 Limite della crittografia asimmetrica

Esistono anche problemi con la crittografia asimmetrica, come possiamo essere certi che la chiave pubblica di Alice appartenga effettivamente ad Alice? Se Alice cifra con la sua chiave privata e Bob decifra il messaggio con una chiave pubblica di un soggetto diverso da Alice, otterrà un messaggio leggibile e comprensibile. Questo significa che Bob autenticcherà X invece di Alice, quindi è necessaria **una certificazione**.

Un altro esempio è quando Alice intende inviare un messaggio confidenziale a Bob utilizzando la chiave pubblica di cifratura di Bob. Tuttavia, Alice potrebbe erroneamente utilizzare la chiave pubblica di un soggetto diverso da Bob e consegnare il messaggio a un altro utente.

4.6 Riepilogo crittografia simmetrica ed asimmetrica

- **Crittografia simmetrica:** Per condividere un messaggio in modo sicuro, si utilizza un sistema di scambio di chiavi protetto o si trasmette la chiave simmetrica in modo sicuro prima di inviare il messaggio. Questo assicura la segretezza del messaggio.
- **Crittografia asimmetrica:** Per verificare il proprietario di una chiave pubblica, è necessaria una certificazione, che può essere ottenuta da un'autorità di certificazione affidabile. Questo processo conferisce autenticità alla chiave pubblica del destinatario.

Utilizzando queste tecniche, è possibile ottenere le proprietà di sicurezza di livello 1 (Segretezza, Autenticazione ed Integrità).

4.7 Crittosistema sicuro

- Sia calcolato $E(m,k)$ per ogni testo m e chiave k ;
- Sia calcolato $D(E(m,k),k^{-1}) = m$ per ogni chiave k' tale che $k' \neq k$ se il crittosistema è simmetrico, o $k' \neq k^{-1}$ se il crittosistema è asimmetrico;
- Allora l'accesso a m non aumenti significativamente la probabilità di un attaccante di indovinare m o sue porzioni.

Crittosistema sicuro se la probabilità di scoprire la chiave crittografica non migliori nonostante la pubblicità dell'algoritmo.

4.8 Hash crittografico

Una funzione crittografica di hash è un algoritmo matematico che converte dati di lunghezza arbitraria in una stringa binaria di dimensione fissa, chiamata valore di hash o message digest. È progettata per essere unidirezionale, rendendo difficile invertirla. Deve essere unica per ogni messaggio, deterministica (lo stesso input produce sempre lo stesso output), e veloce da calcolare. Le

sue proprietà la rendono adatta per firme digitali, codici di autenticazione dei messaggi e altre forme di autenticazione, oltre che per la rilevazione di impronte digitali, dati duplicati e checksum per la corruzione dati.

Caratteristiche delle funzioni di hash:

- Il calcolo dell'hash è molto semplice;
- È impossibile generare il messaggio originale partendo dall'hash;
- Anche cambiando un solo bit nel messaggio, l'hash cambierà completamente, generando codici totalmente diversi.

Le funzioni di hashing e di cifratura sono diverse perché tramite l'hashing non è possibile rigenerare il messaggio originale dall'hash, mentre con la cifratura si possono utilizzare due parametri di input per l'operazione.

4.9 CTSS + Hashing

Negli anni '60, al MIT, il Compatible Time Sharing System (CTSS) è stato introdotto. Le password erano memorizzate in chiaro su file di sistema, ma erano protette da politiche di sicurezza. Si trattava di un'idea circolare: il controllo dell'accesso era basato sull'autenticazione, e viceversa. Tuttavia, si registrarono numerosi attacchi. Per affrontare questa vulnerabilità, venne proposto di potenziare CTSS con una funzione di hash, come sviluppato presso l'Università di Cambridge nel 1967. In questo sistema, il file delle password memorizzava l'hash di ciascuna password, migliorando così la sicurezza complessiva del sistema.

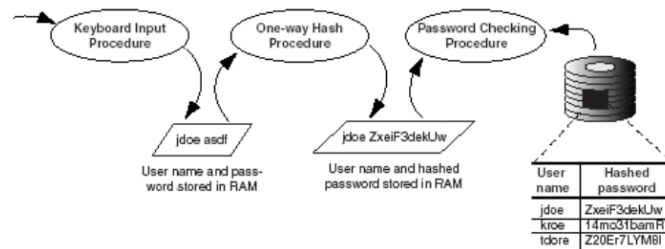


Figure 10: CTSS + Hashing

4.9.1 Salting

Il **salt** è un messaggio casuale, inizialmente di 12 bit ma oggi considerato insufficiente, che viene aggiunto a una password per proteggerla dagli attacchi dizionario prima di applicare l'hash. Unix salvava le password in memoria generando un salt per ciascuna password da memorizzare. Utilizzava la password per codificare una stringa di zeri insieme al salt tramite la funzione di

encryption `crypt(3)` basata su DES. L'uso di una funzione di encryption per generare una funzione hash, come `crypt(3)`, è un esempio storico. Criptava una stringa di zeri estesa con 12 bit di salt randomico, funzionando come una discreta funzione hash per il suo tempo. Questo approccio sfrutta in modo astuto un algoritmo di encryption che soddisfa le caratteristiche di una funzione hash. Il salt complica gli attacchi, mentre il pepper, sebbene fisso, allunga l'input e aumenta la complessità dell'hash, rendendo più difficile la pre-elaborazione degli input possibili.

4.10 Unix: aggiunta di una nuova password

1. Si ottengono 56 bit utilizzando i 7 bit meno significativi dei primi 8 caratteri della password;
2. Questi 56 bit vengono utilizzati come chiave per criptare il salt e una stringa di zeri;
3. La password non è più memorizzata come testo in chiaro, ma viene utilizzato il segreto da proteggere come chiave anziché come testo in chiaro.

Non viene utilizzato l'hashing.

4.10.1 Tabella Rainbow

Le tabelle rainbow sono un metodo di ottimizzazione per gli attacchi di forza bruta contro le funzioni di hash, mentre l'attacco per invertire l'hash sfrutta queste tabelle precalcolate per trovare corrispondenze tra hash target e valori originali.

4.11 Unix: verifica di una password

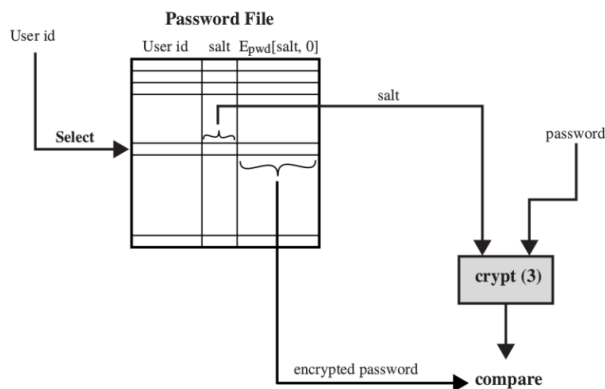


Figure 11: Verifica di una password

Una misura che fornisce segretezza ed autenticazione è l'hash crittografico, come crypt3, che considera solo i primi 8 caratteri della password (è consigliabile utilizzare password più lunghe). Questo metodo è definito tramite una primitiva crittografica. Le tabelle rainbow sono una pre-tabulazione degli hash ottimizzata in termini di spazio, utilizzata per invertire gli hash. Durante la verifica di una password, il programma di autenticazione cerca nella tabella il salt associato all'utente e, insieme alla password, calcola l'hash crittografico. Se il risultato corrisponde a quanto registrato nella tabella, l'utente viene autenticato. Tuttavia, finora non abbiamo menzionato nulla riguardo all'integrità.

4.12 Freshness (Essere recente)

Attributo, caratterizzazione, di almeno due proprietà fondamentali della sicurezza: confidenzialità ed autenticazione. Più è recente un segreto maggiore sarà l'affidabilità, importante quindi cambiare spesso la password. La freshness dell'autenticazione, quindi, è **fondamentale**. Facciamo un esempio di autenticazione bancaria:

1. John si autentica all'accesso;
2. John effettua una operazione finanziariamente sensibile (operazione dispositiva) tramite 2FA (Normativa PSD2) ulteriore misura di sicurezza;
3. La banca onora la richiesta 2.

Facciamo un esempio più specifico, in base all'esempio precedente:

1. John si autentica alle 10:00;
2. John effettua una richiesta di bonifico alle 10:15, la banca dovrebbe concedere questa operazione?

Come facciamo a capire che la richiesta di bonifico provenga dallo stesso client che si è autenticato alle 10:00?

Cosa fa la banca alla richiesta dell'autenticazione?

1. Controllo dell'identità di John;
2. Se il controllo va a buon fine si crea un nuovo ID di sessione;
3. Inserimento dell'ID di sessione all'interno dei cookie di sessione;
4. Invio del cookie al client;
5. Memorizzazione del cookie di sessione

Ogni qualvolta si voglia effettuare un'operazione dispositiva si invia il session cookie, viene verificato ed autorizzato il movimento. Session cookie fondamentale dal punto di vista della segretezza, esiste un protocollo (TLS) che gestisce l'autenticazione ed i cookie di sessione. Chiunque sia in possesso del session ID

può operare a nome di chi si è autenticato.

Perché il session ID? Perché altrimenti l'utente dovrebbe autenticarsi ad ogni operazione effettuata.

Nonce: Number Only Once, numero random utilizzato a livello HTTP che viene generato solo una volta, il session ID è un nonce.

Che caratteristiche deve avere il Session ID? Il session ID è fondamentale per la sicurezza, poiché permette agli utenti di eseguire operazioni senza doversi autenticare ad ogni passaggio. Deve essere unico nel tempo e sufficientemente grande da evitare attacchi brute force. Un attacco che sfrutta la mancanza di freshness è il replay attack, in cui un attaccante registra e riproduce una sequenza di azioni per ottenere un vantaggio illegittimo. Alcune misure sono il Timestamp (marcatore temporale) e la Nonce (numero random che è usato solo una volta). Approfondiremo queste due misure nella prossima parte.

5 Protocolli di sicurezza basilari

5.1 Protocolli basilari per la freshness

- Timestamp: lo inserisce chi *deve* dare garanzia di freshness. Quindi inserisce il timestamp (potrebbe barare), associandolo al messaggio target in maniera affidabile (altrimenti chiunque altri potrebbe attaccare). Gli orologi distribuiti devono essere sincronizzati;
- Nonce: lo inserisce chi *vuole* ricevere garanzia. Quindi chi vuole la garanzia di freshness pubblica la nonce e aspetta traffico che la citi, il quale risulterà posteriore all'istante di creazione della nonce, pur potendo citare materiale vecchio. Non servono orologi sincronizzati.

5.2 Dalla crittografia alla sicurezza

Per ottenere le tre proprietà fondamentali della sicurezza - segretezza, autenticazione ed integrità - i protocolli di sicurezza utilizzano modelli crittografici come il modello Dolev-Yao, che analizzano i casi peggiori di attacco, assumendo che la crittografia funzioni. Per quanto riguarda l'autenticazione e l'integrità, si utilizza il timestamp, un marcatore temporale che è utile per verificare la freshness, calcolando la differenza tra l'orario corrente e l'orario di generazione del timestamp. Tuttavia, è fondamentale considerare che il timestamp diventa critico e sensibile se alterato nel traffico o falsificato dal mittente. Per garantire l'affidabilità del timestamp, è necessario un orologio sincronizzato, ad esempio tramite il Network Timing Protocol (NTP), che assicura la sincronizzazione dell'orologio.

5.3 Sintassi dei messaggi crittografici



Figure 12: Sintassi

Come autenticare un messaggio concatenato? Posso io ricevente avere una garanzia di autenticazione sul messaggio, posso derivare la proprietà che il messaggio sia autentico? **NO**.

Facciamo un esempio di messaggio concatenato:

Alice riceve un messaggio concatenato (m, n) che non è possibile autenticare, perchè nell'invio del messaggio tra Alice e Bob l'attaccante Charlie potrebbe modificare n con n' . Dunque:

Alice --- (m, n) --- $\rightarrow$ *Bob*

Attacco: *Alice* --- (m, n) --- *Charlie* --- (m, n') --- $\rightarrow$ *Bob*

5.4 Sintassi di un protocollo di sicurezza

L'etichetta del mittente diventa irrilevante, es: IP mittente nel pacchetto può essere modificato per effettuare un attacco, non è affidabile e non garantisce una misura di autenticazione. A garantire l'autenticazione è il traffico stesso tramite crittografia, sappiamo che la crittografia garantisce autenticazione.

1. *Alice* $\rightarrow$ *Bob* : A, N_a ;
2. *Bob* $\rightarrow$ *Alice* : $N_{ak_b^{-1}}$

5.5 Protocolli basilari per la segretezza

Per segretezza si intende la *segretezza del messaggio m per Alice e Bob*.

5.5.1 Crittografia simmetrica

Supponiamo che esista una chiave di sessione k_{ab} condivisa tra Alice e Bob e solo da loro. Il messaggio è confidenziale solo in virtù di questa condizione: Alice invierà a Bob un messaggio cifrato con la chiave k_{ab}

1. $Alice \longrightarrow Bob : m_{k_{ab}}$

5.5.2 Chiave asimmetrica

In questo caso, i prerequisiti sono 2:

- Bob deve avere una chiave privata valida e certificata (sicura, non scaduta);
- Alice deve poter verificare che la chiave pubblica sia di Bob

1. $Alice \longrightarrow Bob : m_{k_b}$

5.6 Protocolli basilari per l'autenticazione

5.6.1 Crittografia simmetrica

I prerequisiti sono 2:

- Prerequisito 1: Chiave condivisa solo tra Alice e Bob;
- Prerequisito 2: Bob deve poter verificare l'assunzione 1;

1. $Alice \longrightarrow B : \text{"Sono io!" } k_{ab}$

La differenza tra l'Assunzione (prerequisito) 1 e l'Assunzione 2 è che nell'assunzione 1 non è specificato chi deve sapere il contenuto, nell'assunzione 2 si specifica che deve essere solo B a sapere il contenuto, l'assunzione 2 permette a B di rilevare il mittente del messaggio e quindi di autenticarlo.

5.6.2 Crittografia asimmetrica

Anche qui i prerequisiti sono 2:

- Prerequisito 1: Alice deve avere una chiave privata valida;
- Prerequisito 2: Bob deve poter verificare che la chiave k_a sia di Alice

1. $Alice \longrightarrow Bob : \text{"Sono io!" } k_{a-1}$

Serve anche questa volta certificazione, chi decifra decide che il mittente è il proprietario della chiave pubblica. Obiettivo è l'autenticazione, non mi interessa il contenuto del messaggio, posso scrivere ciò che voglio.

5.7 Combinare segretezza ed autenticazione

L'obiettivo quindi sarà la segretezza del messaggio m per *Alice* e *Bob*, autenticazione di *Alice* con *Bob*.

5.7.1 Crittografia simmetrica

- Prerequisito 1: Chiave k_{ab} deve essere condivisa solo tra *Alice* e *Bob*;
- Prerequisito 2: *Bob* deve poter verificare il prerequisito 1
 1. $Alice \rightarrow Bob : m_{k_{ab}}$

5.7.2 Crittografia asimmetrica

- Prerequisito 1: *Bob* deve avere una chiave privata valida;
- Prerequisito 2: *Alice* deve poter verificare che la chiave k_b sia effettivamente di *Bob*;
- Prerequisito 3: *Alice* deve avere una chiave privata valida;
- Prerequisito 4: *Bob* deve poter verificare che la chiave k_a sia effettivamente di *Alice*
 1. $Alice \rightarrow Bob : \{m_{k_a^{-1}}\}_{k_b}$
 2. $Alice \rightarrow Bob : \{m_{k_b}\} : k_a^{-1}$

5.8 Come ottenere integrità

L'integrità è sempre a rischio, utilizziamo le funzioni hash con la caratteristica che l'entropia dell'input sia magnificata nell'entropia dell'output. Utilizzo del checksum sui download, quando scarichiamo un file e ci viene fornito l'hash bisogna verificarlo per ottenere la proprietà di integrità. Questo protocollo ha delle vulnerabilità: l'attaccante potrebbe cambiare m in m' , **ricalcolare l'hash** di m' ed inviare il tutto al destinatario. Quando scarichiamo un file $(m, h(m))$ Alice invia a Bob solamente m , $h(m)$ è un semplice testo visualizzato a video, se l'attaccante cambia m in m' deve anche violare il sito modificando $h(m)$ in $h(m')$ per far credere a Bob di aver scaricato il file corretto. Come impedire che l'attaccante riesca a cambiare $h(m)$ in $h(m')$? Possiamo cifrare l'hash con la chiave privata del mittente.

1. $Alice \rightarrow Bob : m, h(m)_{k_a^{-1}}$

Una volta ricevuto il messaggio, *Bob* deve:

- applicare la funzione hash alla prima componente;
- decodificare la seconda componente;
- confrontare i due risultati;

Ottiene garanzia di integrità se e solo se i due risultati combaciano. Questo protocollo prende il nome di **firma digitale**.

1. $sign_A(m) = m, h(m)_{k_a^{-1}}$

5.9 La firma digitale

Assicura sia l'integrità che l'autenticazione. Non esiste una soluzione che garantisca l'integrità senza l'autenticazione. Sfruttiamo anche la proprietà di autenticazione, cifrando con la chiave privata del mittente. Effettuare una firma digitale significa utilizzare un paio di algoritmi, uno per creare la firma e uno per verificarla. Si invia il documento in chiaro insieme alla cifratura del digest. Questo processo combina autenticazione e integrità, ma non confidenzialità e integrità. Nessuno può firmare per conto di un altro, ma tutti possono verificare la firma digitale, poiché tutti conoscono la chiave pubblica del mittente.

5.9.1 Creare una firma

Cleartext $\rightarrow$ *Hashing* $\rightarrow$ *Digest*

Digest $\rightarrow$ *Encrypt with sender's private key* $\rightarrow$ *Encrypted Digest*

Cleartext + *Encrypted digest* = *Digital signature*

5.10 Verificare una firma

Cleartext + *Encrypted digest*

Encrypted digest $\rightarrow$ *Decrypt with sender's public key* $\rightarrow$ *Digest 1*

Cleartext $\rightarrow$ *Hashing* $\rightarrow$ *Digest 2*

Digest 1 = *Digest 2* ?

yes = *valid*

no = *invalid*

5.11 Protocolli basilari per l'integrità

Si parla dell'integrità del messaggio m nella trasmissione da *Alice* e *Bob*. In questo caso, la crittografia simmetrica verrà omessa.

5.12 Crittografia asimmetrica

- Prerequisito 1: *Alice* deve avere una chiave privata k_a^{-1} ;
- Prerequisito 2: *Bob* deve poter verificare che la chiave k_a sia effettivamente di *Alice*

1. *Alice* $\rightarrow$ *Bob* : $sign_A(m)$

5.13 Combinare tutte le proprietà di livello 1

La firma digitale risolve il problema delle falsificazioni. La firma tradizionale offre solo un livello di autenticazione ed è facilmente falsificabile. Per quanto riguarda la segretezza del messaggio (m) tra A e B, l'autenticazione di A con B e l'integrità di m durante la trasmissione da A a B, è necessario considerare che tutti i protocolli basilari presentati sono vulnerabili agli attacchi di ripetizione e non incorporano meccanismi di freschezza. Prima di esaminare protocolli più avanzati, manteniamo due promesse fondamentali:

1. Utilizzo della crittografia simmetrica: condivisione di un segreto iniziale.
2. Utilizzo della crittografia asimmetrica: certificazione.

Per ottenere ciò, verranno impiegati protocolli dedicati:

1. Per la crittografia simmetrica: Diffie-Hellmann (DH) o RSA Key Exchange.
2. Per la crittografia asimmetrica: Public-Key Infrastructure (PKI).

5.14 Protocollo Diffie-Hellman

1. Alice e Bob concordano due parametri pubblici molto grandi, α e β , correlati tra loro;
2. Alice genera un numero casuale privato, X_a . Successivamente calcola $Y_a = \alpha^{X_a} \bmod \beta$ (pubblico);
3. Bob genera un numero casuale privato, X_b . Successivamente calcola $Y_b = \alpha^{X_b} \bmod \beta$ (pubblico);
4. Alice e Bob eseguono lo scambio di chiavi;
5. Bob, alla ricezione, calcola $Y_a^{X_b} \bmod \beta$;
6. Alice, alla ricezione, calcola $Y_b^{X_a} \bmod \beta$.

In questo modo si ottiene lo stesso numero, che rappresenta la **chiave di sessione condivisa** tra Alice e Bob. Questo protocollo **NON** è legato alla crittografia asimmetrica, ma la crittografia asimmetrica si ispira ad esso. Le Y sono pubbliche mentre le X sono private. Questo schema presenta delle vulnerabilità? Esiste un'operazione che consenta a chiunque intercetti le Y di derivare le X ? Per calcolare l'esponente, sarebbe necessario applicare l'operazione del logaritmo base α all'espressione α^{X_a} , il che è uguale a X_a . Tuttavia, essendo applicata anche l'operazione modulo, bisognerebbe effettuare l'operazione di logaritmo discreto, che risulta essere intrattabile e onerosa per numeri α e β molto grandi. In pratica, dato Y_a , non è possibile ricavare X_a in tempi ragionevoli. *N.B.:* Non vi è autenticazione tra Alice e Bob; sebbene la confidenzialità sia assicurata, l'assenza di autenticazione non garantisce la confidenzialità, poiché non possiamo essere certi di comunicare con Bob ma potremmo trovarci a comunicare con Charlie.

5.15 Attacco MiTM su Diffie-Hellman

Presupponiamo che l'attaccante C abbia una sua coppia X_c e Y_c standard. Quando $Alice$ e Bob eseguono il protocollo di scambio, $Charlie$ blocca e altera. Vediamolo meglio passo passo:

- $Alice$ invia a Bob Y_a ;
- $Charlie$ intercetta Y_a e invia a Bob Y_c ;
- Alla ricezione, Bob calcola $Y_c^{X_b}$ mod beta;
- Bob invia ad $Alice$ Y_b ;
- $Charlie$ intercetta Y_b e invia ad $Alice$ Y_c ;
- Alla ricezione, $Alice$ calcola $Y_c^{X_a}$ mod beta.

I problemi associati a Diffie-Hellman sono di due tipi. Se l'attaccante avesse semplicemente inviato le sue proprie Y_c senza conoscere Y_a e Y_b , non sarebbe stato in grado di completare l'attacco e di calcolare k_1 (che sarebbe $Y_c^{X_a}$) e k_2 (che sarebbe $Y_c^{X_b}$).

Il problema sta proprio nel fatto che questo protocollo non usa alcuna tecnica di **autenticazione**, che permette a $Charlie$ di fare un duplice attacco di autenticazione, ovvero quando modifica le richieste con Y_c . L'altro problema è che il protocollo non usa alcuna tecnica di **segretezza**, dunque $Charlie$ fa un duplice attacco di segretezza, ovvero che utilizza in maniera indebita Y_a e Y_b .

5.16 Irrobustire Diffie-Hellman

Il primo ricorso ufficiale alla crittografia asimmetrica è una delle mosse per rafforzare Diffie-Hellman. Una delle due versioni di Diffie-Hellman è utilizzata in IPsec. L'approccio congiunto tra crittografia asimmetrica e simmetrica, che è più veloce ed efficiente, è un'altra strategia di irrobustimento.

Per proteggere contro l'attacco alla confidenzialità, si può adottare la cifratura con la chiave pubblica del destinatario. In questo modo, anche se l'attaccante intercetta i valori Y_a e Y_b , non sarà in grado di decifrarli senza la chiave privata del destinatario.

Per prevenire l'attacco all'autenticazione, si può utilizzare la codifica con la chiave privata del mittente. Anche se l'attaccante riuscisse a intercettare Y_a e Y_b , non sarebbe in grado di fornire ai destinatari la propria Y_c , poiché dovrebbe cifrarla con la chiave del mittente, che l'attaccante non possiede.

5.17 RSA Key Exchange

Alternativa a Diffie-Hellman basata su crittografia asimmetrica. Nel mondo asimmetrico esiste ancora il problema della certificazione. Il mittente inventa la chiave di sessione randomicamente e la manda in una busta digitale

1. $Alice \rightarrow Bob : \{k_{ab}\}_{k_b}$

Si nota che anch'esso manca di autenticazione, ma l'attaccante non può calcolare la chiave di sessione. Affinché il protocollo funzioni, *Alice* deve innanzitutto procurarsi il **certificato** (della chiave pubblica) di *Bob*.

5.18 Personal Identifiable Information (P.I.I.)

Il nome e il cognome non identificano univocamente una persona; sono necessarie ulteriori informazioni, come il codice fiscale. Nella carta di identità, un elemento che attesta l'autenticità è il numero di carta di identità, che autentica anche il produttore della carta di identità. Gli elementi fondamentali in una carta di identità sono almeno tre:

- Anagrafica;
- Fotografia;
- Produttore.

Queste tre informazioni devono essere prodotte per garantire l'integrità e per essere riconoscibili e non sostituibili. A e K_a rappresentano l'anagrafica e la chiave privata di A , mentre CA (Autorità di Certificazione) è l'ente che emette il certificato. Il compito della CA è quello di mettere insieme, tramite cifratura, $\{A, k_a\}$, cifrati con la chiave inversa di $CA(K_{ca-1})$. Questo avviene attraverso una firma digitale primitiva crittografica, che rappresenta una misura che combina autenticazione ed integrità. Esiste una catena di certificati e, di conseguenza, una catena di atti di fiducia che culmina con la root, che nel caso della carta di identità è il ministro degli interni. I certificati vengono firmati sempre dall'ente che si trova più in alto nella catena, fino ad arrivare al livello 0, in cui il certificato viene firmato dallo stesso ente che lo produce. Sul certificato radice, facciamo un atto di fede.

RCA: Root Certification Authority (ministero degli interni nel nostro caso).

5.19 Certificazione (crittografia asimmetrica)

Un certificato è paragonabile a una carta d'identità e possiede caratteristiche di grande valore, tra cui la fiducia. Un'autorità di certificazione vende fiducia, e dietro le CA ci sono spesso consorzi di banche. Il valore di un certificato deriva dall'autorevolezza di chi lo emette e dalla libertà di chi lo riceve nel decidere se accettarlo o meno. Questa certificazione serve ad associare una chiave pubblica al suo legittimo proprietario, ovvero al proprietario della metà privata. L'associazione è realizzata da un **certificato digitale** (anche quello cartaceo fa un'associazione, fra foto e anagrafica). I certificati seguono uno standard chiamato **X.509**. L'origine dell'albero delle certificazioni è autocertificante, quindi rappresenta un atto di fede. I fornitori di identità digitale sono regolamentati dalla legge italiana. Gli URL hanno bisogno di questi certificati, e ci sono problematiche relative alla certificazione dei wildcard per coprire anche i sottodomini. Ad esempio, se unict.it è certificato, questo implica che anche

dmi.unict.it lo sia? Non necessariamente, poiché non esiste un'implicazione logica diretta. TLS è un protocollo crittografico che utilizza una combinazione di cifratura simmetrica e asimmetrica, e richiede la certificazione, ma la radice della certificazione richiede un atto di fede. Come fa il browser a sapere chi è l'autorità di certificazione radice? Ogni browser possiede un database di certificati TLS. Se il browser possiede la root, può certificare l'intera catena di certificati. Ogni browser fornisce un insieme diverso di certificati di root, che possono variare. Senza autenticazione, i contenuti di una pagina web non possono essere considerati affidabili. Ma dove risiede la misura dell'autenticazione? I browser autenticano un sito tramite il simbolo del lucchetto verde, e l'utente deve fidarsi della genuinità del browser. La conferma dell'autenticazione viene comunicata dall'utente al browser attraverso il lucchetto chiuso, affermando che l'URL corrisponde a ciò che dichiara e che il sito è autentico. Tuttavia, l'autenticazione non garantisce la qualità; alcuni siti potrebbero essere compromessi da vulnerabilità, rendendoli potenzialmente pericolosi.

5.20 Certificato X.509

Il formato ufficiale dei certificati X.509 comprende:

- Versione;
- Numero di certificato;
- Identificativo della firma;
- Nome del fornitore del certificato;
- Periodo di validità (con possibilità di revoca in caso di compromissione della chiave privata o altri eventi, con una relativa fiducia nel certificato);
- Nome del soggetto (*Alice* nel nostro caso);
- Informazioni pubbliche (K_a);
- Identificativo dell'emittente (CA);
- Identificativo del soggetto (A);
- Estensioni;
- Firma (che può comprendere sia la coppia di chiavi che solo la seconda componente, ovvero la cifratura del digest della parte chiave).

La firma, essenziale per la verifica dell'autenticità del certificato, può includere sia la coppia di chiavi che solo la cifratura del digest della parte chiave.

5.21 Public-Key Infrastructure (PKI)

L'infrastruttura a Chiave Pubblica è una struttura ad albero composta da certificazioni. Ad esempio, il rettore certifica le firme dei direttori di dipartimento, i direttori di dipartimento a loro volta certificano i propri docenti. Quando si deve certificare una "foglia" (ossia un'entità finale, come un singolo docente), non è necessario certificare l'intero albero, ma solo il percorso a ritroso dalla foglia fino alla radice. Questo processo richiede uno sforzo con complessità logaritmica.

5.22 Chain of Trust (Catena di fiducia)

Il certificato di root (RCA) è l'**unica entità autocertificata**, e lo store dei certificati di root è vulnerabile e suscettibile agli attacchi. Se un aggressore riesce maliziosamente ad inserire un certificato di root ($\{...RCA...k_{rca}...\}_{k_{rca}-1}$), questo equivale a convertire la fiducia in un attacco diretto (diventerebbe $\{...badRCA...k_{rca}...\}_{k_{badrca}-1}$). A sua volta, *badRCA* potrebbe emettere certificati per gli URL di sua proprietà, generando una nuova coppia di chiavi e firmando con la chiave di root. Fare un attacco del genere era relativamente semplice qualche anno fa, poiché l'importazione non richiedeva autorizzazioni di amministratore. Oggi, il metodo di alzare i privilegi rende l'attacco più complesso, ma rimangono possibili vulnerabilità pratiche (bug) che potrebbero consentire l'attacco. La trust base del browser è estensibile per scopi funzionali, quindi è possibile aggiungere certificati. Qual è la differenza tra un certificato di root di Verisign e uno homemade? Sintatticamente sono identici. Come vengono gestiti i sottodomini? Usando i "wildcards", il fornitore offre un numero limitato di certificati che coprono tutti i suoi sottodomini. La perdita di fiducia si propaga verso il basso lungo la catena.

5.23 Segretezza vs Fiducia

Se ipotizziamo che un certificato sia compromesso, ad esempio se un'autorità di certificazione perde la propria chiave privata, ciò non implica automaticamente che lo stesso problema si propaghi ai suoi sotto-alberi, ovvero ai certificati emessi da essa. I problemi di sicurezza, come la perdita della chiave privata, non si estendono oltre la propria entità; tuttavia, i problemi di fiducia si propagano. Questo significa che se un'autorità di certificazione perde la fiducia a causa di una compromissione, anche i certificati emessi da essa rischiano di non essere più considerati affidabili, causando una diffusione del problema lungo la catena di fiducia.

5.24 Certificati auto-emessi (self-issued)

Quando il browser non riesce a verificare l'identità di un sito web, l'utente ha tre opzioni:

- Non accettare il certificato e non connettersi al sito web;

- Accettare temporaneamente il certificato, rendendolo valido solo per la sessione corrente;
- Accettare il certificato in modo permanente.

Se si accetta il certificato temporaneamente, si espone a numerose possibilità di attacchi di "man-in-the-middle". Ad esempio, ogni volta che si tenta di effettuare l'accesso, le richieste possono essere intercettate sulla rete locale e reindirizzate verso un sito fasullo non certificato. Questo crea un rischio concreto di spoofing. Accettare il certificato in modo permanente, d'altra parte, equivale a fare un atto di fede nei confronti del sito web, poiché si assume che tutte le future connessioni saranno sicure. In termini di impatto, entrambe le opzioni sono equivalenti, ma la verosimiglianza di un potenziale rischio è differente. Fortunatamente, questo scenario è sempre meno comune per diverse ragioni:

- **Costi ridotti della certificazione:** grazie a servizi come Let's Encrypt, che offre certificati gratuiti, e alla diminuzione dei costi complessivi della certificazione, ottenere un certificato è diventato più accessibile economicamente;
- **Facilità di ottenimento dei certificati:** Let's Encrypt, supportato da un consorzio di autorità di certificazione, ha semplificato notevolmente il processo di ottenimento dei certificati, rendendolo disponibile per tutti i tipi di siti web, compresi i sottodomini;
- **Tipologie di certificati:** Anche se Let's Encrypt offre principalmente certificati di tipo Domain Validated (DV), che richiedono solo una validazione del dominio, ci sono altre tipologie di certificati come quelli di tipo Extended Validation (EV) e Organization Validated (OV) che offrono livelli di validazione più elevati, specialmente per siti sensibili come le banche;
- **Differenze visive nei browser:** Alcuni browser forniscono indicazioni visive diverse per i diversi tipi di certificati, come le barre degli indirizzi colorate o i segni di sicurezza, che aiutano gli utenti a identificare i siti affidabili;
- **Certificate Transparency:** Questa tecnologia aiuta a garantire che i certificati siano autentici e rende più difficile per gli aggressori utilizzare certificati fraudolenti;
- **Rischi associati ai cambi di certificato:** Il cambio repentino di certificato può essere considerato sospetto e potrebbe essere un segno di un possibile attacco in corso.

In generale, l'aumentata disponibilità e accessibilità dei certificati, insieme a migliori pratiche di sicurezza e tecnologie aggiuntive come Certificate Transparency, hanno contribuito a rendere il web più sicuro e affidabile per gli utenti.

5.25 Certificate Revocation List (CRL)

La necessità di revocare un certificato può sorgere in vari contesti, come la perdita della chiave privata o il cambio del Subject Identifier. Revocare un certificato è un'operazione altrettanto sensibile quanto rilasciarlo. Richiede la stessa autorità e solo l'entità che ha emesso il certificato può revocarlo. Una Certificate Revocation List (CRL) è una lista di certificati revocati. Essenzialmente, una CRL è un oggetto statico, una sequenza di bit, un messaggio. Se il browser non viene informato della CRL, potrebbe validare certificati che sono stati revocati. Tuttavia, attualmente non esiste un protocollo definito per la distribuzione delle CRL. In passato, le CRL venivano scaricate da entità come Verisign e integrate nel browser. Oggi, esiste un protocollo chiamato Online Certificate Status Protocol (OCSP), che funziona verificando lo stato di validità di un certificato direttamente presso l'ente di certificazione ogni volta che un utente cerca di accedere a un URL. Questo processo aiuta a garantire che i certificati revocati non siano considerati validi durante le connessioni web.

6 Protocolli di Sicurezza storici

6.1 Meccanismo di challenge-response

In questa sezione tratteremo come impostare un protocollo di autenticazione di Bob con Alice con aggiunta di freshness, utilizzando un meccanismo di challenge-response, si seguono i seguenti passaggi:

- **Invio della challenge da parte di Alice:** Alice invia a Bob una nonce cifrata con la chiave pubblica di Bob, $\{N_a\}_{K_b}$;
- **Risposta di Bob:** Bob riceve il messaggio cifrato da Alice e lo decifra con la sua chiave privata, ottenendo la nonce N_a . Successivamente, Bob invia la nonce in chiaro ad Alice.

In questo protocollo, chiunque desideri confermare l'autenticità di Bob deve includere la nonce nella sua risposta e riceverla indietro intatta. Questo garantisce la "freshness" dei dati scambiati durante il processo di autenticazione. Ma come **mutualizziamo la garanzia di freshness** e autenticare Alice con Bob? Si può seguire questo protocollo:

- **Invio della challenge da parte di Bob:** Bob invia ad Alice una nonce cifrata con la chiave pubblica di Alice, $\{N_b\}_{K_a}$;
- **Risposta di Alice:** Bob riceve il messaggio cifrato da Alice e lo decifra con la sua chiave privata, ottenendo la nonce N_b . Successivamente, Bob invia la nonce in chiaro ad Alice.

In questo modo, entrambi Bob e Alice possono essere certi che l'altro abbia ricevuto e utilizzato una nuova nonce, garantendo così la freschezza dei dati scambiati e mutua autenticazione.

6.2 Ottimizzazione protocollo

Partiamo con la premessa che, per capire cosa viene adesso, bisogna stare ben attenti e consapevoli della scrittura crittografica. Possiamo unificare i 2 step centrali dove Bob invia informazione ad Alice, il protocollo passa da:

- Alice invia a Bob una nonce cifrata con la chiave pubblica di Bob $\{N_a\}_{K_b}$ ($A \rightarrow B : \{N_a\}_{K_b}$);
- Bob invia ad Alice il nonce N_a decifrato con la sua chiave privata ($B \rightarrow A : N_a$);
- Bob invia ad Alice una nonce cifrata con la chiave pubblica di Alice $\{N_b\}_{K_a}$ ($B \rightarrow A : \{N_b\}_{K_a}$);
- Alice invia a Bob il nonce N_b decifrato con la sua chiave privata ($A \rightarrow B : N_b$).

a:

- Alice invia a Bob una nonce cifrata con la chiave pubblica di Bob $\{N_a\}_{K_b}$ ($A \rightarrow B : \{N_a\}_{K_b}$);
- Bob invia ad Alice il nonce N_a decifrato con la sua chiave privata ed una nonce cifrata con la chiave pubblica di Alice ($B \rightarrow A : N_a, \{N_b\}_{K_a}$);
- Alice invia a Bob il nonce N_b decifrato con la sua chiave privata ($A \rightarrow B : N_b$).

Questa versione è sufficiente per l'autenticazione con freshness, possiamo però aggiungere un ulteriore livello di sicurezza cifrando tutte le nonce:

- Alice invia a Bob una nonce cifrata con la chiave pubblica di Bob $\{N_a\}_{K_b}$ ($A \rightarrow B : \{N_a\}_{K_b}$);
- Bob invia ad Alice il nonce N_a ed una nonce N_b cifrati entrambi con la chiave pubblica di Alice ($B \rightarrow A : \{N_a, N_b\}_{K_a}$) $K_{ab} = N_a, N_b$ è un segreto condiviso;
- Alice invia a Bob il nonce N_b cifrato con la chiave pubblica di Bob ($A \rightarrow B : \{N_b\}_{K_b}$).

Quando Bob riceve da Alice il primo nonce, non sa a chi replicare il messaggio è necessario quindi aggiungere nel messaggio un riferimento al mittente, il protocollo diventa quindi:

- Alice invia a Bob una nonce cifrata con la chiave pubblica di Bob $\{N_a\}_{K_b}$ insieme ad un riferimento a se stessa per poter ricevere indietro la nonce cifrata ($A \rightarrow B : \{Alice, N_a\}_{K_b}$);
- Bob invia ad Alice il nonce N_a ed una nonce N_b cifrati entrambi con la chiave pubblica di Alice ($B \rightarrow A : \{N_a, N_b\}_{K_a}$) $K_{ab} = N_a, N_b$ è un segreto condiviso;

- Alice invia a Bob il nonce N_b cifrato con la chiave pubblica di Bob ($A \rightarrow B : \{N_b\}_{K_b}$).

6.2.1 Analisi del protocollo di sicurezza

- Il protocollo soddisfa il requisito di autenticazione tra Alice e Bob? Sì;
- Il protocollo soddisfa il requisito di autenticazione tra Bob e Alice? Sì;
- Il protocollo soddisfa la confidenzialità della coppia $\{N_a, N_b\}$? Sì, perché le nonce sono sempre cifrate.

6.3 Scenario d'attacco

Nel tentativo di comunicare con Ive, Alice invia $\{Alice, N_a\}_{K_{Ive}}$. Tuttavia, l'attaccante Dolev-Yao si finge Ive e intercetta il messaggio, ottenendo così la nonce N_a di Alice. L'attaccante, impersonando Ive, manipola il flusso di comunicazione coinvolgendo Bob.

Ive, fingendosi Alice, riutilizza la nonce di Alice e, accanto ad essa, inserisce 'Alice', cifrando il tutto con la chiave pubblica di Bob. Successivamente, Bob invia a Ive la coppia $\{N_a, N_b\}$ cifrata con la chiave pubblica di Alice. Ive, non potendo decifrare il messaggio poiché non possiede la chiave privata di Alice, lo reinoltra ad Alice.

Alice riceve $\{N_a, N_b\}_{K_{Alice}}$ e nota che la nonce N_a è tornata indietro, soddisfacendo così la challenge-response e autenticando erroneamente Ive senza rilevare alcuna anomalia nel protocollo.

Successivamente, Alice estrae la seconda nonce N_b e la rimanda a Ive, appena autenticato, cifrandola con la chiave pubblica di Ive: $\{N_b\}_{K_{Ive}}$. In questo modo, Ive ottiene la nonce N_b cifrandola con la chiave pubblica di Bob, precedentemente nota a Ive, mentre Bob, rivedendo la sua stessa nonce N_b , non rileva anomalie e ritiene di aver autenticato con successo Alice, ignaro del fatto che il suo interlocutore è Ive.

Ora abbiniamo **autenticazione e confidenzialità**:

L'hash al messaggio 3 costituisce una soluzione a questo attacco, poiché, sebbene Ive non sia interessato al contenuto della nonce N_b , la sua mancanza di conoscenza di questa nonce previene l'attacco.

6.3.1 Potenziali soluzioni

Possiamo apportare queste modifiche al protocollo originale.

Primo fix:

- Alice invia a Bob: $\{\{Alice, N_a\}_{K_{Alice-1}}\}_{K_{Bob}}$;

Mettiamo una cifratura con chiave privata nel messaggio 1 è ancora possibile effettuare l'attacco non è quindi un fix corretto.

Secondo fix:

- Alice invia a Bob: $\{\{Alice, N_a\}_{K_{Alice-1}}\}_{K_{Bob}}$
- Bob invia ad Alice: $\{\{N_a, K_b\}_{K_{Bob-1}}\}_{K_{Alice}}$.

Possibile soluzione Alice si accorgerebbe di non star parlando con l'interlocutore corretto.

Terzo Fix:

- Bob invia ad Alice: $\{\{N_a, K_b\}_{K_{Bob-1}}\}_{K_{Alice}}$.

Possibile semplificare il tutto modificando solo il secondo passaggio del protocollo.

Quarto fix:

- Bob invia ad Alice: $\{N_a, N_b, Bob\}_{K_{Alice}}$.

Possibile anche rimuovere il secondo livello di encryption, Alice non riesce ad identificare Eve ed Eve non può spacciarsi per Alice agli occhi di Bob. Il senso del fix è quello di esplicitare il mittente del messaggio all'interno del crittotesto come accade esattamente al passo 1. Da ciò ne deriva che esplicitare l'identità del mittente è rilevante.

Quinto fix:

- Alice invia a Bob: $\{Alice, Bob, N_a\}_{K_{Bob}}$

Nel messaggio 1 non ci sono vulnerabilità, modificare quello non porterebbe alcuna soluzione all'attacco. Il problema dell'attacco sta al passaggio 3 che è il sintomo del problema ed è quindi necessario fixare il messaggio 3, modificando il messaggio 2 informando Alice se eventualmente tutti i controlli non sono andati a buon fine.

6.3.2 Conseguenze dell'attacco

Le conseguenze di questo attacco sono significative. L'attaccante riesce a scoprire la chiave di sessione autenticandosi come Alice presso Bob, il che significa che l'attaccante può impersonare Alice e accedere ai servizi offerti da Bob utilizzando la chiave di sessione rubata. Questo è particolarmente critico nel contesto di un'applicazione web, dove il client si autentica al server tramite un cookie di sessione che contiene le nonces N_a e N_b . Utilizzando questa chiave di sessione compromessa, l'attaccante potrebbe anche cifrare i messaggi, aumentando il potenziale danno dell'attacco. Sebbene abbiamo discusso delle soluzioni per proteggere la nonce N_b , potrebbe sembrare che la protezione della nonce N_a non sia stata considerata. Tuttavia, la considerazione principale è che, poiché Alice condivide la nonce N_a con Eve per la sessione corrente, le sorti di N_a sono meno rilevanti. Bob acquisisce N_a in modo involontario a causa dell'attività fraudolenta di Eve, quindi la policy associata a N_a è che è stata concepita da Alice per

condividerla con Ive. Cambiarlo a un modello di attaccante più generale, come un attaccante generico, rende il quesito sulla confidenzialità di N_a più rilevante. In questo caso, il fatto che Bob abbia acquisito la nonce in modo indesiderato potrebbe sollevare preoccupazioni sulla sicurezza dei dati e sulla privacy degli utenti.

In sintesi, mentre la protezione di N_b è prioritaria, non bisogna trascurare la protezione di N_a , specialmente considerando gli scenari in cui la sicurezza dei dati e la privacy degli utenti sono in gioco.

6.3.3 Vendetta nel modello general attacker

Nel modello di attaccante generico, Bob, scoprendo l'importanza della nonce N_a , potrebbe pianificare una vendetta contro Ive. Supponendo che Alice sia una banca, Bob potrebbe eseguire un attacco come segue:

- Bob si impersona come Ive, utilizzando la chiave di sessione rubata $\{N_a, N_b\}$, e invia a Alice la seguente richiesta:

$\{N_a, N_b, \text{"Trasferisci 20000€ dal conto di Ive al conto di Bob"}\}_{K_{Alice}}$

In questo scenario, Alice (la banca) riceve la richiesta e la interpreta come legittima poiché soddisfa la challenge con la nonce N_b . Di conseguenza, esegue il trasferimento di denaro dal conto di Ive al conto di Bob, credendo che la richiesta provenga da Ive stesso. Questo tipo di attacco rappresenta un grave rischio per la sicurezza e la fiducia nell'ambiente online. Dimostra l'importanza di proteggere sia la nonce N_a che la nonce N_b e sottolinea la necessità di implementare misure di sicurezza robuste per prevenire l'usurpazione delle identità e le frodi finanziarie.

6.4 Protocollo di Woo-Lam

Protocollo a cifratura simmetrica, unidirezionale e di tipo challenge-response, il problema è quello della condivisione della chiave iniziale risolvibile con Diffie-Hellman, una misura tipica è la presenza di un garante delle comunicazioni (TTP: Trusted Third Party). Se tra Alice e Bob è presente un TTP incorruttibile e fidato è possibile agevolare i due interlocutori, una delle ipotesi di lavoro è che il TTP conosca tutte le chiavi a lungo termine degli agenti. Questo equivale a dire che Alice e Bob sono registrati su un server che si chiama TTP che conosce entrambe le loro password. L'obiettivo del protocollo è autenticare unidirezionalmente Alice con Bob, non necessariamente viceversa, a differenza del protocollo precedente dove l'autenticazione è mutua.

1. Passo 1: Alice prova ad autenticarsi con Bob e lo interessa in maniera molto essenzialista, rappresenta un "Hello Message" inviando a Bob un riferimento a se stessa: $A \longrightarrow B : A$ ma chiunque potrebbe dire a Bob di essere Alice;
2. Passo 2: Bob invia ad Alice un challenge, una nonce $N_b : B \longrightarrow A : N_b$;

3. Passo 3: Alice cifra con la sua chiave K_a , il suo segreto a lungo termine, la nonce appena ricevuta: $A \longrightarrow B : \{N_b\}_{K_a}$;
4. Passo 4: Bob riceve questo messaggio o challenge cifrato e non se ne fa nulla in quanto non è in grado di decifrarlo, lo reinoltra quindi al TTP insieme al riferimento arrivato al passo 1. Questa query è cruciale, se nel mezzo Dolev-Yao scombina questo abbinamento fallisce il protocollo, è necessario quindi cifrare entrambi con la sua chiave K_b a lungo termine: $B \longrightarrow TTP : \{A, \{N_b\}_{K_a}\}_{K_b}$, questo non garantisce l'integrità, il TTP decifra con la chiave di Bob ed otterrà una coppia dove la prima componente è leggibile;
5. Passo 5: TTP ha ricevuto il messaggio 4 che è un crittotesto, guarda il protocollo di trasporto e capisce che viene da B, ma se il protocollo di trasporto viene manomesso TTP prenderà la chiave sbagliata, non viene violata nè l'autenticazione nè la confidenzialità, ci sarà solo una interruzione di servizio (DDOS) in quanto il messaggio non sarà leggibile, la sessione avrà un aborto. Se nulla di tutto ciò accade, dal suo database accede alla chiave di Bob, decifra, cerca la chiave di A dal suo database, decifra la componente e tira fuori la Nonce N_b che invia nuovamente a Bob cifrandola con la chiave di Bob. A questo punto Bob valuterà se autenticare Alice o meno in base alla nonce ricevuta. $TTP \longrightarrow B : \{N_b\}_{K_b}$.

Ma perché il messaggio 5 di Woo-Lam è cifrato?

Il messaggio 5 nel protocollo di Woo-Lam è cifrato per garantire l'autenticità del mittente, ovvero il Trusted Third Party (TTP), e per proteggere la confidenzialità della nonce N_b . Questo è importante perché Bob riceverà il messaggio cifrato con la sua chiave privata, che potrebbe essere decifrato solo da qualcuno che conosce la sua chiave a lungo termine, il che nel contesto del protocollo può essere solo il TTP.

Uno scenario di attacco sarebbe indurre Bob a credere che Alice sia online quando in realtà è offline, evidenzia l'importanza di garantire l'autenticità e l'integrità delle comunicazioni nel contesto del protocollo di autenticazione. La cifratura del messaggio 5 con la chiave di Bob contribuisce a mitigare questo tipo di rischio, proteggendo Bob da potenziali tentativi di inganno.

6.4.1 Attacco su Woo-Lam

Alice è offline e l'attaccante si spaccia per lei. Questo è possibile tramite l'ausilio di due sessioni, Bob farà abort in una sessione ed Ok nell'altra quella dove Charlie si impersonifica in Alice.

- Sessione 1: L'attaccante Charlie dice a Bob di essere Alice: $1. C \longrightarrow B : A$;
- Sessione 2: L'attaccante Charlie dice a Bob di essere Charlie: $1'. C \longrightarrow B : C$;
- Sessione 1: Bob genera una nonce N_b e la invia ad Alice: $2. B \longrightarrow A : N_b$;

- Sessione 2: Bob genera una nonce N'_b e la invia a Charlie: 2'. $B \longrightarrow A : N'_b$.

Ma ricordiamo che Alice e Charlie sono la stessa persona ma con 2 sessioni differenti.

- Sessione 1: Charlie invia a Bob la nonce N_b cifrata con la sua chiave N_c : 3. $C \longrightarrow B : \{N_b\}_{K_c}$;
- Sessione 2: Charlie invia a Bob la nonce N_b cifrata con la sua chiave N_c : 3'. $C \longrightarrow B : \{N_b\}_{K_c}$.

Invia la stessa nonce o meglio lo stesso challenge ad ambedue le sessioni.

- Sessione 1: Bob prende il challenge cifrato e lo accoppia ad Alice cifrando il tutto con la propria chiave a lungo termine e lo invia al TTP: 4. $B \longrightarrow TTP : \{A, \{N_b\}_{K_c}\}_{K_b}$;
- Sessione 2: Bob prende il challenge cifrato e lo accoppia a Charlie cifrando il tutto con la propria chiave a lungo termine e lo invia al TTP: 4'. $B \longrightarrow TTP : \{C, \{N_b\}_{K_c}\}_{K_b}$;
- Sessione 1: : Il TTP prova a decifrare e non ottiene nulla di significativo, una nonce che chiamiamo N''_b e rispedisce a Bob cifrando con la chiave di Bob: 5. $TTP \longrightarrow B : \{N''_b\}_{K_b}$;
- Il TTP decifra il messaggio, questa volta l'associazione è corretta ed ottiene N_b e rispedisce a Bob cifrando con la chiave di Bob: 5'. $TTP \longrightarrow B : \{N_b\}_{K_b}$

Bob rivedendo indietro N_b autentica Alice ma che in realtà è Charlie non riuscendo a distinguere le due sessioni avviate.

6.4.2 Soluzione all'attacco su Woo-Lam

Chi fa l'errore? Bob, non il TTP: è necessario quindi operare solo sul passaggio 5. Basterebbe informare Bob nel messaggio 5 dell'identità dell'agente la cui chiave a lungo termine è stata usata per decifrare il challenge privato, aggiungiamo Alice al messaggio 5 dentro l'encryption per non sbagliarsi sulle sue conclusioni. Nel protocollo modificato i passi delle rispettive sessioni sarebbero così riformulati:

- Sessione 1: $TTP \longrightarrow B : \{N''_b, A\}_{K_b}$;
- Sessione 2: $TTP \longrightarrow B : \{N_b, C\}_{K_b}$

Adesso Bob effettuerà l'abort di entrambe le sessioni non riconoscendo nella prima la nonce N''_b e nella seconda riconosce la nonce N_b ma non c'è l'associazione tra la nonce ricevuta e Charlie che non è il suo interlocutore originale.

6.5 Principio di disegno: Explicitness

Se le identità del mittente e del ricevente sono significative per il messaggio, allora è prudente menzionarle esplicitamente. È preferibile specificare chiaramente le identità del mittente e del destinatario quando queste informazioni sono rilevanti. Ma quanto sono rilevanti? In modo sorprendente, includere il nome "Alice" nel messaggio 1 potrebbe non essere essenziale. Eliminare il riferimento ad "Alice" dal messaggio 1 potrebbe sembrare insignificante dal punto di vista della sicurezza, poiché chiunque potrebbe inserire "Alice" in quel messaggio. L'identità del mittente potrebbe essere dedotta dal livello di trasporto. Seguire il principio di esplicitazione non è da solo garanzia di successo del protocollo, ma rappresenta un principio di prudenza fondamentale.

6.6 Symmetric Needham-Schroder

- Alice manda un messaggio in chiaro al TTP, richiede la generazione di una chiave di sessione per comunicare con Bob: 1. $A \rightarrow TTP : A, B, N_a$;
- Il TTP genera la chiave di sessione K_{ab} e risponde ad Alice. Tutto il messaggio è cifrato con la chiave di Alice (K_a), quindi solo lei può leggerlo: 2. $TTP \rightarrow A : \{N_a, B, K_{ab}, \{K_{ab}, A\}_{K_a}\}_{K_a}$;
- Alice decifra il messaggio del Passo 2, prende il "pacchetto" destinato a Bob e glielo inoltra. Bob decifra il messaggio con la sua chiave segreta (K_b). Ora anche Bob possiede la chiave di sessione K_{ab} e sa che Alice vuole comunicare con lui: 3. $A \rightarrow B : \{K_{ab}, A\}_{K_b}$;
- Bob vuole essere sicuro che dall'altra parte ci sia davvero Alice in questo preciso momento (e non un attaccante che ha intercettato il messaggio del Passo 3). Questa è una "sfida" (challenge). Bob dice: "Se sei davvero Alice e hai la chiave K_{ab} , decifra questo numero": 4. $B \rightarrow A : \{N_b\}_{K_{ab}}$;
- Alice risolve la sfida di Bob. Decifra il messaggio, prende N_b , esegue un'operazione matematica semplice (sottrae 1, ovvero $N_b - 1$), cifra nuovamente il risultato con K_{ab} e lo rimanda a Bob: 5. $A \rightarrow B : \{N_b - 1\}_{K_{ab}}$.

Il limite di questo protocollo sta nella chiave di sessione: se un attaccante dovesse scoprire una chiave di sessione vecchia, potrebbe autenticarsi come Alice.

6.6.1 Replay Attack

Spacciare informazione (chiavi,...) obsoleta, magari violate, come recente.

Come anticipato, se un attaccante riuscisse a scoprire una vecchia chiave di sessione K_{ab} compromessa, potrebbe riutilizzare il pacchetto del Passo 3 ($\{K_{ab}, A\}_{K_b}$) per ingannare Bob, facendogli credere che Alice voglia avviare una nuova sessione. Bob non ha modo di sapere se quel pacchetto è vecchio o nuovo perché

non contiene un timestamp o un suo nonce precedente.

Per mitigare questo attacco, basta sostituire i nonce con il **Timestamp**: se il messaggio scade dopo pochi minuti, anche se Eve lo intercetta e lo reinstalla, Bob lo rifiuterà immediatamente perché l'ora interna al messaggio non corrisponde a quella attuale.

7 IPSec

La sicurezza informatica si colloca sia a livello della comunicazione che al di sopra di essa, e si basa su misure fondamentali come:

- Crittografia;
- Steganografia;
- Chaffing & Winnowing;

La crittografia trasforma i dati in modo che siano illeggibili per chi non possiede la chiave di decifratura, mentre la steganografia nasconde informazioni all'interno di altri dati, solitamente modificando i bit meno significativi di un'immagine o di un altro file. Tuttavia, questo approccio non è particolarmente robusto dal punto di vista della sicurezza, poiché nascondere informazioni in un luogo ovvio aumenta il rischio di scoperta da parte di terze parti.

7.1 Alternative alla crittografia: Chaffing & Winnowing

Abbiamo già un minimo introdotto cos'è la Steganografia, che è un'alternativa alla crittografia. Parliamo di **Chaffing & Winnowing**, che letteralmente significa **Mischiare/Mescolare e Separare**.

Inventata da Ron Rivest, il co-creatore di RSA, questa tecnica dimostra che la sicurezza non è raggiungibile solo attraverso la crittografia. Il funzionamento è il seguente: invece di cifrare direttamente il messaggio, viene applicato un MAC (Message Authentication Code), una sorta di impronta digitale, al segreto. Quando il destinatario riceve il MAC, lo confronta con quello che dovrebbe essere sulla base della coppia ricevuta. Contemporaneamente, vengono inviate al destinatario una serie di coppie fittizie, come se fossero messaggi subliminali inseriti all'interno della comunicazione reale. Un potenziale ascoltatore vede un'enorme quantità di traffico e presume che tutte le coppie siano valide, mentre solo il destinatario può identificare la coppia corretta grazie al controllo di integrità che va a buon fine solo con la coppia giusta. Tuttavia, con questo schema, un attaccante può ancora vedere il segreto, ma non riesce a capire quale sia tra tutte le altre coppie fittizie. Se il segreto è una password, l'attaccante può tentare di indovinarla provando tutte le possibili combinazioni. Mentre con la crittografia l'informazione viene trasformata in modo che non sia deducibile sintatticamente (**confidenzialità tramite non deducibilità**), il metodo Chaffing & Winnowing garantisce la confidenzialità attraverso l'indistinguibilità tra le informazioni genuine e quelle fittizie.

7.2 Sicurezza a Livello di Rete

IPSEC rappresenta un protocollo di sicurezza operante a livello IP. Tuttavia, come può essere realizzata l'autenticazione, la confidenzialità e l'integrità a livello IP? La soluzione consiste nella crittografia dei pacchetti IP. Si tratta di una tecnologia universale per garantire la sicurezza dei protocolli di rete. Integra le tecnologie e i protocolli di rete con i metodi precedentemente discussi. È essenziale proteggere i pacchetti attraverso la crittografia per garantire la confidenzialità; otterremo l'autenticazione e l'integrità attraverso la costruzione del Message Authentication Code (Cifratura simmetrica). Il prerequisito essenziale è la **Condivisione della chiave**.

7.3 Sicurezza a quale livello

La sicurezza può essere integrata a vari livelli nello stack TCP/IP. Al livello inferiore, diventa più trasparente e rigida, mentre al livello superiore richiede una gestione dedicata, offrendo così maggiore flessibilità.

7.3.1 Sicurezza a livello rete

La sicurezza a livello di rete presenta vantaggi e svantaggi distinti:

- Pro:
 - Le applicazioni e gli utenti possono rimanere all'oscuro delle procedure di sicurezza.
- Contro:
 - Tuttavia, può appesantire significativamente le comunicazioni, favorendo attacchi di tipo DoS;
 - Potrebbe richiedere modifiche al sistema operativo.

7.3.2 Sicurezza a livello trasporto

Sicurezza a livello di trasporto offre vantaggi e svantaggi distinti:

- Pro:
 - Essenzialmente, può essere implementata o a livello di trasporto o direttamente nelle applicazioni;
 - I principali browser sono dotati di client TLS integrato.
- Contro:
 - Naturalmente, richiede modifiche (anche minime) al livello che la riceve.

7.3.3 Sicurezza a livello applicazione

Sicurezza a livello di applicazione presenta vantaggi e svantaggi distinti:

- Pro:
 - Consente una sicurezza personalizzabile, gestita direttamente dalle singole applicazioni.
- Contro:
 - Tuttavia, progettare la sicurezza a questo livello può essere ingannevole e comportare rischi imprevisti.

7.4 IPSec

Ritorniamo a parlare di IPSec. IPsec è una suite di tre protocolli fondamentali:

- **IKE (Internet Key Exchange)**: Implementa il protocollo Diffie-Hellman per avviare la sicurezza delle comunicazioni;
- **AH (Authentication Header)**: Fornisce autenticazione e integrità dei dati;
- **ESP (Encapsulated Security Payload)**: Garantisce la confidenzialità dei dati.

Mentre è obbligatorio per IPv6, per IPv4 è facoltativo. Non è necessario che tutti i nodi di una rete siano compatibili con IPsec; è sufficiente che siano compatibili con IP, a differenza di IPv4 dove ogni nodo deve essere conforme a IP. In IPsec, è sufficiente che i nodi mittente e destinatario siano conformi a IPsec.

7.5 Security Association (SA)

La Security Association (SA) rappresenta una regola fondamentale o policy di IPsec. Si tratta di una relazione unidirezionale tra mittente e destinatario; sono necessarie due SA per una comunicazione bidirezionale. L'identificatore della policy è chiamato SPI (Security Parameter Index). Una SA specifica il tipo di sicurezza utilizzato (AH o ESP) e comprende tre campi principali:

- **SPI**: Security Parameter Index, che identifica la policy;
- Indirizzo IP di destinazione;
- Identificatore del protocollo (AH e/o ESP).

7.6 Security Association Database (SAD)

Ogni nodo della rete deve gestire un Database delle Security Association (SAD) con tutte le SA attive su quel nodo. Ogni voce nel database contiene tutti gli elementi della SA e include ulteriori informazioni:

- Informazioni AH: dettagli aggiuntivi sull'autenticazione;
- Informazioni ESP: dettagli aggiuntivi sulla confidenzialità;
- Durata della SA (Lifetime).

7.7 Authentication Header (AH)

Authentication Header (AH) è un frammento aggiunto a ciascun pacchetto IP per supportare l'autenticazione e l'integrità dei pacchetti. Per poter calcolare il Message Authentication Code (MAC), mittente e ricevente devono aver concordato in precedenza una chiave IKE. AH autentica l'intero pacchetto, escludendo i campi variabili dell'header IP che potrebbero essere modificati dai nodi intermedi, come il tipo di servizio, i flag, l'offset del frammento, il time to live, e così via. Questo protocollo previene sia gli attacchi di replay che l'IP spoofing. Il replay attack viene contrastato utilizzando un meccanismo di freshness come attributo della proprietà di autenticazione. Per contrastare l'IP spoofing, viene utilizzato il MAC, che fornisce sia autenticazione che integrità.

7.7.1 Header AH: Formato

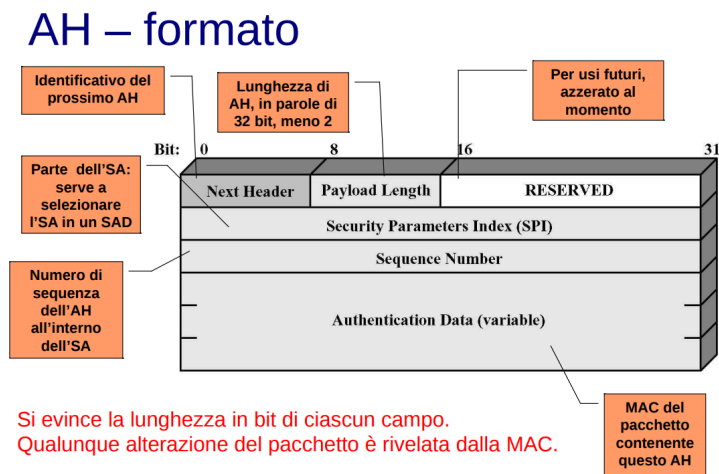


Figure 13: Formato AH

- Next Header: Identifica il prossimo header dopo l'AH;

- Payload Length: Lunghezza del payload (dati) protetto da AH;
- SPI (Security Parameters Index): Identificatore che seleziona la Security Association (SA) nel Security Association Database (SAD);
- Sequence Number: Numero di sequenza dell'AH all'interno della SA;
- MAC (Message Authentication Code): Codice di autenticazione del messaggio per il pacchetto contenente l'AH.

Si evince la lunghezza in bit di ciascun campo. Qualunque alterazione del pacchetto è rivelata dalla MAC.

7.8 Funzionamento di base IPSEC

Il funzionamento di base di IPsec segue questi passaggi:

- Ogni nodo mittente seleziona una Security Association (SA) utilizzando il proprio Security Policy Database (SPD);
- L'Identificatore della Security Association (SPI) dell'SA scelta viene incluso nell'Authentication Header (AH) di ciascun pacchetto.
- Quando un nodo riceve un pacchetto con l'AH, estrae l'SPI dall'header per selezionare la corrispondente SA dal proprio Security Association Database (SAD).

Il Security Policy Database (SPD) è una componente di IPsec che definisce le politiche per la gestione del traffico di rete. Decide quali pacchetti devono essere protetti da IPsec e come, basandosi su regole predefinite. In sostanza, l'SPD è il set di istruzioni che determina se e come un pacchetto di dati deve essere sicuro o meno.

7.9 Prevenzione dei replay attack

I pacchetti, una volta autenticati, sono protetti dal rischio di essere replicati grazie all'utilizzo del campo Sequence Number nell'Authentication Header (AH). Il mittente assegna un numero sequenziale compreso tra 0 e $2^{32} - 1$. Se necessita di ulteriori pacchetti, deve negoziare una nuova Security Association (SA). Il ricevente deve rifiutare i pacchetti che sono vecchi, ripetuti o falsificati. Per fare ciò, accetta una "finestra" di dimensione W (tipicamente $W=64$) di numeri di sequenza, dove N rappresenta il massimo numero nella finestra. Quando un pacchetto arriva, se il suo numero rientra nella finestra e non è già stato ricevuto, viene autenticato tramite il Message Authentication Code (MAC), e la posizione relativa viene segnata. Se il numero del pacchetto è maggiore di N , viene estesa la finestra verso destra fino a quel numero. Se il numero del pacchetto è inferiore o uguale a $N-W$, oppure il pacchetto è già presente o non è autenticato tramite MAC, viene segnalata un'anomalia.

Questo meccanismo a finestra protegge da attacchi Dolev-Yao? Facciamo un

esempio reale: Tick and Cross di studenti prenotati all'esame. Se due persone con lo stesso nome si presentano all'esame solo il primo di loro potrà effettuarlo. Non è un meccanismo di sicurezza fin quando non è possibile per uno studente cambiare il suo nome.

Nella pratica non è una misura di sicurezza dal momento in cui l'IP non venga modificato. Tuttavia, va notato che questo meccanismo, da solo, non è una misura di sicurezza completa, in quanto dipende dalla corretta gestione del MAC. Il MAC permette di verificare che il pacchetto sia recente e che sia stato autenticato in modo integro, rendendo il meccanismo a finestra affidabile. Se il MAC è valido, ciò significa che il payload non è stato alterato e che il pacchetto è recente. Pertanto, il sistema MAC garantisce l'integrità e l'autenticità dei dati, rendendo il meccanismo a finestra un metodo efficace per prevenire i replay attack quando utilizzato correttamente.

7.10 Protezione mediante AH: trasporto vs tunnel

La protezione tramite Authentication Header (AH) può avvenire in due modalità: trasporto e tunnel.

- Modalità trasporto: Si applica solo ai dati del pacchetto e ai campi non variabili dell'header IP, fornendo protezione contro l'IP spoofing. In IPv6, questa modalità estende la protezione anche alle estensioni dell'header.
- Modalità tunnel: Protegge l'intero pacchetto IP (dati e header IP) e i campi non variabili dell'indirizzo IP esterno, garantendo così la sicurezza contro l'IP spoofing. Anche in IPv6, questa modalità estende la protezione alle estensioni dell'header.

Con IPSec è possibile creare un tunnel che funge da VPN. Trasformando un tunnel di rete in IPSec, si ottiene una VPN. Nella modalità tunnel, se un pacchetto attraversa un nodo non compatibile con IPSec, il nodo lo inoltrerà senza interpretarne il contenuto, basandosi solo sull'header. Nella modalità trasporto, invece, un nodo non compatibile con IPSec non saprebbe come gestire il pacchetto, poiché non sarebbe in grado di comprendere il suo trattamento.

7.10.1 AH in IPv4 e in IPv6

AH in IPv4

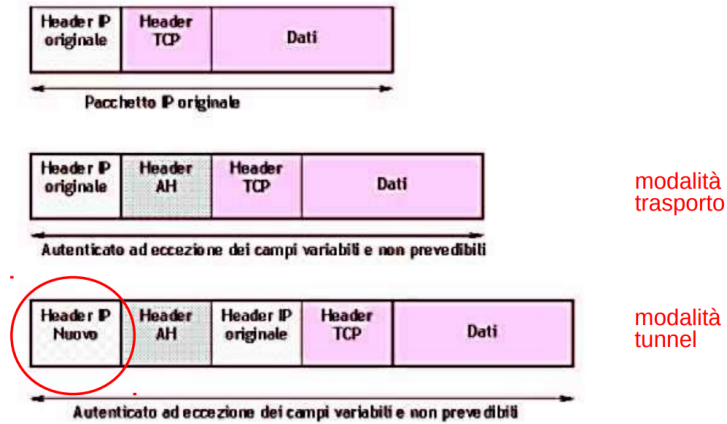
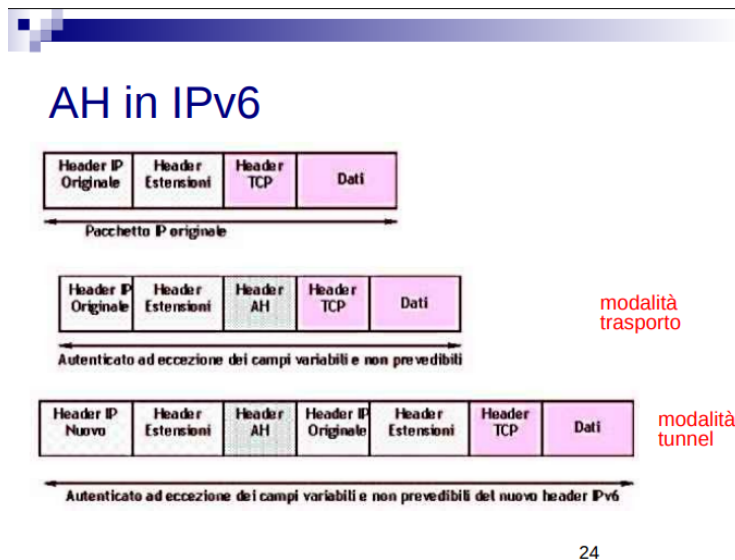


Figure 14: AH-IPv4



24

Figure 15: AH-IPv6

7.10.2 Usi di AH: trasporto vs tunnel

Nella modalità di trasporto di AH, sia il mittente che il ricevente devono utilizzare IPSec per garantire l'autenticazione punto-punto dei pacchetti scambiati. Questo significa che entrambi i nodi devono essere compatibili con IPSec e utilizzarlo per proteggere le comunicazioni tra di loro.

D'altra parte, nella modalità tunnel di AH, sia il mittente che il ricevente possono non utilizzare IPSec direttamente. Tuttavia, il tunnel IPSec fornisce comunque garanzie sull'autenticazione intermedia del percorso seguito dai pacchetti. Anche se i nodi intermedi non supportano IPSec, il tunnel protegge i pacchetti lungo il percorso e garantisce l'autenticazione intermedia tra i nodi.

7.10.3 AH in tunnel

Nel contesto di un tunnel AH, consideriamo il NAT come una misura necessaria, spesso adottata per garantire la sicurezza. Ciò è dovuto al fatto che in molte situazioni non è possibile comunicare direttamente con un dispositivo su Internet, quindi si crea una rete di collegamenti in cui i pacchetti attraversano altri dispositivi, eseguendo il NAT. Questa situazione può essere paragonata a una triangolazione: sebbene il collegamento tra il client mittente e il server possa essere sicuro, il collegamento tra il server e il destinatario potrebbe non esserlo. Un esempio pratico può essere il controllo remoto di una lampada Tapo. Tuttavia, se si utilizza la crittografia come nel caso di un tunnel AH, nessuno potrà violare le proprietà di sicurezza durante il passaggio attraverso la rete intermedia. Un esempio di utilizzo di AH in modalità tunnel potrebbe essere il seguente:

- Il nodo A su una rete NA genera un pacchetto IP classico destinato al nodo B su una rete NB;
- Il router o il firewall di NA incapsula questo pacchetto in un pacchetto IPSec in modalità tunnel e lo inoltra al router o al firewall di NB;
- Questo dispositivo di NB estrae ed autentica il pacchetto IP originale, quindi lo inoltra al nodo B sulla rete NB.

7.11 Encapsulating Security Protocol (ESP)

ESP (Encapsulating Security Payload) fornisce confidenzialità dei dati e, opzionalmente, autenticazione.

- **Confidenzialità del contenuto:** ESP cifra il contenuto del pacchetto, garantendo che sia inaccessibile a chiunque non sia il destinatario autorizzato;
- **Confidenzialità del flusso di traffico:** ESP protegge anche il flusso di traffico da analisi da parte di intermediari, fornendo così un ulteriore livello di sicurezza;

- **Autenticazione opzionale:** ESP può anche autenticare i dati, se configurato di conseguenza.

ESP può essere utilizzato da solo o in sequenza insieme ad AH (Authentication Header). ESP è considerato un altro header a sé stante, nonostante possa fornire funzionalità simili ad AH come l'autenticazione. Storicamente, AH e ESP sono stati sviluppati come concetti separati. Tuttavia, un'estensione di ESP permette anche l'autenticazione, portando ad una sovrapposizione di funzionalità tra i due protocolli. In sintesi, ESP è utilizzato principalmente per garantire la confidenzialità dei dati, ma offre anche la possibilità di autenticazione, fornendo così una solida protezione per le comunicazioni.

7.11.1 ESP- formato

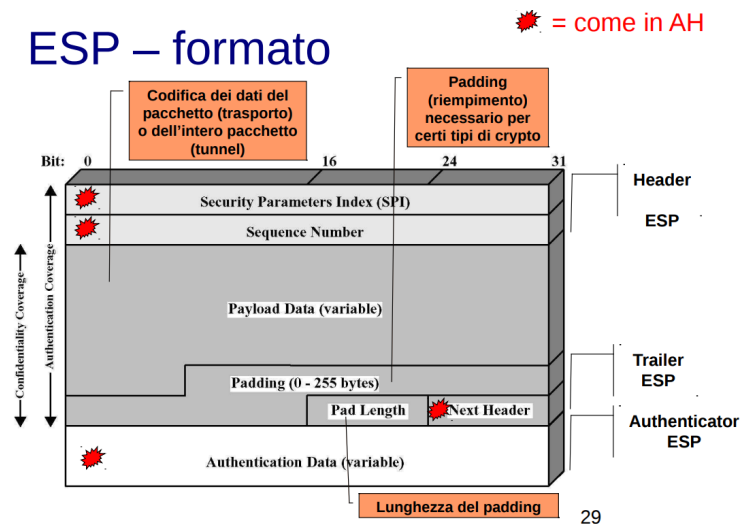


Figure 16: Formato ESP

ESP, l'Encapsulating Security Payload, protegge il contenuto dei pacchetti, ma non tutti gli elementi sono criptati. Ad esempio, l'indice di SPI (Security Parameters Index) è trasportato in chiaro, rendendolo soggetto ad analisi statistica. Tuttavia, ESP include diverse componenti:

- **SPI (Security Parameters Index):** Identifica i parametri di sicurezza associati a un flusso di dati;
- **Numero di sequenza:** Aiuta a garantire la freschezza dei dati e fornisce meccanismi anti-replay;
- **Payload:** Contiene i dati effettivi del pacchetto;
- **Padding:** Porzioni di dati riservate utilizzate per allineare i dati o per scopi di sicurezza;

- **Lunghezza del padding:** Indica la lunghezza del padding aggiunto al pacchetto;
- **Next Header:** Specifica il tipo di dati contenuti nel payload, consentendo al destinatario di interpretarlo correttamente.

Anche se ESP protegge il contenuto dei pacchetti, alcuni elementi rimangono visibili e potrebbero essere soggetti ad analisi, come nel caso dell'SPI. Pertanto, è importante considerare questo aspetto quando si valuta la sicurezza delle comunicazioni protette da ESP.

7.11.2 ESP in IPv4 e IPv6

ESP in IPv4

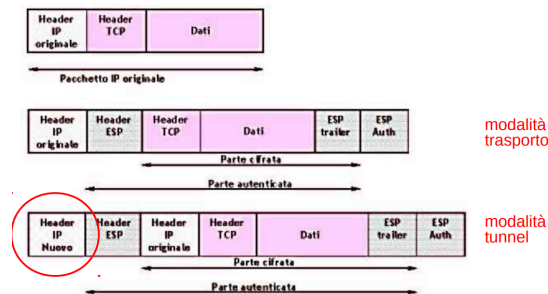


Figure 17: ESP IPv4

ESP in IPv6

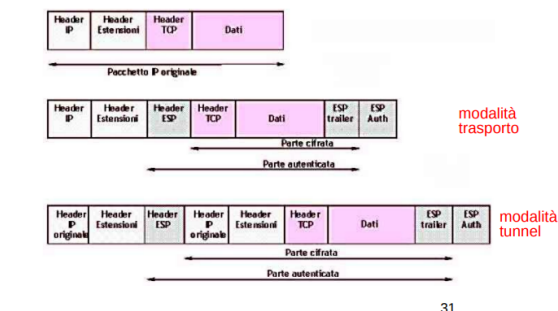


Figure 18: ESP IPv6

7.11.3 ESP in modalità trasporto

In modalità trasporto, ESP cifra l'header TCP, il payload e l'ESP trailer, ma non copre altri elementi come l'IP originale. Di conseguenza, l'indirizzo IP di destinazione rimane visibile e non è criptato, lasciando l'interlocutore esposto. Tuttavia, cifrare l'header IP potrebbe compromettere il funzionamento del protocollo IP.

Per ovviare a questa limitazione, possiamo optare per l'utilizzo di ESP in modalità tunnel. In questo caso, l'header IP originale viene cifrato e l'intero pacchetto, compreso l'header IP cifrato, viene incapsulato in un nuovo header IP. Un dispositivo intermedio, come un server anonimizzatore compatibile con IPSec, può fungere da punto di ingresso sicuro per il traffico.

Con la modalità tunnel, la crittografia avviene tra le reti, ad esempio tra due "nuvolette", piuttosto che direttamente tra i dispositivi finali. Questo approccio semplifica la gestione, poiché ogni tunnel può essere gestito singolarmente, rispetto alla gestione di ogni singolo dispositivo. Anche se non c'è una cifratura end-to-end, questa soluzione fornisce comunque un livello di sicurezza significativo per il traffico tra le reti coinvolte.

7.12 Come rendere la rete domestica più sicura?

Per rendere la rete domestica più sicura, è possibile adottare diversi approcci:

1. **Attivare una VPN tramite il modem:** Se il software del modem lo consente, attivare una VPN direttamente sul modem può fornire una maggiore protezione per le comunicazioni in entrata e in uscita dalla rete domestica. Una VPN crittografa il traffico Internet, nascondendo così i dati sensibili dagli occhi indiscreti;
2. **Utilizzare servizi di NAT forniti dagli ISP:** Molti Internet Service Provider (ISP) offrono servizi di Network Address Translation (NAT), che forniscono una forma di protezione aggiuntiva per i dispositivi all'interno della rete domestica. Il NAT nasconde gli indirizzi IP locali dei dispositivi dietro un unico indirizzo IP pubblico;
3. **Considerare l'utilizzo di indirizzi IP pubblici:** Anche se gli indirizzi IP pubblici sono spesso considerati come un extra o offerti solo a un costo aggiuntivo, possono aumentare la sicurezza della rete domestica, specialmente se si desidera consentire l'accesso remoto ai dispositivi interni;
4. **Attivare una VPN tra il modem e il fornitore del modem:** Una VPN tra il modem e il provider del modem può creare un ulteriore strato di sicurezza, garantendo che il traffico sia crittografato lungo tutto il percorso tra la rete domestica e il provider di servizi Internet.

Adottando queste misure, è possibile aumentare significativamente la sicurezza della propria rete domestica, proteggendo i dati personali e impedendo l'accesso non autorizzato alla rete.

7.13 Combinazioni di SA

Non ha senso combinare più di due modalità di trasporto (ESP in trasporto & AH in trasporto). Ha senso, però, combinare (sovrapporre) molteplici tunnel, perché ogni tunnel può avere i propri nodi di inizio e di fine. Ogni nodo compatibile IPsec deve supportare 4 tipi di combinazioni SA.

7.13.1 Tipo 1: sicurezza punto-punto

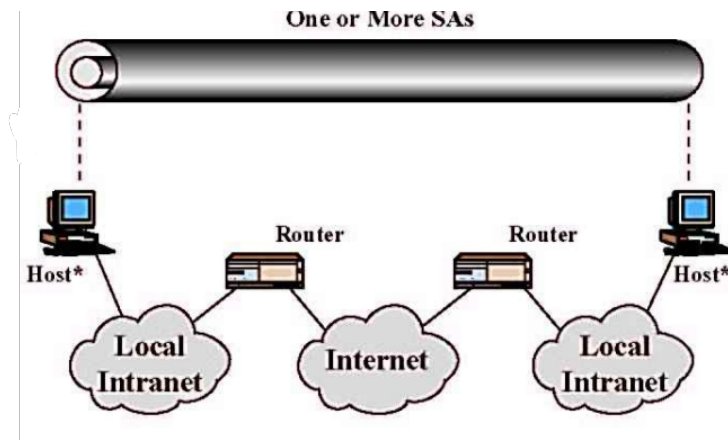


Figure 19: Tipo 1

- AH in trasporto;
- ESP in trasporto;
- 1 & 2;

I nodi intermediari non devono per forza essere IPSEC-compliant.

7.13.2 Tipo 2: sicurezza fra intermediari

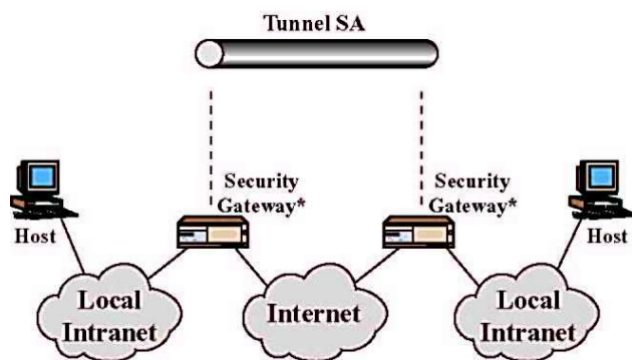


Figure 20: Tipo 2

- AH in tunnel;
- ESP in tunnel;

Gli host non devono per forza essere IPSEC-compliant e gli attacchi avverranno su internet.

7.13.3 Tipo 3: tipo 1 & tipo 2

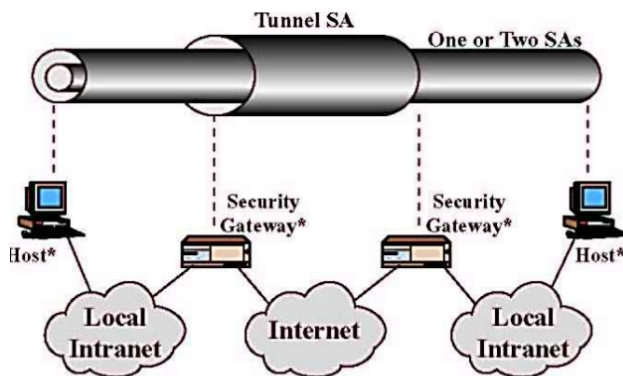


Figure 21: Tipo 3

Combinazioni delle topologie appena viste:

- Combinazione di tipo 1 punto-punto;
- Combinazione di tipo 2 fra intermediari.

Aggiunta di un tunnel tra i gateway, se ci fosse un problema sui gateway ci sarebbe il tunnel interno a protezione della comunicazione.

7.13.4 Tipo 4: tipo 1 & sicurezza punto intermedio

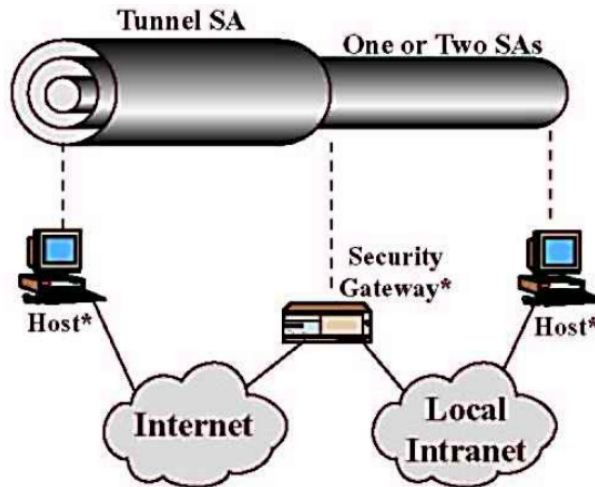


Figure 22: Tipo 4

- Combinazione di tipo 1 punto-punto;
- AH in tunnel o ESP in tunnel.

Creo un tunnel tra l'host ed il gateway, i nodi devono essere IPSEC.

7.14 ESP con/senza autentica

Una SA può sancire l'utilizzo di ESP con autenticazione. In questo caso, l'uso di AH appare ridondante. Anche se, molte opzioni di IPsec appaiono ridondanti. Delle **Potenziati semplificazioni di IPsec** sono:

1. Eliminare AH: usare sempre ESP con autenticazione;
2. Eliminare modalità trasporto e usare sempre solo tunnel (IPsec dovrebbe essere su ogni nodo);
3. Ridefinire le SA come associazioni bidirezionali.

7.15 Internet Key Exchange (IKE)

Protocollo per la generazione delle chiavi da utilizzare con AH ed ESP. IKE si fonda su una variante arricchita di Diffie-Hellman, integrata con ISAKMP, quindi consta di due protocolli integrati a livello applicazione:

- **Oakley**: che sarebbe una variante di Diffie-Hellman, fornisce la chiave iniziale;
- **ISAKMP** (Internet Security Association and Key Management Protocol): a livello di trasporto, crea chiavi di sessione usando la chiave iniziale.

L'uso della cifratura asimmetrica favorisce la rapida e efficiente condivisione delle chiavi simmetriche. IPSEC, tramite il protocollo ISAKMP, elimina la necessità di avere chiavi aggiornate continuamente.

8 Intrusion Detection

8.1 Intusion Detection Working Group

Gruppo di lavoro per lo sviluppo di sistemi (distribuiti) per il rilevamento delle intrusioni. Esso intende produrre:

- Una descrizione dei requisiti funzionali di alto livello per le comunicazioni fra i moduli del sistema;
- Un linguaggio per la specifica di intrusioni;
- Una comparativa dei protocolli di comunicazione esistenti più appropriati secondo i requisiti di cui al punto 1.

8.2 Intrusione versus Malware

Un intruso è un processo utente reale con determinati privilegi. I virus raramente possono essere considerati tipi particolari di intrusi. Un virus non è paragonabile a un batterio, per intenderci. Un worm, più facilmente, può essere considerato un tipo di intruso. Come un batterio, è autonomo. Un intruso non è né un virus né un worm, ma può installarne uno. Si fa riferimento ai processi intrusi come evidenziato dal comando "top" in Linux.

8.3 Intrusione versus Dos

DoS	Intrusioni
■ Effetti temporanei	■ Effetti generalm. permanenti
■ Immediatamente pubblico	■ Spesso non reso pubblico
■ Blocco delle risorse	■ Uso illecito di risorse

Gli attacchi DoS bloccano una macchina, gli intrusi sfruttano il sistema si parla di stealth e non valutabile.

8.4 Negazione del servizio DoS

La negazione del servizio (DoS) è un tipo di attacco informatico che mira a impedire la normale operatività di un sistema, rendendolo inaccessibile agli utenti legittimi. Esistono diverse tipologie di attacchi DoS:

- **cDos (Consumo di risorse computazionali):** Questo tipo di attacco sfrutta il consumo eccessivo di risorse computazionali del sistema bersaglio, come la CPU, impedendo così il normale funzionamento dei servizi;
- **mDoS (Consumo della memoria):** In questo caso, l'attaccante satura la memoria del sistema bersaglio, impedendo l'accesso ai dati e ai servizi;
- **bDoS (Consumo della banda di trasmissione):** Questo tipo di attacco sovraccarica la banda di trasmissione del sistema, rendendo difficile o impossibile per gli utenti legittimi accedere ai servizi.

Inoltre, c'è il concetto di **DDoS (Distributed Denial of Service)**, che è una variante più sofisticata del DoS, in cui l'attacco è orchestrato da più macchine collegate in rete, spesso sfruttando una botnet. Questo rende l'attacco più potente e difficile da mitigare. Gli attacchi DoS sono indipendenti dall'autenticazione e possono essere eseguiti anche su sistemi autenticati. L'obiettivo principale è sovraccaricare il sistema o esaurire le sue risorse critiche, rendendolo inutilizzabile per gli utenti legittimi. Nel contesto dell'Internet of Things (IoT), la sicurezza riveste un ruolo cruciale, poiché i dispositivi connessi potrebbero essere vulnerabili agli attacchi DoS e potrebbero essere utilizzati come parte di una botnet per lanciare attacchi DDoS su larga scala.

8.4.1 DOS vs DDOS

DoS è un attacco che sfrutta un router con funzionalità di broadcast. L'attaccante invia un pacchetto PING utilizzando l'indirizzo IP della vittima. Questo pacchetto viene poi riflesso da tutte le macchine nella sottorete, creando un attacco DDoS che sovraccarica il sistema bersaglio. In un'altra variante, l'attaccante istruisce un certo numero di "zombie" (macchine compromesse) a contattare un particolare indirizzo IP, il che porta alla saturazione della macchina dietro quell'IP.

8.4.2 Come reagire ad un DoS

Come affrontare un attacco DoS è ancora una sfida complessa oggi. Esistono firewall dotati di moduli antidos che possono filtrare il traffico, mentre un'alternativa strategica è l'implementazione della trasformazione dei cookie. Ma quale dovrebbe essere la risposta?

Per un attacco DoS, una strategia è quella di chiudere le connessioni provenienti dalla rete attaccante. Ma per un attacco DDoS, chiudere tutte le connessioni può essere difficile e sconsigliato. Una soluzione potrebbe essere il filtraggio del numero di connessioni accettate, ad esempio tramite htaccess con un firewall. Inoltre, una euristica di prevenzione potrebbe consistere nel complicare

computazionalmente l'accesso, ad esempio tramite la trasformazione dei cookie (transformation cookie).

8.4.3 Anti-DoS: Cookie Transformation

Se il protocollo è semplice, l'attaccante può sfruttarlo per inviare più richieste contemporanee, causando confusione al server che potrebbe non essere in grado di gestirle tutte.

Una strategia per contrastare questo tipo di attacco è la trasformazione dei cookie. Il server, anziché rispondere direttamente con il messaggio m2, invia inizialmente un messaggio più semplice, m2s, sotto forma di cookie. Il protocollo richiede quindi al client di inviare lo storico dei messaggi m1 e m2s, insieme all'hash tra m1 e m2s. La verifica dell'hash è un'operazione veloce e consente al server di capire se il client è attivo dall'altra parte. Solo a questo punto, il server risponderà, poiché ha confermato l'attività del client.

Se il client non fosse attivo e cercasse di eseguire un attacco del tipo fork bomb, creando un loop malevolo di richieste, non sarebbe in grado di mantenere lo storico dei messaggi e calcolare l'hash richiesto. Di conseguenza, non avrebbe la possibilità di completare l'operazione richiesta, fornendo una protezione aggiuntiva contro gli attacchi DoS.

8.5 Intrusione

Ottenere illecitamente privilegi superiori a quelli posseduti lecitamente.

L'intrusione si verifica quando un individuo ottiene in modo illecito privilegi superiori a quelli legittimamente posseduti. Questo può avvenire attraverso l'acquisizione di accesso al sistema e l'estensione dei propri privilegi. Non è necessariamente limitato alle intrusioni dalla rete, ma può coinvolgere anche l'inserimento di software dannoso tramite dispositivi come floppy disk o pen drive. Spesso coinvolge la violazione dei meccanismi di autenticazione e controllo degli accessi.

In sintesi, l'intrusione comporta l'ottenimento di privilegi non autorizzati, violando le regole di accesso e autenticazione del sistema.

8.5.1 Tecniche di intrusione

- Violazioni di password: tecniche standard e non-standard;
- Intercettazione di informazioni sensibili: scovarle (non solo password) se transitano in rete senza crittografia oppure violare protocolli di sicurezza;
- Uso di combinazioni di SW nocivo: sullo stile del worm Morris.

8.6 Intrusion Detection vs Intrusion Management

- **Intrusion Detection (rilevamento dell'intrusione):** È il processo di individuare attività sospette o non autorizzate all'interno di un sistema o

di una rete. L'obiettivo principale è rilevare l'intrusione il prima possibile per prendere provvedimenti e prevenire danni maggiori;

- **Intrusion Management (gestione dell'intrusione):** Una volta individuato un intruso, questo processo mira ad espellerlo dal sistema o dalla rete il più rapidamente possibile per limitare i danni che può causare. L'obiettivo è minimizzare l'impatto negativo sull'integrità, la disponibilità e la riservatezza dei dati e dei servizi.

8.7 Intrusion Detection System (IDS)

Un Sistema di Rilevamento delle Intrusioni (IDS) registra e valuta i log, analizzandoli per individuare attività sospette. L'approccio consiste nel registrare tutti i dettagli, il che si rivela prezioso in caso di problemi, consentendo un'auditing efficace dei log (come il comando "history" in Linux). In caso di attacco, l'IDS è responsabile di analizzare i log. Questo sistema può operare sia in tempo reale che in modalità post-mortem, consentendo di individuare intrusioni nel sistema in tempo reale. Un Security Operations Center (SOC) è un insieme di tecnologie che consente di esternalizzare la gestione della rilevazione e prevenzione degli attacchi. Queste tecnologie, alimentate dall'intelligenza artificiale, offrono opportunità di lavoro per gli analisti SOC. Un insieme di tecnologie IDS costituisce una tecnologia nota come SIEM / SOAR (Security Incident and Event Management), che comprende sistemi di rilevamento delle intrusioni. Un IDS può essere implementato a livello di nodo sulla nostra macchina o come un sistema di rete. Il suo obiettivo è riconoscere i processi non autorizzati. Per definire quali processi siano considerati non autorizzati, si possono adottare diverse strategie. Ad esempio, si può utilizzare una whitelist dei processi su una distro standard: i processi non presenti nella whitelist vengono considerati alieni. Tuttavia, questa soluzione statica potrebbe non essere sempre efficace. In alternativa, è possibile adottare un approccio dinamico che studia il comportamento del processo, analizzandolo per identificare eventuali comportamenti dannosi.

In sintesi, un IDS registra il comportamento del sistema, analizzando i log attraverso tecniche di pattern matching e operando in tempo reale. Un IDS distribuito può coinvolgere più processi o macchine per la registrazione, l'analisi e la presa di decisioni.

8.7.1 Definire i comportamenti

Il principale ostacolo per un Sistema di Rilevamento delle Intrusioni (IDS) è identificare i comportamenti dell'intruso in contrasto con quelli dell'utente legittimo. Questo può essere affrontato considerando il comportamento medio nel tempo e utilizzando tecniche di machine learning.

Tuttavia, l'obiettivo è trovare il miglior compromesso tra scambiare un utente legittimo per un intruso (falso positivo) e non rilevare un intruso come tale (falso negativo).

8.7.2 Record di auditing

Ci servono i log, le entry di un log si chiamano record.

1. **Record Nativi:** Tutti i sistemi operativi includono funzionalità per la raccolta di informazioni generiche sulle attività degli utenti. Questi possono includere la storia dei comandi della shell. Sebbene questa sia una soluzione semplice, non sempre fornisce informazioni dettagliate e utili.
2. **Record Specifici per il Rilevamento:** Si tratta di software aggiuntivo progettato per raccogliere esclusivamente informazioni specifiche per il rilevamento delle intrusioni. Questo tipo di record può evidenziare deviazioni dal modello statistico o comportamenti sospetti. Anche se può appesantire il sistema, fornisce informazioni mirate e utili per il rilevamento delle intrusioni.

Gli accessi vanno loggati e per legge ci vuole in certi casi.

8.7.3 Record di Denning

I record di Denning sono un esempio di record specifici per il rilevamento delle intrusioni, ciascuno contenente sei campi distinti:

1. **Soggetto:** Rappresenta l'utente o il processo che esegue azioni tramite comandi specifici. Possono essere raggruppati in classi di accesso, noti anche come ruoli;
2. **Azione:** Indica l'operazione che il soggetto svolge sul sistema;
3. **Oggetto:** Rappresenta l'entità su cui viene svolta l'azione. È importante notare che un soggetto può anche essere un oggetto, consentendo un'analisi più dettagliata delle interazioni nel sistema;
4. **Eccezione:** Specifica se è stata prodotta un'eccezione in risposta all'azione e, in tal caso, quale eccezione è stata generata;
5. **Utilizzo delle risorse:** Questo campo fornisce informazioni dettagliate sulle risorse utilizzate durante l'azione, come il numero di righe stampate o visualizzate, il numero di settori letti o scritti, il tempo CPU impiegato e le unità di I/O utilizzate;
6. **Timestamp:** Serve come identificatore temporale che indica il momento in cui è iniziata l'azione.

Questi record offrono un quadro completo delle attività nel sistema, consentendo un'analisi approfondita per il rilevamento e la gestione delle intrusioni.

8.8 Tecniche di rilevamento

- **Indipendenti dal Passato (Standard):** Queste tecniche si basano su criteri predefiniti che non tengono conto delle precedenti attività nel sistema. Un esempio è l'imposizione di un limite massimo di tentativi di inserimento della password, ad esempio massimo 3 fallimenti consentiti;
- **Dipendenti dal Passato:** Queste tecniche si basano sull'osservazione o campionamento del funzionamento o comportamento del sistema in passato. Si assume che il comportamento futuro sarà simile a quello passato.

Le tecniche di rilevamento possono essere suddivise in:

- **Rilevamento Statistico:** Si tratta di creare un modello statistico del comportamento di un utente legittimo. Quando un utente o un processo si discosta in modo significativo da questo modello, viene identificato come un potenziale intruso. Questo metodo è particolarmente efficace per individuare attaccanti esterni che possono comportarsi in modo diverso dagli utenti legittimi;
- **Rilevamento a Regole:** In questo approccio, vengono definite regole per il comportamento di un intruso. Quando un'attività corrisponde a una di queste regole, viene identificata come potenziale intrusione. Questo metodo è particolarmente efficace nel rilevare attaccanti interni che potrebbero avere una conoscenza approfondita del sistema e delle sue regole di funzionamento.

Queste tecniche non sono intelligenti, con nuove tecniche di machine learning è possibile prevedere eventuali cambi di programma.

- **Rilevamento statistico:** Creo un modello statistico per il comportamento di un utente legittimo. Se l'intruso si discosta sufficientemente dal processo allora lo flaggo come intruso;
- **Rilevamento a regole:** Fare più di n tentativi in una certa fascia oraria viene flaggato come intruso (due regole in questo caso).

8.8.1 Rilevamento Statistico - Modelli

1. **Media e Deviazione Standard:** Questo modello calcola la media e la deviazione standard di un parametro nel corso del tempo. Questi valori forniscono un'indicazione del comportamento medio del parametro e della sua variabilità;
2. **Modello operativo:** Questo modello definisce un limite di accettabilità per un parametro e segnala un'eventuale intrusione quando questo limite viene superato. È un approccio basato su soglie di allarme;
3. **Multivariazione:** In questo modello, vengono considerate più variabili simultaneamente per identificare modelli complessi di comportamento anomalo;

4. **Processo di Markov:** Questo modello utilizza la teoria dei processi stocastici per modellare il comportamento del sistema nel tempo e prevedere le transizioni tra diversi stati;
5. **Serie Temporale:** Questo modello analizza le variazioni di un parametro nel tempo, consentendo di individuare tendenze, stagionalità e comportamenti anomali nel tempo.

Ad esempio, il tempo trascorso dall'ultimo login potrebbe essere utilizzato come parametro per il rilevamento statistico. Un account inattivo da troppo tempo che effettua il login potrebbe indicare un account compromesso o non più utilizzato, noto anche come "account death".

8.8.2 Rilevamento a Regole

Il rilevamento a regole si basa su un vasto database di regole predefinite, ciascuna progettata per identificare modelli specifici di comportamento sospetto o attività anomale nel sistema.

Esempi noti di sistemi basati su regole includono Snort e ntop. Snort, ad esempio, utilizza un linguaggio di regole flessibile e potente, che permette agli amministratori di definire regole personalizzate per il rilevamento di attacchi e intrusioni.

Ogni regola di Snort, in sostanza, è simile a scrivere una riga di controllo di iptables, fornendo un metodo efficace per identificare e rispondere a varie tipologie di minacce informatiche.

8.9 Tecniche di Protezione

1. **Limitazione dell'Automazione:** Questa tecnica mira a limitare il numero di tentativi di accesso automatico per prevenire attacchi di forza bruta o di tentativi ripetuti. Ad esempio, il sistema può essere configurato per ripristinare la connessione dopo un numero specifico di inserimenti di password errate (come 3), o il telefono cellulare può bloccare la SIM dopo un certo numero di tentativi di inserimento del PIN errati;
2. **Utilizzo di Esche (Honey Pot):** Le esche sono risorse fittizie progettate per attirare gli attaccanti e distoglierli dalle risorse più importanti o per farli rimanere più a lungo nel sistema, consentendo al sistema di identificarli e intraprendere azioni preventive. Ad esempio, possono essere creati URL come "transazioni.grossasocieta.com" o "conticorrenti.banca.com", che sembrano legittimi ma conducono a risorse monitorate;
3. **Utilizzo di Trappole:** Questa tecnica prevede il massiccio monitoraggio di risorse fittizie o esche, e appena l'attaccante accede a una di esse, viene attivata una trappola che permette di rilevare e rispondere all'attacco in tempo reale.

8.10 Intrusion Management System (IMS)

L'intrusione va:

1. Contenuta (containment);
2. Rimossa (eradication);
3. Riassorbita (recovery);
4. Punita (punishment);

Questa definizione descrive l'Intrusion Management System (IMS) non solo come un mero strumento tecnologico di rilevamento (come i classici IDS/IPS), ma come un processo strategico e operativo completo per gestire l'intero ciclo di vita di un attacco informatico. Invece di limitarsi a dire "c'è un problema", questo approccio definisce esattamente cosa fare dopo che l'intrusione è stata identificata. È una visione fortemente pragmatica, orientata alla resilienza aziendale e alla giustizia.

- **Contenuta:** Una volta che l'intruso è penetrato nelle difese, il tempo è il peggior nemico. L'obiettivo del contenimento è limitare i danni e impedire che l'attacco si diffonda (movimento laterale) ad altri sistemi critici. Si isolano le macchine compromesse dalla rete, si chiudono porte specifiche, si disabilitano temporaneamente gli account violati o si segmenta la rete;
- **Rimossa:** Una volta "messo in sicurezza" il perimetro del danno, bisogna eliminare completamente la presenza dell'attaccante e le vulnerabilità che ha sfruttato. Si eliminano i malware, si cancellano i file malevoli, si chiudono le backdoor create dall'attaccante e, fondamentale, si applicano le patch alla vulnerabilità iniziale che ha permesso l'accesso;
- **Riassorbita:** Il sistema deve tornare alla totale normalità operativa nel minor tempo possibile, garantendo l'integrità dei dati. Si ripristinano i sistemi partendo da backup puliti e verificati, si reimpostano tutte le password aziendali, si riavviano i servizi interrotti e si avvia un monitoraggio intensivo per assicurarsi che i sistemi "guariti" si comportino normalmente;
- **Punita:** Questo è l'elemento che spesso differenzia un approccio puramente tecnico da uno strutturato a livello legale e societario. Non ci si limita a riparare il danno, si passa alla controffensiva legale. Entra in gioco la Digital Forensics (informatica forense). Si raccolgono e cristallizzano le prove dell'intrusione (log, artifact, timeline) seguendo procedure rigorose affinché siano ammissibili in tribunale. Si sporge denuncia alle autorità competenti (es. Polizia Postale) per identificare e perseguire i responsabili.

9 Software nocivo (Malware)

Bug e vulnerabilità: Un bug è una proprietà inattesa del software. Anche se non necessariamente nocivo, un bug può essere sfruttato da altri software. Ad esempio, un'applicazione utente su Windows 2000 potrebbe sovrascrivere il modulo di sicurezza in memoria. I bug sono solitamente preterintenzionali, ovvero non introdotti deliberatamente.

Software Nocivo: Il software nocivo (malware) è scritto con l'esplicito scopo di violare la sicurezza di un sistema. Non è detto che sfrutti dei bug per farlo. Ad esempio, un virus può infettare un sistema anche se non ci sono vulnerabilità evidenti. È possibile scaricare un allegato contenente un eseguibile (.exe, .bat) che rappresenta il malware, il quale sfrutta il sistema ospitante indipendentemente dalla presenza di bug.

Un bug preterintenzionale non rende necessariamente nocivo il software che lo ospita. Tuttavia, la presenza di malware implica sempre un'intenzione malevola.

Nesso tra Attività Offensiva e Malware: Il legame tra attività offensiva e malware non è sempre evidente. Ad esempio, un penetration tester etico può impiantare malware per testare la sicurezza di un sistema. Questo è un caso di sfruttamento di vulnerabilità, noto come exploitation. Un sistema può avere vulnerabilità, come un buffer overflow, che possono essere scoperte tramite hacking etico.

Trojan: I trojan sono software nocivi che spesso vengono utilizzati dai penetration tester. Queste attività sfruttano vulnerabilità del sistema. Tuttavia, un sistema solido in termini di vulnerabilità può comunque essere compromesso tramite malware se l'utente esegue un file infetto.

Difesa contro il Malware: Microsoft Defender è un servizio che include antivirus, IDS (Intrusion Detection System) e firewall in un unico pacchetto. Questo tipo di software, sebbene chiuso e monolitico, rappresenta una difesa importante contro il malware.

Affrontare le Vulnerabilità: Lo scenario malware richiede un sistema anti-malware o un security center. Tuttavia, affrontare le vulnerabilità by design è una sfida diversa. Gli antivirus non rilevano sempre le vulnerabilità, e spesso non lo fanno. La soluzione è l'aggiornamento del software, che richiede una fiducia costante ed un atto di fede nel processo di sviluppo e distribuzione degli aggiornamenti.

9.1 Approfondimento sulle Vulnerabilità

Le vulnerabilità sono specifiche istanze di problemi di sicurezza su determinati sistemi.

Gestione delle Vulnerabilità:

- Repository Pubblici: Gli hacker etici pubblicano le vulnerabilità scoperte in repository pubblici;
- Vulnerability Response:: Prima della pubblicazione, le vulnerabilità de-

vono essere risolte, dando al venditore il tempo necessario per proteggere il sistema;

- **Aggiornamenti Software:** Risolvono le vulnerabilità note (n-day) presenti nei repository pubblici.

Vulnerabilità Zero Day: Una vulnerabilità zero day è una falla di sicurezza non ancora documentata. Gli attaccanti sfruttano queste vulnerabilità prima che siano conosciute e risolte. I penetration tester possono scoprire vulnerabilità zero day durante le loro attività offensive.

Esempio di Exploit: Zippando un file eseguibile con una password e allegandolo a una mail, è possibile evitare che il sistema di sicurezza lo riconosca immediatamente. Alcuni gestori di posta elettronica, come Google, possono comunque bloccare questi file per proteggere gli utenti.

9.2 Caratteristiche di un software nocivo

- **Carico:** Specifica violazione di sicurezza, è sempre presente.
- **Propagazione:** Meccanismo di trasmissione fra macchine, non sempre presente.
- **Pericolosità:**
 - **Massimo Carico:** Causa gravi danni al sistema infettato;
 - **Massima Propagazione:** Si diffonde rapidamente, infettando molti sistemi.

In conclusione, la pericolosità dipende dal contesto: un carico elevato può distruggere un sistema critico, mentre un'alta propagazione può causare epidemie su larga scala. Entrambi rappresentano seri rischi.

9.3 Tassonomia di un software nocivo

Si suddividono in software che necessitano un programma ospitante e quelli indipendenti.

Quelli che necessitano di un programma ospitante sono le Trapdoor, i Virus, Bomba Logica, Trojan. Quelli indipendenti sono Zombie (potenzialmente tutti i dispositivi IoT se vulnerabili possono essere trasformati in zombie) e Worm (Spyware), Si propagano solo virus, zombie e worm.

9.3.1 Trapdoor (Backdoor)

Un punto segreto di accesso a un programma che bypassa le procedure di sicurezza normalmente previste.

Una trapdoor oltrepassa le misure di sicurezza previste, rappresentando una vulnerabilità. Originariamente concepita per semplificare il beta-testing. La

conoscenza della trapdoor è necessaria per il suo utilizzo. Tipicamente implementata per comodità in passato, spesso per motivi di testing e basata sulla "security by obscurity".

9.3.2 Bomba logica

Frammento di codice all'interno di un programma non nocivo che "esplode" quando si verificano certe condizioni.

Esso è un particolare tipo di malware, si attiva e si manifesta al raggiungimento di una condizione specifica (ad esempio, un certo numero di utenti collegati simultaneamente a una piattaforma).

9.3.3 Trojan

Un programma utile o apparentemente utile che, durante l'esecuzione, compie violazioni di sicurezza.

Caratteristiche:

- Numerosi usi:
 - Rinominare defrag.exe come format.exe;
 - Un utente che esegue clear potrebbe ritrovarsi tutti i propri file con permessi modificati (r-rwxrwx).
- Metodo di Inserimento: spesso inserito nel sistema tramite trapdoor, backdoor o tecniche di ingegneria sociale;
- Funzionamento: presenta un'interfaccia apparentemente innocua, ma contiene un payload dannoso. Esempio: Creazione di un link simbolico tra ls e rm, ingannando l'utente a eseguire un comando dannoso.

9.3.4 Zombie

Programma nocivo che sfrutta una macchina remota già violata per lanciare nuovi attacchi, difficilmente riconducibili all'autore del malware.

Caratteristiche:

- Diffusione: estremamente diffusi oggi su internet;
- Utilizzo: tipicamente usati per attacchi di negazione del servizio (DoS).

Una macchina violata viene identificata come zombie, così come il software che utilizza la macchina per lanciare attacchi. La natura distribuita degli attacchi rende difficile tracciare l'entità del malware e il suo autore.

9.3.5 Worm

Programma nocivo che infetta macchine remote, ognuna delle quali infetta a sua volta altre macchine.

Caratteristiche:

- **Trasmissione:** Più dannoso di uno zombie perché si trasmette autonomamente da macchina a macchina. Non richiede intervento umano per propagarsi.

Un esempio molto famoso è il worm di Morris, che infettò 6000 macchine in poche ore. I worm sono pericolosi perché si diffondono rapidamente tra più dispositivi, aumentando esponenzialmente l'infezione.

9.3.6 Worm Morris

Il worm Morris, anche conosciuto come worm Internet, è stato uno dei primi worm informatici a diffondersi su una vasta rete di macchine Unix. Il suo creatore, Robert Morris, è stato multato e condannato a servizi civili.

Attacco e propagazione: Sfruttava tre vulnerabilità note in Unix per infettare macchine remote. Utilizzava tecniche come l'attacco **known-ciphertext** su file di password, **buffer overflow** su programmi come finger e backdoor nel programma sendmail. Una volta violata la macchina, il worm scaricava un codice C di 99 linee che si auto-compilava ed eseguiva, propagando così ulteriormente il worm.

Problemi e impatto:

- **Bug e Malfunzionamenti:** A causa di bug nel worm, alcune versioni non terminavano correttamente e si replicavano continuamente sulla stessa macchina;
- **Impatto sulle macchine:** Questo comportava un serio calo delle prestazioni delle macchine infettate;
- **Disconnessione dalla Rete:** Migliaia di macchine sono state disconnesse dalla rete per proteggersi o per proteggere altre macchine.

Il worm Morris è stato un esempio pionieristico di malware che ha dimostrato la vulnerabilità delle reti informatiche e l'importanza di implementare soluzioni di sicurezza robuste per prevenire tali attacchi.

Attacco Known-Ciphertext L'attacco known-ciphertext è un tipo di attacco crittografico che sfrutta informazioni note sul testo cifrato per ottenere informazioni sul testo in chiaro o sulla chiave di cifratura.

Descrizione dell'Attacco:

- **Vulnerabilità nel File di Password:** Sebbene il file di password sia criptato, le sue tre colonne (contenenti nomi utente, hash delle password e altri dati) sono leggibili da tutti; questo è dovuto a un errore

nell'implementazione della politica di sicurezza, che rende visibili informazioni sensibili.

Metodi Utilizzati da Morris: Morris ha utilizzato un attacco dizionario con una lista di parole note. Prova a codificare possibili candidati, partendo dai nomi utente e loro permutazioni. Se non ha successo, utilizza una lista di candidati statistici. Se ancora senza successo, prova tutte le parole nella directory locale /local. Se riesce ad ottenere una corrispondenza, può effettuare il login su una macchina remota per eseguire ulteriori azioni.

Questo tipo di attacco sfrutta la debolezza nella gestione dei file di password, permettendo agli attaccanti di ottenere accesso non autorizzato alle risorse di sistema. L'attacco di Morris è stato uno dei primi esempi di utilizzo di questa tecnica per diffondere malware e compromettere sistemi Unix.

Buffer Overflow mediante finger Il buffer overflow è una vulnerabilità che si verifica quando un programma scrive dati oltre i limiti di un buffer allocato in memoria, causando la sovrascrittura di dati adiacenti o addirittura del puntatore di ritorno della funzione.

Scenario di attacco:

1. Il worm lanciava il comando finger, e il servizio fingerd rispondeva dalla macchina remota;
2. La vulnerabilità si sfruttava tramite un buffer overflow nel buffer di input di fingerd, che andava oltre il suo limite e sovrascriveva l'indirizzo di ritorno;
3. Prima di terminare, fingerd eseguiva la shellcode caricata nel buffer, la quale scaricava e eseguiva le 99 linee di codice del worm.

Questa vulnerabilità permetteva al worm di ottenere l'esecuzione remota di codice sulla macchina bersaglio, aprendo la porta per l'infezione e la propagazione del malware.

Backdoor in Sendmail Il backdoor in Sendmail è una falla di sicurezza che permetteva di utilizzare Sendmail come un interprete di comandi, aprendo la porta per l'esecuzione remota di codice.

Scenario d'attacco:

- Sendmail è un demone di sistema eseguito in background, in attesa di indirizzi email da gestire;
- Il backdoor consisteva nel far passare Sendmail in "modalità di debug", in cui non solo gestiva le email ma eseguiva anche comandi arbitrari.

Robert Morris sfruttò questa trapdoor presente in Sendmail su macchine remote. Utilizzando questa vulnerabilità, Morris era in grado di inviare comandi al Sendmail remoto, incluso il comando per scaricare e eseguire le 99 linee di

codice del suo worm.

Questo backdoor ha permesso a Morris di ottenere un accesso non autorizzato e remoto alle macchine bersaglio, facilitando la propagazione del suo worm e causando un impatto significativo sulla sicurezza e sulla stabilità dei sistemi interessati.

9.3.7 Virus

Un virus è un programma nocivo che infetta altri programmi non nocivi, sfruttandoli per propagarsi.

La modalità di violazione è l'**aggiunta di codice indesiderato**: Tipicamente, un virus aggiunge pezzi di codice indesiderato ai programmi ospiti, compromettendoli.

Ci sono due tipologie di Virus:

1. Virus per Applicativi o per Dati;
2. Virus per il Settore di Boot.

Caratteristiche:

- **Difficoltà di Individuazione:** I virus possono essere difficili da individuare, specialmente se sono polimorfi (cambiano forma) o criptati (nascondono le proprie tracce);
- **Modifiche dannose:** Modificando casualmente i bit di un'eseguibile, un virus può comprometterlo, rendendolo non funzionante.

Per identificare e riconoscere i virus si utilizzano due modi:

- **Firma del virus:** Gli antivirus utilizzano le firme dei virus per riconoscerli. Queste firme devono essere aggiornate regolarmente per rilevare le nuove varianti;
- **Differenza dalla Firma Digitale:** La firma del virus è diversa dalla firma digitale e viene utilizzata esclusivamente per il riconoscimento dei virus.

I virus possono essere ovunque e sono in grado di adattarsi, cambiando forma o firma per eludere la rilevazione.

Macrovirus I macrovirus sono virus scritti come macro di un'altra applicazione, spesso utilizzati per infettare documenti di Microsoft Office.

Caratteristiche:

- **Utilizzo delle Macro in MS Office:** L'uso delle macro in MS Office può essere una vulnerabilità, in quanto le macro possono essere utilizzate per eseguire codice dannoso;

- **Livelli di Eseguitività delle Macro:** Le macro possono essere configurate per essere eseguite all'apertura del documento. Inizialmente, i macrovirus possono infettare senza dare sintomi evidenti.

Le macro di Office hanno aperto la porta a una nuova categoria di virus che sfruttano le macro per propagarsi. I filtri di sicurezza possono presentare vulnerabilità che gli attaccanti possono sfruttare per nascondere eseguibili maligni all'interno di documenti infetti.

9.4 Distinzione delicata del SW nocivo

	Tipo	Caratteristiche essenziali
Non replic.	Trapdoor	Concede accesso non-autorizzato
	Bomba logica	Si attiva solo su specifiche condizioni
	Cavallo di troia	Ha funzionalità illecite inattese
Replicando	Virus	Attacca un programma e si propaga attraverso qualunque mezzo, anche fisico
	Zombie	Usa macchina già violata per attaccare
	Worm	Viola macchine "ricorsivamente" attraverso la rete

9.5 Struttura di un Semplice Virus

La firma del Virus è una stringa usata per identificare il virus all'interno dei file, di solito si trova all'inizio del file bersaglio.

Funzionamento:

1. Riconoscimento della Firma:
 - Il virus si riconosce tramite la sua firma.
 - Se la firma è presente nel file target (ad esempio, nella prima riga), il file viene considerato già infetto e viene saltato.
 - Altrimenti, il virus procede con l'infezione.
2. Infettare il file: Il virus infetta un file "pulito" aggiungendo la sua firma e il proprio codice al suo interno;
3. Esecuzione del Danno Aggiuntivo: Dopo l'infezione, il virus può eseguire ulteriori danni al sistema, come attivare una bomba logica;
4. Restituire il controllo: Infine, il virus restituisce il controllo al programma ospitante, permettendo al file infetto di funzionare come previsto, ma con il codice dannoso del virus aggiunto.

9.6 Struttura ideale di un Antivirus

Scansione dei File: L'antivirus dovrebbe scansionare i file uno dopo l'altro, controllando la firma dei virus presenti nelle prime righe dei file. Nel caso dei virus polimorfi, si potrebbero utilizzare pattern noti per cercare firme in diverse parti del file.

Automazione della Scansione: Per rendere l'operazione meno onerosa per il sistema, l'antivirus dovrebbe automatizzare la scansione, decidendo autonomamente la periodicità e la profondità della scansione. Attualmente, molti antivirus sono residenti in memoria e utilizzano le capacità computazionali in modo efficiente. **Approccio alle Generazioni di virus:**

- **Seconda Generazione:** Utilizza euristiche e strategie per riconoscere i virus;
- **Terza Generazione:** Ricerca azioni illecite all'interno dei file;
- **Quarta Generazione:** Utilizza un insieme di tecniche di controllo di accesso, inclusi sistemi di prevenzione delle intrusioni (IPS).

Rilevamento dei File: L'antivirus può percepire l'arrivo di un nuovo file tramite il cambiamento delle dimensioni dei file presenti nel sistema, dovuto all'inserimento di firme al loro interno. Questo può attivare la scansione dei nuovi file per rilevare eventuali minacce.

Tecniche avanzate: usare un **controllore di esecuzione**, che gestisce contemporaneamente:

- **Emulatore di CPU:** macchina astratta che verifica le istruzioni di file eseguibili invece di farle eseguire direttamente alla CPU;
- **Controllore di firme:** modulo che scandisce i potenziali obiettivi alla ricerca di firme di virus; firme note mantenute in un DB da aggiornare.

9.7 Virus con Funzionalità di Compressione

Funzionamento del virus:

1. Compressione del Nuovo File: Il virus comprime il nuovo file da infettare;
2. Inserimento del Virus: Appendendo il proprio codice, il virus infetto viene aggiunto all'inizio del file compresso;
3. Decompressione del File Ospite: Il virus decomprime il resto del file ospite dopo il suo codice;
4. Esecuzione del File: Il file viene eseguito con il virus attivo.

Per rilevare un virus, si potrebbero guardare le dimensioni dei File: il virus potrebbe essere rilevato controllando le dimensioni del file, tuttavia, se il virus comprime il file in modo che la dimensione del file compresso più il virus sia

simile a quella del file originale, questa strategia di rilevamento potrebbe non essere efficace. Tuttavia, è necessario sviluppare nuove strategie di difesa che vadano oltre il semplice controllo delle dimensioni dei file, ad esempio:

- Analisi del comportamento del file per rilevare azioni sospette;
- Analisi delle firme del virus all'interno del file compresso;
- Utilizzo di tecniche di machine learning per identificare pattern di comportamento maligno;
- Implementazione di controlli di integrità per verificare se il file è stato alterato;

9.7.1 Esempio di Virus: Brain

Il virus Brain attacca sistemi Microsoft, rinominando il volume del disco in "BRAIN" e in alcune versioni cancellando frammenti del disco rigido. Si propaga naturalmente.

Carico del virus: Il virus viene caricato in memoria alta, e esegue una chiamata di sistema per resettare il limite di memoria alta al di sotto dell'area che lo contiene. Modifica il vettore degli interrupt in modo che l'indirizzo della procedura per gestire le letture da disco punti al proprio codice. In questa prima versione, non causa danni.

Gestendo le letture, il virus controlla se i byte 5° e 6° dei dati letti contengono la sua firma (valore esadecimale 1234):

- Se la firma non è presente, infetta 6 settori casuali;
- Se la firma è presente, si propaga con i dati.

Carico maggiore del Virus: Marca i settori infetti come "danneggiati" per impedire al sistema operativo di utilizzarli. Continua ad infettare settori finché ce ne sono di non infetti sul disco, riducendo lo spazio disponibile sul disco continuamente.

9.8 Rimozione di Software Nocivo

- **Antivirus: metodo comune ed evolutivo:** L'antivirus è il metodo più comune e in continua evoluzione per rimuovere software nocivo. Tuttavia, è limitato dalla necessità di aggiornamenti costanti per rilevare le nuove minacce;
- **Prototipo IBM degli anni '90:** Sviluppato negli anni '90 da IBM, il prototipo dei sistemi immuni sfrutta tecniche simili a quelle degli antivirus. Si basa su un'intera struttura distribuita di prevenzione per rilevare e rimuovere il software nocivo.
Ogni macchina del sistema usa antivirus delle prime tre generazioni. Una macchina per l'analisi di virus deve essere sempre on-line. Essa ha tutta

la responsabilità di scovare e rimuovere il virus, poi informare i client. Continuamente aggiornata coi campioni provenienti da tutte le macchine che partecipano all'infrastruttura.

- **Software Sentinella:** integrato con il sistema operativo, Il software sentinella è integrato direttamente nel sistema operativo. Blocca l'esecuzione di codice nocivo e protegge il sistema da potenziali minacce (come tentativi di formattare il disco, modifica di impostazioni critiche del sistema o inizio di comunicazioni distribuite).

9.8.1 Ruolo delle Difese

L'antivirus rappresenta solo una delle difese disponibili contro i virus.

Gli "Immune System" e i software sentinella sono altre aree importanti della difesa, lavorando in sinergia per proteggere i sistemi da software nocivi.

Inoltre, i sistemi operativi stessi spesso incorporano meccanismi di sicurezza per controllare il codice eseguito e prevenire l'esecuzione di software nocivo.

10 WWW Security Protocols

SSL e TLS costituiscono il protocollo fondamentale di rete esposto tramite HTTP(S).

Il WWW è uno specifico servizio per il web, questi protocolli si possono trovare dietro altri tipi di servizi S/MINE 3D-Secure ed SET. Il pagamento è un obiettivo funzionale sensibile dal punto di vista della sicurezza. 3D-Secure mette in sicurezza la transazione tra Customer e Vendor, il vendor invia la merce in modo (anche) digitale ed il pagamento arriverà all'interno dello stesso tunnel in quanto la voglio mettere in sicurezza. Dietro il 3D-Secure ci sta qualcosa di più rispetto a ciò che sta dietro TLS/SSL che costruiscono un tunnel sicuro tra 2 agenti. SSL è omologo ad IPSEC solo che sta ad un altro livello. Per realizzare un tunnel sicuro, bisogna sapere che TLS prevede segretezza, autenticazione ed integrità.

Ma come mettere in sicurezza un pagamento digitale? Mettere in sicurezza il pagamento vuol dire che il vendor prima di elargire il servizio verifica il pagamento. Se non fa questo non è il tunnel di connessione sicuro ad essere il problema ma il customer stesso.

Il sistema di pagamento annota le transazioni effettuate Tra Client e Vendor c'è però un terzo agente che è la banca. Non è possibile quindi costruire un semplice tunnel tra Client e Vendor.

Come strumenti di pagamento usiamo le carte di debito/credito che permetta al vendor di ricevere il denaro. Come strumento di annotazione del pagamento si usa il numero della carta di credito, il customer trasferisce questo numero al gestore del pagamento. Inizialmente non era così il numero veniva trasmesso direttamente al vendor senza alcun controllo aggiuntivo (batch processing) che successivamente tramite i numeri di carta di credito acquisiti andrà in banca a richiedere i soldi.

Facilmente attaccabile, al customer bastava inviare un numero di carta di credito fasullo ma ben formato e riceveva comunque il servizio richiesto.

Il pagamento digitale per essere sicuro deve avere una verifica sulla sensatezza del numero della carta di credito acquisito.

3D-SECURE ed SET sono più complessi di TLS/SSL perchè mettono in sicurezza l'intero pagamento a differenza di TLS/SSL.

SSL nasce come un prodotto proprietario TLS è la sua versione open-source, usa strumenti standard contrariamente a quanto faceva SSL.

10.1 HTTPS

Se faccio scansione dei pacchetti mi accorgo dell'utilizzo della porta 443 a differenza della porta 80 utilizzata da HTTP, cifra contenute payload form cookie ed header HTTP.

Lo strumento per la gestione per l'autenticazione sul web sono i cookie che gestiscono l'ID di sessione.

Un particolare header HTTP è HSTS, introdotto in una versione recente di TLS per contrastare un particolare tipo di attacco.

10.2 Attacco di stripping su SSL

Attacco di downgrade delle scelte di sicurezza, ad esempio l'utente richiede HTTPS l'attaccante riesce a strappare ed indirizzare l'utente su HTTP (per ragioni di legacy il sito potrebbe non essere compatibile)

Un altro esempio è l'utilizzo di un dominio squatted o fasullo e richiede possesso del DNS, requisiti dell'attacco più importanti e meno realistici.

Lo stripping ad oggi è più difficile i browser implementano sempre più il supporto ad HSTS che è una strategia per risolvere questo problema.

Per mitigare il rischio della seconda tipologia di SSL stripping le aziende acquistano domini simili al proprio, attacco di natura socio tecnologica, se l'utente fosse attento si accorgerebbe che in realtà sta accedendo ad un sito errato.

10.3 SSL vs TLS

SSL è dei primi anni 90, netscape era un browser che implementava SSL, nel 95 nasce SSL 3 ed il suo RFC nasce molti anni dopo, l'RFC è una descrizione specifica di un prodotto open source. SSL non nasceva come un prodotto aperto ma come un prodotto proprietario, TLS nel 99 costruisce un protocollo per standardizzare SSL. **TLS è quindi la versione standardizzata di SSL.**

10.4 SSL Protocol Suite

SSL non è un solo protocollo ma un insieme di protocolli:

- Protocollo di handshake;
- Protocollo per cambiare le specifiche crittografiche;

- Protocollo di Alert;
- Protocollo di Record.

Questi protocolli non stanno tutti allo stesso livello, dietro questi 4 protocolli esiste un protocollo che è il vero protocollo di sicurezza che distribuisce la chiave condivisa in IPSEC esiste IKE in SSL esiste l'handshake protocol.

Il record protocol è quello che usa la chiave e costruisce il substrato di sicurezza per tutte quelle applicazioni che usano HTTPS e costruisce il tunnel crittografico. Utilizza il materiale crittografico per garantire un livello di sicurezza per l'applicazione.

10.4.1 Record Protocol

I dati dell'applicazione vengono frammentati, vengono compressi e si aggiunge il MAC (Message Authentication Code) primitiva crittografica che usa chiave simmetrica per ottenere autenticazione ed integrità $\Rightarrow m, h(m, k)$ firma digitale se la chiave è simmetrica. Una volta aggiunto il MAC si cifra l'intero frammento, infine ad ogni frammento si aggiunge un header che contiene informazioni come:

- Content Type: A quale protocollo questo frammento si riferisce;
- Major Version: Numero di versione del protocollo;
- Minor version;
- Compressed Length: Lunghezza del frammento compresso, aiuta per la futura decompressione.

10.4.2 MAC di SSL

L'indentazione evidenzia che si tratta di un hash annidato, il motivo è che in qualche modo si pensava di aumentare la difficoltà di inversione dell'hash. Non esistono tabelle rainbow per hash di hash.

- MAC_Write_Secret: Nome SSL per la chiave di sessione
- SSLCompressed.fragment: Input per il MAC
- SSLCompressed.type: Tipo di frammento SSL
- pad_1, pad_2: padding, serve a garantire l'applicabilità della funzione hash

Si concatena nuovamente la chiave si inserisce del padding e si applica nuovamente l'hash.

10.5 SSL CCSP

Messaggio di un singolo byte, negoziamo delle preferenze crittografiche con il protocollo di handshake e con questo "schiocco di dita" le mettiamo in atto, ci sincronizziamo per mezzo di questo protocollo. Lo stato corrente ha un nome e si chiama cypher suite, la suite di scelte crittografiche.

Le scelte crittografiche sono tante, ma di chiave di sessione ne abbiamo più di una, una per il MAC ed una per l'encryption, il protocollo di handshake distribuisce sì la chiave di sessione ma più precisamente stabilisce la cypher suite.

10.6 SSL AP

Protocollo di alert, perchè metterlo in sicurezza? Gli alert se modificati possono essere sfruttati dagli hacker.

Composto da 2 byte. Un alert di tipo fatal chiude la connessione: unexpected_message, bad_record_mac, decompression_failure. Un alert di tipo warning riguarda la validità o la scadenza dei certificati: unsupported_cert, cert_revoked, cert_expired.

In caso di alert fatal, il flag is_resumable viene settato a 0 (non è possibile ripristinare la connessione SSL).

Connessione nuova e connessione ripristinabile: Una nuova connessione deve ripetere tutto il protocollo da 0 ed è pesante a livello computazionale, il ripristino si fa per motivi di efficienza ed è cruciale per ridurre la latenza.

10.7 SSL Handshake Protocol

Questo protocollo è il più complicato, è il protocollo di sicurezza ausiliario. Esso garantisce le 3 proprietà di sicurezza fondamentali (confidenzialità con la crittografia simmetrica, l'autenticazione con la crittografia asimmetrica, l'integrità con l'hashing), l'obiettivo è stabilire un master secret che otterrò con Diffie-Hellman. Il formato è:

- Tipo: numero che identifica 1 dei 10 messaggi;
- Lunghezza;
- Contenuto;

10.8 Messaggi SSL

I messaggi hello sono tra i 10 i più importanti, all'interno dell'handshake posso distribuire le chiavi o tramite diffie hellman o tramite RSA:

- hello_request: Messaggio dal contenuto nullo, è facoltativo (S->C)
- client_hello: Va dal client al server, contiene versione, random, id di sessione, cipher suite, metodo di compressione

- `server_hello`: Dal server al client, stessi parametri
- `certificate`: catena di certificati X 509, vogliamo che il nostro browser (client) autentichi la nostra url tramite cifratura asimmetrica
- `server_key_exchange`: Il server manda il primo messaggio di scambio di chiave di sessione il contenuto del messaggio è composto da parametri e firma del messaggio
- `certificate_request`: Messaggio opzionale che il server manda al client per autenticarsi
- `server_done`: Il server ha finito
- `certificate_verify`: Il client può verificare la firma
- `client_key_exchange`: Secondo parametro di Diffie Hellman
- `finished`: valore di hash

10.8.1 Fase 1: Stabilimento della connessione

Fase cruciale del protocollo, si inviano il numero del protocollo, l'ID della sessione, cipher suite, metodo di compressione, e numeri random meglio conosciute come nonce. Il client invia questi 5 campi:

- `Version`: La più alta che riesce a supportare
- `Random`: numero random che previene attacchi di replay `client_hello_random`
- `ID di sessione`: 0 in caso di sessione nuova o altro numero per il ripristino della sessione, il client memorizza le sessioni aperte ed invia al server l'ID per ripristinarla
- `Cipher Suite`: Lista di coppie del tipo Cipher Spec (che tipo di encryption o hash uso) ed algoritmo di key exchange
- `Compression Method`: Metodi di compressione

Il server invia questi 5 campi:

- `Version`: La più alta che lui supporta e la più bassa tra quella ricevuta
- `Random`: numero random con nome `server_hello_random`
- `Session ID`: Se il client non ha inviato 0, il server verifica l'ID della sessione ricevuto e ne verifica la presenza reinoltrandolo o in caso contrario generandone uno nuovo terminando il protocollo (non si parla quindi di ripristino)
- `Cipher Suite`
- `Compression Method`

Client Hello Parameters Il numero di versione è significativo per prevenire attacchi di downgrade, la cipher suite è il sistema per la distribuzione della chiave di sessione e le specifiche cipher quali la scelta del crittosistema, la scelta della funzione hash

Chi riceve la richiesta (il server) può scegliere di fare abort oppure sceglie la versione più bassa tra quella ricevuta e la più alta che lui supporta. Se il client manda un numero di sessione diverso da 0 il server può decidere se ricopiarlo ripristinando la sessione o inviare un nuovo numero di sessione creandone una nuova. Se il client invia 0 allora il server genera un nuovo id di sessione.

Componenti Cipher Suite Fatta da Cipher Spec e Key Exchange Algorithm (algoritmo o protocollo di scambio delle chiavi). **Cipher Spec:**

1. Algoritmo di encryption
2. Algoritmo usato per il MAC che include una funzione hash
3. Cipher Type: stream o a blocchi (bit a bit o chunk)
4. isExportable: flag
5. Dimensione dell'hash: 16 byte per MD5 o 20 byte per SHA1
6. Key Material: Sequenza di byte per la generazione di altre chiavi, serve da generatore per nuovo materiale crittografico
7. Initialization Vector: Vettore di inizializzazione per il crittosistema

Key Exchange Algorithm:

1. Scambio delle chiavi RSA
2. Versione tradizionale o anonima di Diffie-Hellmann
3. Diffie-Hellmann Effimero: versione con il fix dove i parametri pubblici sono autenticati con la firma digitale
4. Diffie-Hellman con fix: i parametri pubblici sono bloccati e computazionalmente derivati da certificati di chiave pubblica
5. Fortezza: deprecato

Esempio di Cipher Suite (Vedi Slide 24 SSLTLS): `SSL_RSA_EXPORT_WITH_RC4_40_MD5:RSA_EXPORT`

SSL: Protocollo.

RSA_EXPORT: scambio di chiavi e autenticazione, dove RSA è l'algoritmo asimmetrico per l'autenticazione, e EXPORT sono le leggi di esportazione degli stati uniti degli anni '90.

RC4_40: algoritmo di cifratura per il traffico dati.

MD5: algoritmo di Hashing utilizzato.

10.8.2 Fase 2: Autenticazione del server e scambio delle chiavi

- Certificati;
- Scambio delle chiavi server;
- Server hello done;
- Le firme digitali trasporta le nonce ha freshness per evitare attacchi di replay, DSS usa Sha1 mentre le firme con RSA usano una concatenazione di MD5 e SHA1.

10.8.3 Fase 3: Autenticazione del client e scambio delle chiavi

Il client invia i certificati se richiesti, si fa una verifica dei certificati tramite digest che garantisce integrità ed autenticazione garantito dalla firma digitale. La chiave di sessione si chiama pre-master-secret si computano segreti, il primo segreto sarà il seme per computare il successivo. Il master secret è determinato computazionalmente dal pre-master-secret.

Sullo stesso stile dell'OTP, ma non è uguale, si fa un calcolo offline per ottenere lo stesso materiale crittografico, con un calcolo semplice si generano chiavi sicure quanto la vecchia ed è quindi prevedibile quanto la prima. Si usano quindi i PRF.

PRF: Pseudo Random Functiuon: funzione che si basa sugli hash e genera chiavi pseudo randomiche.

10.8.4 Fase 4: Finish

- change_cipher_spec: byte 1 schiocco di dita, un segnale che indica che da questo momento è possibile utilizzare la suite negoziata in questa sessione. Il tutto è messo in sicurezza perchè sopra il record protocol.
- Messaggi finished: I due attori si scambiano un intero digest dove si scambiano le specifiche concordate per capire se il tutto è andato a buon fine.

10.9 Master Secret

Si usa MD5 che prende come parametri il pre-master-secret concatenato con un'altra applicazione interna di SHA i cui parametri sono il pre-master-secret i due numeri (le 2 nonce) random e la stringa A. Ottengo altri nuovi 16 byte concatenando il tutto prima modificando solo la stringa da 'A' in 'BB' e poi in 'CCC'. Lo scopo è quello di prendere il primo segreto condiviso ed ottenerne di più. Il tutto è grande 48 byte, il pre-master-secret è il seme, i valori random e le stringhe non sono altro che sale. *N.b.: è una concatenazione di 3 applicazioni di MD5.*

Master Secret

```
master_secret = MD5(pre_master_secret || SHA('A' ||
    pre_master_secret || ClientHello.random ||
    ServerHello.random)) ||
MD5(pre_master_secret || SHA('BB' ||
    pre_master_secret || ClientHello.random ||
    ServerHello.random)) ||
MD5(pre_master_secret || SHA('CCC' ||
    pre_master_secret || ClientHello.random ||
    ServerHello.random))
```

Figure 23: Master key

10.10 Key Blocks

Stesso procedimento di quanto visto in precedenza modificando solo il parametro del pre-master-secret con il master-secret si usa il master-secret come seme, condito con le nonce per freshness insieme al meccanismo delle stringhe continuando all'infinito o fino a generare il numero di bit di cui ho bisogno.

10.11 Ridurre la latenza di rete

Concetto di ripristino, si usa per motivi di efficienza. Una sessione è una relazione tra client e server creata dal protocollo che definisce la cipher suite, ripristinarla vuol dire aprire una nuova sessione utilizzando la cipher suite della sessione di riferimento.

L'idea è quello di salvare lo **stato della sessione** per ripristinarlo in una nuova sessione, salvo il master-secret, nella fase 1 si generano nuove nonce che utilizzo per generare nuovi key block.

1. Id della sessione
2. Certificato di peer
3. Metodo di compressione
4. Specifica di cipher
5. Il master secret con la sua specifica crittografica completa
6. isResumable: flag

Ripristinare la sessione vuol dire quindi prendere il master secret e ripristinarlo con nuove nonce.

10.12 Transport Layer Security (TLS)

TLS non è altro che un tentativo di standardizzare SSL che ha oramai 25 anni, SSL era nato come iniziativa commerciale privata. Il MAC viene fatto tramite uno standard chiamato HMAC che si evolve di suo parallelamente.

10.13 MAC

L'HMAC ha due parametri ed anche qui esiste un annidamento di funzioni hash ha come parametro il messaggio, la chiave e più precisamente prende la chiave allungata con degli 0 a sinistra che la portano alla lunghezza del blocco della funzione hash. Tramite or esclusivo si concatena il tutto a due padding di bit. Hashing parametrico la chiave va usata in un certo modo per ottenere la block size attesa dalla funzione hash scelta.

MAC

- By standard HMAC, RFC2104

$$\text{HMAC}_K(M) = H[(K^+ \oplus \text{opad}) \parallel H[(K^+ \oplus \text{ipad}) \parallel M]]$$

where

H = embedded hash function (for **TLS**, either MD5 or SHA-1)
 M = message input to HMAC
 K^+ = secret key padded with zeros on the left so that the result is equal to the block length of the hash code (for MD5 and SHA-1, block length = 512 bits)
 ipad = 00110110 (36 in hexadecimal) repeated 64 times (512 bits)
 opad = 01011100 (5C in hexadecimal) repeated 64 times (512 bits)

- Takes M as a seed and K as a secret
- Applies a chosen hash function H

Giampaolo Bella (giamp@dmi.unict.it) WWW Security Protocols 35 / 43

Figure 24: HMAC

10.14 P_hash

Versione grafica del key block, l'applicazione più interna prende un seme insieme ad un segreto e si crea un HMAC che a sua volta concateno al seed diventando un altro HMAC. Somiglia al calcolo del key block in TLS.

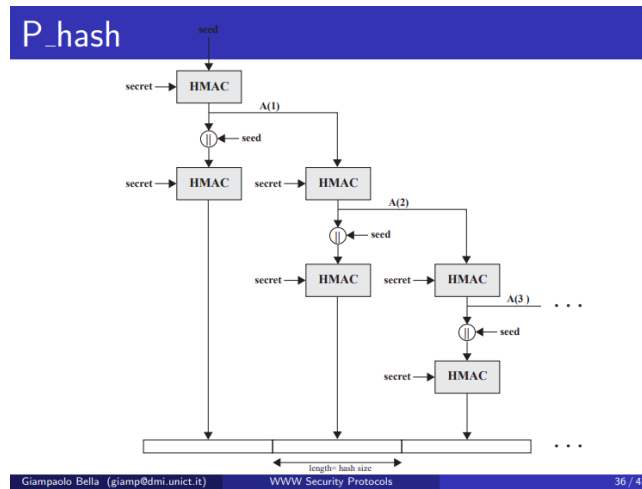


Figure 25: P_{hash}

10.15 Pseudo Random Function (PRF)

La PRF invoca una P_MD5 che è la P_HASH fatta con MD5 che abbia come seme i valori random e come segreto ha il pre-master-secret se vogliamo calcolare il master-secret o il master-secret se vogliamo calcolare nuovo materiale crittografico.

Il segreto viene splittato in S1 ed S2, metà lo do in pasto alla P_MD5 e metà alla P_SHA_1

L'obiettivo primario è generare stringhe di bit nuove in modo rapido per garantire la freshness, si parte da un segreto per arricchire un seme che sono numeri random.

Pseudo Random Function

$$\text{PRF}(\text{secret}, \text{label}, \text{seed}) = \text{P_MD5}(\text{S1}, \text{label} \parallel \text{seed}) \oplus \text{P_SHA-1}(\text{S2}, \text{label} \parallel \text{seed})$$

- Parameter secret is split up into S1 and S2
- Output length controlled by reiterations in HMAC
- **seed** is concatenation of both Random values
- **secret** is PMS to output MS or MS for key block
- **label** is “master secret” to output MS or “key block” to output key block

Giampaolo Bella (giamp@dm.unict.it)
WWW Security Protocols
37 / 43

Figure 26: PRF

10.16 Evoluzione di TLS

TLS 1.0 ha un RFC del 1999 TLS 1.1 nasce il concetto di estensione TLS nel 2006 TLS 1.2 è del 2008 ed il suo RFC spiega cosa sono le estensioni che a loro volta hanno un RFC separato.

Le estensioni sono un allungamento dei messaggi hello aggiungendo altri parametri.

Nel 2004 si scopre che MD5 era debole, nel 2008 si sfruttano le debolezze di MD5, l’RFC della versione TLS 1.2 usa le estensioni e specifica quali funzioni hash accettare e quale algoritmo di firma usare.

MD5 resta ancora in uso ed il suo uso reiterato fa ancora vittime di attacchi informatici.

In TLS 1.2 con il suo RFC la combinazione dei 2 hash è stata rimpiazzata con un singolo hash, la combinazione nella PRF dell’uso di MD5-SHA1 viene rimpiazzata con una PRF cipher-suite-specified, tutte le cipher suite di TLS 1.2 usano SHA256.

11 Domande d’esame

1. Steganografia, a cosa serve?;
2. Sicurezza, obiettivo non funzionale;
3. Come mettere in sicurezza una infrastruttura di rete dipartimentale (Protezione fisica delle porte NAC Network Access Control);
4. Java linguaggio memory safe (JVM), C non è un linguaggio memory safe;
5. Attacco: differenze tra policy errata e policy con dimenticanze;

6. Perché la sicurezza impatta i sistemi operativi: fare degli esempi;
7. Come progettare il cambio password di un sistema operativo moderno (amministratore di sistema? Meglio di no);
8. Abuso di sistema VS Violazione di sistema;
9. Misure di mitigazione di un trojan (Impedire l'eseguibilità di un file nella cartella download, o meglio non darla di default);
10. Criteri di robustezza delle password (limiti di natura socio-tecnologica);
11. Robustezza: Quanto una misura sia forte contro attacchi mirati che intendono rompere e quindi violare una proprietà di sicurezza;
12. Differenza tra sicurezza e privacy;
13. Sicurezza e segretezza differenze;
14. Nesso tra autenticazione ed anonimato (se si è autenticati certamente non si è anonimi) (esempio forum solo link);
15. Differenza tra autenticazione e non ripudio;
16. Firma a cavallo;
17. Randomizzazione ed identificazione dei compiti scritti;
18. Per rispondere alle domande su WATA2 è necessario scrivere il codice presente nell'immagine;
19. Sul token quante firme ci sono? 2 una da parte dello studente e d una per convalidare l'integrità del compito da parte dello studente;
20. Perché la firma del docente viene inserita nella fase di preparazione? Per evitare che il docente veda l'id del compito già assegnato ad un candidato specifico ed evitare di de-anonimizzarlo;
21. Quando finisce l'esame? Fino al momento in cui il voto non viene verbalizzato e registrato in un sistema affidabile e riconosciuto;
22. Perché l'esame è pubblico? Misura di sicurezza a favore del candidato;
23. Come implementare l'autorizzazione: Autenticazione dell'utente, policy di sicurezza e tecniche che applicano una policy alle utenze;
24. Differenza tra contraddizione e dilemma;
25. (Domanda preferita) Quali sono le inconsistenze di una politica, quali sono i problemi dietro le policy?;
26. ACM, ACL, CaL;

27. Come caratterizzare e classificare gli attaccanti;
28. Problemi delle password (Guessing, Snooping, Spoofing, Sniffing);
29. Iter di attacco su una password;
30. Linee guida NIST del 2004 e del 2016;
31. Limiti delle captcha: per le captcha tradizionali alfanumeriche esiste un algoritmo di Google che riesce a violarlo;
32. Più sicuro un PIN di lunghezza 4 con controllo a soglia di numerosità 3, o password molto robusta con controllo a soglia di numerosità 3 (Risposta: è pressoché uguale probabilità bassissima di 3/10000 per il PIN);
33. PRF;
34. Limiti della cifratura: Un messaggio cifrato non è nascosto, si vede ed è un problema significativo in termini di privacy (informazione utile in termini di analisi forense e spionaggio);
35. Cos'è un crittosistema?;
36. Come scambiare in modo sicuro le chiavi crittografiche;
37. Definizione formale di crittografia simmetrica (usare le formule matematiche di pagina 99);
38. Password e chiave a lungo termine requisiti di segretezza e confidenzialità;
39. Generazione delle chiavi di cifratura;
40. Come ottenere segretezza con crittografia asimmetrica?;
41. Come ottenere autenticazione con crittografia asimmetrica?;
42. Weakness della cifratura asimmetrica (certificazione);
43. Proprietario di una chiave pubblica: Chi ha l'altra metà privata della chiave;
44. Definizione di Crittosistema sicuro;
45. Differenza tra hashing ed encryption;
46. Brute forcing sull'hash per invertire l'hash;
47. Utilizzo di salt e pepper nella memorizzazione delle password;
48. Funzioni hash proteggono la rappresentazione in memoria di massa delle password, come funziona il controllo della password al login?;
49. Perché la freshness è importante e come si fa (usando le nonce);

50. Come usare il timestamp per ottenere garanzia della freshness;
51. Perché Dolev Yao non rompe la crittografia;
52. Differenza tra assunzione1 ed assunzione2 (slide 121 crittografia simmetrica);
53. Perché 4 assunzioni nel combinare sicurezza ed autenticazione? Sono l'unione dei 2 protocolli visti;
54. Scrivere un protocollo elementare basato su cifratura asimmetrica che garantisca segretezza ed autenticazione;
55. Non sottovalutare le assunzioni all'esame orale;
56. Schema base per l'integrità (abbinamento delle assunzioni che servono per l'autenticazione);
57. La firma grafometrica, vulnerabilità (alterabilità, falsificazioni);
58. Operazione insiemistica per derivare le assunzioni che permettono di combinare le proprietà di livello 1? (Le sommo);
59. Diffie Hellmann, protocollo che appare elementare ma che in realtà è molto complesso;
60. Primitiva crittografica (Encryption, Hashing, Firma Digitale);
61. Consorzi di banche dietro le CA;
62. Periodo di validità di un certificato (Motivi di business, il certificato si paga, motivi di sicurezza);
63. Disamina in termini di verosimiglianza o impatto delle 3 opzioni dei certificati self-issued;
64. Ipotesi di fix con funzione hash dello scenario di attacco (si mette l'hash nella nonce al passo 3);
65. Che vuol dire prudenza nel contesto del principio di disegno;
66. Quali sono i limiti e le critiche agli attacchi visti contro Needham-Schröder;
67. Quali sarebbero le prerogative della nonce? Se si rivede indietro qualcosa deve essere accaduto;
68. Perché il messaggio 5 di Woo-Lam è cifrato?;
69. Proprietà di autenticazione di Alice con Bob in Woo-Lam;
70. Freshness: attributo di sicurezza (autenticazione e confidenzialità);
71. Nesso tra autenticazione e replay attack;

72. Esempi legati al rilevamento statistico di intrusioni in caso di attività di login;
73. Differenza tra DoS ed intrusione (bloccare il sistema e sfruttare il sistema);
74. Numero di password failures è un tentativo di controllo a soglia non è adatto ad uno di tipo media e varianza;
75. Esempio di rilevamento statistico di intrusioni legato alle attività di shell di comando: Frequenza di esecuzione di un comando, attività di accesso ad un file quanto leggi e quanto scrivi;
76. Trojan malware;
77. Nesso tra attività offensiva e software nocivo;
78. Allegati nella mail, floppy disk, metodologie di distribuzione dei malware;
79. Bug VS Malware;
80. Differenza tra bug e software nocivo: intenzionalità punita dalla legge;
81. Esempificazione del Morris Worm;
82. Definizione di un virus;
83. Esempificazione del gioco di attacco e sicurezza riportato al caso dei virus e del rilevamento;
84. Differenza tra IPSEC e TLS/SSL (Stanno a livelli differenti);
85. Come realizzare un tunnel sicuro;
86. TLS va bene per i pagamenti online? Va bene solo per la creazione del tunnel sicuro ma il pagamento ha problemi di sicurezza più profondi che vengono gestiti da 3DSECURE e SET;
87. Differenza tra MAC e Firma Digitale;
88. Come è fatto il MAC di SSL e dimostrare che è coerente con lo schema generale di MAC $m, h(m, k)$;
89. Semplicità del protocollo CCSP in SSL, il messaggio è semplice perché è sicuro grazie al record protocol;
90. Diffie Hellman, lo utilizziamo ogni volta che ci serve scambiare una chiave o è necessario solo la prima volta?;
91. Chi sceglie il session ID? Scelta condivisa, il client propone un ID da ripristinare;
92. Tunnel sicuro cos'è?;

- 93. Perché il protocollo di alert dovrebbe essere messo in sicurezza?;
- 94. Nesso tra il messaggio certificate e la catena di certificati X509;
- 95. Cos'è un attacco di downgrade;
- 96. Protocollo di sicurezza definizione: algoritmo distribuito che usa una tecnica di sicurezza come la cifratura;
- 97. Cos'è la cipher spec: attenzionare tutti gli aspetti inclusi quelli crittografici;
- 98. Come calcolare il master secret (16 byte *3 i primi si ottengono per MD5, le altre 2 per concatenazione);
- 99. Quando ripristino la sessione le chiavi crittografiche cambiano? Il master secret non cambia ma tutto il resto delle chiavi di sessione si;
- 100. Formule standardizzate;
- 101. MD5 è debole, ce ne possiamo liberare? Solo in TLS 1.2;